

بازیابی اطلاعات

دانشگاه آزاد شیراز • دانشکده مهندسی کامپیوتر

ترم دوم • سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل دوم: The Term Vocabulary and Postings Lists

استاد درس

دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی)

امیرمحمد اسدجو

علی نیکوان

علی مجاهد

امیرحسین همتی

حمید نامجو

فهرست مطالب

2.1 Document delineation and character sequence decoding

2.1.1 Obtaining the character sequence in a document

2.1.2 Choosing a document unit

2.2 Determining the vocabulary of terms

2.2.1 Tokenization

2.2.2 Dropping common terms: stop words

2.2.3 Normalization (equivalence classing of terms)

2.2.4 Stemming and lemmatization

2.3 Faster postings list intersection via skip pointers

2.4 Positional postings and phrase queries

2.4.1 Biword indexes

2.4.2 Positional indexes

2.4.3 Combination schemes

2.5 Chapter 2 Summary

فصل ۲: از کاراکتر تا واژگان - چالش‌های یک سیستم جستجوی واقعی

 تصور کنید مدیر یک پلتفرم بزرگ فروشگاهی آنلاین به نام «GG-Kala» هستید

شما مسئول ساخت موتور جستجوی این پلتفرم هستید. «GG-Kala» همه‌چیز می‌فروشد: از لپ‌تاپ و گوشی‌های هوشمند گرفته تا لوازم خانگی هوشمند. داده‌های شما از همه‌جا می‌آیند: فایل‌های PDF، گزارش‌های داخلی در قالب Word، کامنت‌های کاربران در پایگاه داده، و مشخصات فنی محصولات در جداول. هدف شما، تبدیل این اقیانوس آشفتۀ داده‌ها به یک کتابخانۀ دیجیتال منسجم و قدرتمند است که کاربران بتوانند در کسری از ثانیه به هر اطلاعاتی که نیاز دارند برسند.

برای ساختن این موتور جستجو، باید از مراحل اصلی که در فصل قبل معرفی شد، عبور کنیم. این مراحل، پایه‌های علمی یک سیستم بازیابی اطلاعات مدرن هستند:

1. جمع‌آوری اسناد (Collect Documents)

اولین قدم، گردآوری تمام داده‌ها. در «GG-Kala»، این یعنی جمع‌آوری فایل‌های PDF از بخش‌های مختلف، استخراج گزارش‌های Word از سرورهای داخلی، و یکپارچه‌سازی کامنت‌های کاربران که در پایگاه داده پراکنده شده‌اند. در این مرحله، ما به محتوای اسناد کاری نداریم، فقط آن‌ها را از منابع مختلف جمع می‌کنیم.

2. توکن‌سازی (Tokenize Text)

حالا دیوار متنی بلند و بی‌نظم را داریم. وظیفه ما، تبدیل این دیوار به آجره‌های معنادار (توکن) است. «توکن» کوچک‌ترین واحد معنادار است؛ معمولاً یک واژه، یک عدد، یا یک علامت نگارشی. این فرآیند، شکستن متن به قطعات کوچک‌تر است تا بتوانیم روی آن‌ها کار کنیم.

3. پیش‌پردازش زبانی (Linguistic Preprocessing)

این مرحله، جادوی اصلی ماست. می‌خواهیم هم 'گوشی'، 'گوشی‌ها' و 'گوشی‌ام' را به یک ریشه واحد برسانیم تا سیستم ما بداند 'گوشی‌ها' و 'گوشی' به یک مفهوم اشاره دارند. این کار شامل نرمال‌سازی (مثل تبدیل همه به حروف کوچک)، حذف علائم نگارشی (مثل نقطه و ویرگول)، و ریشه‌یابی (Stemming) است تا واژه‌های مختلف به یک شکل استاندارد درآیند.

4. نمایه‌سازی (Indexing)

و بالاخره، با این آجره‌های تمیز، دیوار واژگان (Vocabulary) و Postings List‌های خود را می‌سازیم. این همان ساختار **فهرست معکوس** است که در فصل قبل با آن آشنا شدیم: دیکشنری از واژه‌ها و برای هر واژه، لیستی از اسنادی که آن واژه در آن‌ها وجود دارد.

اما در دنیای واقعی، «سند» چیست؟

در تئوری، تعریف یک «سند» ساده به نظر می‌رسد، اما در عمل با چالش‌های پیچیده‌ای روبرو می‌شویم. در «GG-Kala»، یک 'سند' می‌تواند یک فایل PDF پانصد صفحه‌ای باشد، یک ردیف در پایگاه داده، یا حتی یک کامنت کوتاه در یک پست وبلاگ باشد. این تنوع، تعریف ما از «سند» را به چالش می‌کشد.

یک روز، سیستم با یک مشکل عجیب روبرو شد: بخشی از کامنت‌های کاربران به زبان چینی نمایش داده می‌شد! 🔍 (مثل اتفاقی که برای سایت دیوار در تابستان 1404 افتاده بود!) کاربران شکایت داشتند و هیچ‌کس نمی‌دانست مشکل از چیست. مشکل از **رمزگذاری کاراکترها (Character Encoding)** بود. سیستم ما فرض می‌کرد همه‌چیز UTF-8 است، اما برخی داده‌های قدیمی با رمزگذاری دیگری (مثل Latin-1) ذخیره شده بودند. این مشکل به ما یادآوری کرد که قبل از پردازش متن، باید ابتدا زبان و رمزگذاری صحیح آن را تشخیص دهیم.

📝 چالش دوم: جداسازی داده‌های ساختاریافته

مشکل بعدی، فایل‌های گزارش فنی در قالب Word بودند. این فایل‌ها علاوه بر متن، شامل جداول، تصاویر و هدر و فوتر هستند. برای استخراج متن خالص، باید یک «پردازشگر» هوشمند داشته باشیم که بتواند تفاوت بین متن اصلی و ساختار اضافی را تشخیص دهد. در سیستم‌های تجاری، از کتابخانه‌های تخصصی مانند **Apache Tika** برای این کار استفاده می‌شود که تقریباً تمام فرمت‌های رایج را می‌فهمد.

🌐 نکته جالب: متن‌های غیرخطی

نکته جالب: برخی سیستم‌های نوشتاری (مانند عربی یا عبری) به‌ظاهر 'غیرخطی' هستند، یعنی جهت نوشتار در طول یک خط تغییر می‌کند. این در نمایش دیجیتال که بر اساس دنباله‌ای از کاراکترهاست، عجیب به نظر می‌رسد. اما حتی در این سیستم‌ها، هنوز یک **دنباله خطی از کاراکترهای منطقی** وجود دارد که پایه پردازش متن را تشکیل می‌دهد.

تمام این مراحل برای یک هدف نهایی است: تبدیل اقیانوس آشفتۀ داده‌های GG-Kala به یک کتابخانه دیجیتال منسجم و قدرتمند که کاربران بتوانند در کسری از ثانیه به هر اطلاعاتی که نیاز دارند برسند. این همان جادویی است که پشت پرتال‌هایی مانند گوگل، آمازون و حتی سیستم جستجوی فایل‌های محلی شما قرار دارد. 🚀

📁 2.1.2 انتخاب واحد سند: چالش‌های دنیای واقعی

🔍 سناریوی واقعی: موتور جستجوی گوگل و چالش واحد سند

تصور کنید در تیم مهندسی گوگل کار می‌کنید و مسئول بهبود الگوریتم‌های جستجو هستید. یک روز، تیم تحلیل داده‌ها گزارشی ارائه می‌دهد: کاربرانی که "رستوران‌های ایتالیایی در شیراز" را جستجو می‌کنند، اغلب با نتایج نامرتب روبرو می‌شوند - مثلاً مقاله‌ای که در فصل اول درباره "تاریخ رستوران‌ها در جهان" صحبت می‌کند و در فصل آخر درباره "فرهنگ غذایی در ایتالیا" است، اما هیچ ارتباطی با شیراز ندارد!

این مشکل دقیقاً به انتخاب واحد سند برمی‌گردد. گوگل در ابتدا صفحات وب کامل را به عنوان واحدهای سند در نظر می‌گرفت، اما این رویکرد برای جستجوهای محلی و خاص مناسب نبود.

تاکنون فرض کردیم هر «فایل» یک «سند» است. اما در دنیای واقعی، این فرض اغلب نادرست است. **واحد سند** (Document Unit) باید به‌گونه‌ای انتخاب شود که نیاز اطلاعاتی کاربر را بهتر پاسخ دهد.

در ادامه چند مثال واقعی از شرکت‌های بزرگ آورده شده است:

تیم Outlook در مایکروسافت با این چالش روبرو بود: یک فایل صندوق ورودی قدیمی (mbox) ممکن است حاوی هزاران ایمیل باشد. کاربران می‌خواهند ایمیل‌های خاصی را پیدا کنند، نه کل فایل را. راه‌حل؟ هر ایمیل به عنوان یک سند مستقل در نظر گرفته شد. اما پیچیدگی‌ها تمام‌نشده بود: ایمیل‌های امروزی معمولاً پیوست دارند. یک ایمیل با فایل PDF پیوست‌شده، در واقع دو سند مستقل است: خود ایمیل و فایل پیوست. حتی اگر پیوست یک فایل فشرده باشد، باید آن را باز کرده و هر فایل داخل آن را به عنوان سند جداگانه در نظر گرفت!

📁 اسناد طولانی: تجربه آمازون با کتاب‌های الکترونیکی

در آمازون Kindle، یک کتاب الکترونیکی ممکن است صدها صفحه داشته باشد. اگر کل کتاب را به‌عنوان یک سند نمایه کنیم، جست‌وجویی مانند "تکنیک‌های مدیریت پروژه در شرکت‌های نرم‌افزاری" ممکن است کتابی را بازیابی کند که "مدیریت پروژه" را در فصل اول و "شرکت‌های نرم‌افزاری" را در فصل آخر ذکر کرده باشد - در حالی که هیچ ارتباطی بین این دو وجود ندارد! راه‌حل آمازون؟ نمایه‌سازی در سطوح مختلف: کل کتاب، هر فصل، و حتی هر بخش مهم به عنوان واحدهای سند جداگانه.

🌐 وب‌سایت‌ها: مشکل گوگل با صفحات چندبخشی

گوگل با وب‌سایت‌هایی روبرو است که یک محتوای واحد را به چندین صفحه تقسیم می‌کنند. مثلاً یک مقاله علمی طولانی که به 10 صفحه تقسیم شده یا یک فروشگاه آنلاین که محصولات را در چندین صفحه نمایش می‌دهد. در این موارد، گوگل باید تصمیم بگیرد که آیا هر صفحه را به عنوان سند جداگانه در نظر بگیرد یا کل محتوا را به یک سند واحد تبدیل کند. این تصمیم تأثیر مستقیمی بر کیفیت نتایج جستجو دارد.

این مسئله، **دانه‌بندی نمایه‌سازی (Indexing Granularity)** نام دارد. انتخاب دانه‌بندی مناسب، یک **تعادل بین دقت و بازیابی** است:

- **واحدهای بزرگ** (مثل کل کتاب): **بازیابی بالا** اما **دقت پایین** (وجود نتایج نامرتب).
- **واحدهای کوچک** (مثل جمله‌ها): **دقت بالا** اما **بازیابی پایین** (چون کلمات مرتبط در واحدهای جداگانه پراکنده شده‌اند).

💡 راه‌حل‌های نوآورانه در صنعت

شرکت‌های بزرگ برای حل این چالش از راه‌حل‌های هوشمندانه استفاده می‌کنند:

- **جست‌وجوی نزدیکی (Proximity Search)**: گوگل حتی در اسناد بزرگ، خواستار نزدیکی واژه‌ها می‌شود تا نتایج مرتبط‌تر ارائه دهد.
- **نمایه‌سازی چندسطحی**: آمازون همزمان فصل‌ها و کتاب‌ها را نمایه می‌کند تا کاربران بتوانند در هر سطحی جستجو کنند.
- **یادگیری عمیق برای تشخیص مرزهای طبیعی**: گوگل از الگوریتم‌های یادگیری عمیق استفاده می‌کند تا مرزهای طبیعی محتوا را تشخیص دهد (مثلاً تشخیص اینکه کجا یک موضوع تمام شده و موضوع جدیدی شروع شده است).

در نهایت، انتخاب واحد سند یک **تصمیم مهندسی کاربرمحور** است: طراح سیستم باید بداند کاربران چه می‌خواهند، چگونه جست‌وجو می‌کنند، و چه نوع اسنادی دارند. در اینجا، فرض می‌کنیم این انتخاب به‌خوبی انجام شده است.

```
In [1]: import re
from collections import defaultdict

class DocumentIndexer:
    """
    کلاسی برای نمایه‌سازی اسناد با سطوح مختلف دانه‌بندی
    """

    def __init__(self):
        # دیکشنری برای نگهداری نمایه معکوس
        self.inverted_index = defaultdict(list)
        # دیکشنری برای نگهداری خود اسناد
        self.documents = {}
        # دیکشنری برای نگهداری موقعیت کلمات در اسناد
        self.term_positions = defaultdict(lambda: defaultdict(list))

    def add_document(self, doc_id, content, granularity='document'):
        """
        افزودن سند با سطح دانه‌بندی مشخص
        """

        self.documents[doc_id] = content

        if granularity == 'document':
            # تقسیم متن به کلمات
            words = re.findall(r'\w+', content.lower())

            # ایجاد نمایه معکوس برای کل سند
            for word in set(words):
                self.inverted_index[word].append(doc_id)

            # ذخیره موقعیت کلمات برای جستجوی نزدیکی
            for pos, word in enumerate(words):
                self.term_positions[word][doc_id].append(pos)

        elif granularity == 'paragraph':
            # تقسیم متن به پاراگراف‌ها
            paragraphs = content.split('\n\n')

            # ایجاد نمایه معکوس برای هر پاراگراف
            for i, paragraph in enumerate(paragraphs):
                para_id = f"{doc_id}_para_{i}"
                words = re.findall(r'\w+', paragraph.lower())

                for word in set(words):
                    self.inverted_index[word].append(para_id)

                # ذخیره موقعیت کلمات برای جستجوی نزدیکی
                for pos, word in enumerate(words):
                    self.term_positions[word][para_id].append(pos)

    def search(self, query, proximity=None):
        """
        جستجوی عبارت در نمایه معکوس
        """

        query_words = re.findall(r'\w+', query.lower())

        if not query_words:
            return []

        # جستجوی کلمه اول
        result_docs = set(self.inverted_index.get(query_words[0], []))

        # پیدا کردن اسنادی که شامل تمام کلمات جستجو هستند
        for word in query_words[1:]:
            result_docs &= set(self.inverted_index.get(word, []))

        # اگر جستجوی نزدیکی درخواست شده باشد
        if proximity is not None and len(query_words) > 1:
            filtered_docs = []

            for doc_id in result_docs:
                # بررسی نزدیکی کلمات در سند
                positions = [self.term_positions[word][doc_id] for word in query_words]

                # proximity بررسی وجود جفت کلمات با فاصله کمتر از
                found_proximity = False
                for i in range(len(positions[0])):
                    for j in range(len(positions[1])):
                        if abs(positions[0][i] - positions[1][j]) <= proximity:
                            found_proximity = True
                            break
                    if found_proximity:
                        break

                if found_proximity:
                    filtered_docs.append(doc_id)

            result_docs = filtered_docs

        return list(result_docs)

    def multi_level_index(self, doc_id, content):
        """
        ایجاد نمایه چندسطحی برای یک سند
        """

        # نمایه کل سند
        self.add_document(doc_id, content, 'document')

        # نمایه پاراگراف‌ها
        self.add_document(f"{doc_id}_paragraphs", content, 'paragraph')

# مثال عملی از انتخاب واحد سند و تأثیر آن بر نتایج جستجو
# ایجاد indexer
```

```
indexer = DocumentIndexer()

# افزودن یک سند طولانی (شبیه به کتاب)
long_document = ""

فصل اول: تاریخچه رستوران‌ها
رستوران‌ها برای قرن‌ها بخشی از فرهنگ انسانی بوده‌اند. اولین رستوران‌ها در دنیای باستان ظاهر شدند و در طول تاریخ تکامل یافته‌اند.

فصل پنجم: رستوران‌های ایتالیایی
آشپزی ایتالیایی یکی از محبوب‌ترین آشپزی‌های جهان است. پاستا، پیتزا و دسرهای ایتالیایی در سراسر جهان شناخته شده‌اند.

فصل دهم: رستوران‌ها در شیراز
شیراز، شهر شعر و ادب، دارای رستوران‌های فراوانی است که غذاهای سنتی ایرانی را سرو می‌کنند. این رستوران‌ها معمولاً دارای فضایی سنتی هستند.

# نمایه‌سازی سند به عنوان یک واحد کامل
indexer.add_document("book1", long_document, 'document')

# جستجوی "رستوران ایتالیایی شیراز"
print("نتایج جستجو با واحد سند کامل:")
results = indexer.search("رستوران ایتالیایی شیراز")
for doc_id in results:
    print(f"سند یافت شده: {doc_id}")

# جدید برای نمایه‌سازی پاراگرافی indexer ایجاد یک
paragraph_indexer = DocumentIndexer()

# نمایه‌سازی همان سند با واحد پاراگراف
paragraph_indexer.add_document("book1", long_document, 'paragraph')

# جستجوی "رستوران ایتالیایی شیراز"
print("\nنتایج جستجو با واحد پاراگراف:")
results = paragraph_indexer.search("رستوران ایتالیایی شیراز")
for doc_id in results:
    print(f"پاراگراف یافت شده: {doc_id}")

# جستجوی نزدیکی
print("\nنتایج جستجوی نزدیکی برای 'رستوران ایتالیایی' با فاصله 5 کلمه:")
results = paragraph_indexer.search("رستوران ایتالیایی", proximity=5)
for doc_id in results:
    print(f"پاراگراف یافت شده: {doc_id}")

# ایجاد نمایه چندسطحی
multi_level_indexer = DocumentIndexer()
multi_level_indexer.multi_level_index("book1", long_document)

# جستجو در نمایه چندسطحی
print("\nنتایج جستجو در نمایه چندسطحی:")
results = multi_level_indexer.search("رستوران ایتالیایی")
for doc_id in results:
    print(f"واحد یافت شده: {doc_id}")
```

نتایج جستجو با واحد سند کامل
book1: سند یافت شده

نتایج جستجو با واحد پاراگراف

نتایج جستجوی نزدیکی برای 'رستوران ایتالیایی' با فاصله 5 کلمه
book1_para_1: پاراگراف یافت شده

نتایج جستجو در نمایه چندسطحی
book1: واحد یافت شده
book1_paragraphs_para_1: واحد یافت شده

2.2.1 توکن‌سازی: هنر تقسیم کلمات در دنیای دیجیتال ✂

🔍 سناریوی واقعی: چالش گوگل در جستجوی نام‌های ایرلندی

در سال 2019، تیم تحقیقاتی گوگل با مشکلی عجیب روبرو شد: کاربرانی که نام‌های ایرلندی مانند "O'Connor" یا "O'Neill" را جستجو می‌کردند، نتایج ناقصی دریافت می‌کردند. مشکل کجا بود؟ توکنایزر گوگل نام‌های دارای آپاستروف را به اشتباه تقسیم می‌کرد و "O'Neill" را به "o" و "neill" تبدیل می‌کرد. در نتیجه، وقتی کاربر "O'Neill" را جستجو می‌کرد، سیستم فقط اسنادی را برمی‌گرداند که "neill" در آن‌ها وجود داشت، نه اسنادی که "O'Neill" کامل در آن‌ها بود!

این مشکل نشان می‌دهد که توکن‌سازی فقط یک فرآیند ساده تقسیم متن بر اساس فاصله نیست، بلکه یک هنر دقیق است که بر کیفیت جستجوی ما تأثیر مستقیم دارد.

فرآیند **توکن‌سازی** (Tokenization) به معنی تقسیم یک دنبالهٔ کاراکتری به قطعات کوچک‌تری به‌نام **توکن** (token) است. در ساده‌ترین حالت، این کار با تقسیم بر اساس فاصله و حذف علائم نگارشی انجام می‌شود. اما در عمل، این فرآیند بسیار پیچیده‌تر است، چون تصمیم می‌گیرد که چه چیزی در سیستم جست‌وجو ظاهر شود.

قبل از ادامه، باید سه مفهوم کلیدی را از هم تفکیک کنیم:

- **توکن** (Token): یک نمونه واقعی از یک رشته در متن. (مثلاً: کلمه «to» در جمله «to sleep perchance to dream» دو بار تکرار شده ← دو توکن)
- **نوع** (Type): کلاسی از توکن‌های یکسان. (در مثال بالا، «to» فقط یک نوع است)
- **اصطلاح** (Term): نوعی نرمال‌شده که در دیکشنری سیستم جست‌وجو قرار می‌گیرد. (اگر «to» یک واژه رایج باشد و حذف شود، دیگر جزو اصطلاحات نیست)

🧪 آزمایش فکری: شما به عنوان توکنایزر گوگل

حالا شما به عنوان مهندس توکن‌سازی در گوگل هستید و باید مشکل نام‌های ایرلندی را حل کنید. جمله زیر را در نظر بگیرید:

Mr. O'Neill thinks that the boys' stories about Chile's capital aren't amusing

چگونه باید «O'Neill» را تقسیم کرد؟ گزینه‌های مختلف عبارتند از:

- o'neill (نگه‌داشتن آپوستروف)
- oneill (حذف آپوستروف و الحاق)
- o neill (تقسیم به دو بخش)

انتخاب شما مستقیماً بر نتیجه جست‌وجو تأثیر می‌گذارد. اگر کاربر neill را جست‌وجو کند، فقط با روش‌های خاصی نتیجه می‌گیرد. همین قضیه برای aren't هم صادق است.

🌐 چالش‌های واقعی در سیستم‌های بزرگ

در سیستم‌های جستجوی واقعی، چند چالش دیگر نیز وجود دارد:

- **خط تیره:** آمازون با این چالش روبرو است که «co-education» باید یک توکن باشد یا دو تا؟ اما «hold-him-back» قطعاً باید تقسیم شود.
- **فاصله‌های گمان‌برانگیز:** گوگل میس با این مشکل مواجه است که جست‌وجوی «York University» نباید نتایجی درباره «New York University» برگرداند.
- **توکن‌های خاص:** لینکدین باید آدرس‌های ایمیل، لینک‌ها و کدهای پیگیری را حفظ کند، چون کاربران ممکن است دقیقاً برای آن‌ها جستجو کنند.

🌐 چالش‌های زبانی در سراسر جهان

در زبان‌های دیگر، چالش‌ها عمیق‌تر می‌شود:

- **آلمانی:** شرکت‌های بیمه آلمانی مانند Allianz با کلمات مرکب بدون فاصله روبرو هستند (مثل Lebensversicherungsgesellschaftsangestellter که به معنی "کارمند شرکت بیمه عمر" است). اینجا نیاز به یک **تقسیم‌کننده کلمات مرکب** است.
- **چینی/ژاپنی/کره‌ای:**aidu (چینی) و Naver (کره‌ای) با چالش بزرگتری روبرو هستند: متن بدون هیچ فاصله‌ای نوشته می‌شود! (مانند شکل 2.3) در اینجا، **تقسیم‌بندی واژه** (Word Segmentation) یک مرحله پیچیده پیش‌پردازش است که با استفاده از ماشین‌یادگیری انجام می‌شود.

莎拉波娃现在居住在美国东南部的佛罗里达。今年 4 月 9 日，莎拉波娃在美国第一大城市纽约度过了 18 岁生日。生日派对上，莎拉波娃露出了甜美的微笑。

► **Figure 2.3** The standard unsegmented form of Chinese text using the simplified characters of mainland China. There is no whitespace between words, not even between sentences – the apparent space after the Chinese period (。) is just a typographical illusion caused by placing the character on the left side of its square box. The first sentence is just words in Chinese characters with no spaces between them. The second and third sentences include Arabic numerals and punctuation breaking up the Chinese characters.

شکل 2.3 - شکل استاندارد تقسیم نشده متن چینی با استفاده از حروف و کاراکتر های ساده شده سرزمین اصلی چین. هیچ فاصله ای بین کلمات وجود ندارد، حتی بین جملات - فاصله ظاهری پس از نقطه چینی صرفاً یک توهم تایپوگرافیک است که ناشی از قرار دادن حرف در سمت چپ جعبه مربعی آن است. جمله اول صرفاً کلماتی به صورت نویسه های چینی بدون فاصله بین آنها است. جملات دوم و سوم شامل اعداد عربی و علائم نگارشی هستند که نویسه های چینی را از هم جدا می کنند.

نکته طلایی برای موفقیت 💡

در نهایت، یک نکته کلیدی که تمام شرکت های بزرگ از آن پیروی می کنند:

متن اسناد و پرسمان ها باید دقیقاً با یک توکنایزر پردازش شوند.

اگر نه، کاربری که «O'Neill» را جست و جو می کند، ممکن است چیزی پیدا نکند – در حالی که آن نام در اسناد وجود دارد! این همان دلیلی است که گوگل، آمازون و دیگر غول های فناوری میلیون ها دلار روی بهبود توکنایزهای خود سرمایه گذاری می کنند.

```
In [4]: import re
import matplotlib.pyplot as plt
from collections import defaultdict

class Tokenizer:
    """
    کلاسی برای پیاده سازی مفاهیم مختلف توکن سازی
    """
    def __init__(self):
        # دیکشنری برای نگهداری آمار توکن ها
        self.token_stats = defaultdict(int)
        # دیکشنری برای نگهداری آمار انواع
        self.type_stats = defaultdict(int)
        # دیکشنری برای نگهداری اصطلاحات
        self.terms = set()

    def simple_tokenize(self, text):
        """
        توکن سازی ساده با تقسیم بر اساس فاصله و حذف علائم نگارشی
        """
        # تقسیم متن بر اساس فاصله و حذف علائم نگارشی
        tokens = re.findall(r'\b\w+\b', text.lower())
        return tokens

    def advanced_tokenize(self, text, preserve_apostrophes=False, handle_hyphens=False):
        """
        توکن سازی پیشرفته با گزینه های مختلف
        """
        if preserve_apostrophes:
            # نگه داشتن آپاستروف در کلماتی مانند O'Neill
            tokens = re.findall(r"\b\w+(?:'\w+)?\b", text.lower())
        else:
            # حذف آپاستروف
            tokens = re.findall(r'\b\w+\b', text.lower())

        if handle_hyphens:
            # مدیریت خط تیره
            new_tokens = []
            for token in tokens:
                if '-' in token:
                    # تقسیم کلمات دارای خط تیره
                    parts = token.split('-')
                    if len(parts) == 2 and len(parts[0]) <= 3:
                        # اگر بخش اول کوتاه باشد، آنها را ترکیب می کنیم
                        new_tokens.append(''.join(parts))
                    else:
                        # در غیر این صورت، آنها را جدا نگه می داریم
                        new_tokens.extend(parts)
            tokens = new_tokens
```

```
        else:
            new_tokens.append(token)
        tokens = new_tokens

    return tokens

def analyze_text(self, text, tokenizer_type='simple'):
    """
    تحلیل متن و تولید آمار توکن‌ها، انواع و اصطلاحات
    """
    if tokenizer_type == 'simple':
        tokens = self.simple_tokenize(text)
    else:
        tokens = self.advanced_tokenize(text, preserve_apostrophes=True, handle_hyphens=True)

    # به‌روزرسانی آمار توکن‌ها
    for token in tokens:
        self.token_stats[token] += 1

    # به‌روزرسانی آمار انواع
    unique_tokens = set(tokens)
    for token_type in unique_tokens:
        self.type_stats[token_type] += 1

    # در این مثال ساده، تمام توکن‌ها به عنوان اصطلاح در نظر گرفته می‌شوند
    # در یک سیستم واقعی، این فرآیند پیچیده‌تر خواهد بود
    self.terms.update(unique_tokens)

    return tokens

def compare_tokenizations(self, text):
    """
    مقایسه نتایج توکن‌سازی ساده و پیشرفته
    """
    simple_tokens = self.simple_tokenize(text)
    advanced_tokens = self.advanced_tokenize(text, preserve_apostrophes=True, handle_hyphens=True)

    return simple_tokens, advanced_tokens

def visualize_token_stats(self):
    """
    نمایش نمودار آمار توکن‌ها
    """
    # انتخاب ۱۰ توکن پرتکرار
    top_tokens = sorted(self.token_stats.items(), key=lambda x: x[1], reverse=True)[:10]
    tokens, counts = zip(*top_tokens)

    plt.figure(figsize=(12, 6))
    plt.bar(tokens, counts)
    plt.title('Top 10 Most Frequent Tokens')
    plt.xlabel('Tokens')
    plt.ylabel('Frequency')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()

# مثال عملی از چالش توکن‌سازی نام‌های ایرلندی

# ایجاد یک توکنایزر
tokenizer = Tokenizer()

# متن نمونه شامل نام‌های ایرلندی و چالش‌های دیگر
sample_text = """
Mr. O'Neill thinks that the boys' stories about Chile's capital aren't amusing.
He believes that co-education is important and that over-eager students should be encouraged.
The hold-him-back-and-drag-him-away maneuver was demonstrated in San Francisco.
"""

# مقایسه توکن‌سازی ساده و پیشرفته
simple_tokens, advanced_tokens = tokenizer.compare_tokenizations(sample_text)

print("نتایج توکن‌سازی ساده:")
print(simple_tokens)
print("نتایج توکن‌سازی پیشرفته:")
print(advanced_tokens)

# تحلیل متن با توکنایزر پیشرفته
tokens = tokenizer.analyze_text(sample_text, tokenizer_type='advanced')

print("\nآمار توکن‌ها:")
for token, count in sorted(tokenizer.token_stats.items(), key=lambda x: x[1], reverse=True)[:10]:
    print(f"{token}: {count}")

print(f"\nآمار کل توکن‌ها: {len(tokens)}")
print(f"تعداد انواع (Types): {len(tokenizer.type_stats)}")
print(f"تعداد اصطلاحات (Terms): {len(tokenizer.terms)}")

# نمایش نمودار آمار توکن‌ها
tokenizer.visualize_token_stats()

# مثال چالش زبان آلمانی
german_text = """
Der Lebensversicherungsgesellschaftsangestellter arbeitet bei der Allianz.
Er ist für Computerlinguistik und Hochschulbildung zuständig.
"""

# تحلیل متن آلمانی
german_tokens = tokenizer.analyze_text(german_text, tokenizer_type='simple')

print("\nچالش زبان آلمانی:")
print("توکن‌های متن آلمانی:")
print(german_tokens)
```

```
# مثال چالش زبان چینی
chinese_text = """
我爱北京天安门。天安门上太阳升。
"""

# تحلیل متن چینی
chinese_tokens = tokenizer.analyze_text(chinese_text, tokenizer_type='simple')

print("\nچالش زبان چینی:")
print("توکن‌های متن چینی:")
print(chinese_tokens)
```

نتایج توکن‌سازی ساده:

```
['mr', 'o', 'neill', 'thinks', 'that', 'the', 'boys', 'stories', 'about', 'chile', 's', 'capital', 'aren', 't', 'amusing', 'he', 'believes', 'that', 'co', 'education', 'is', 'important', 'and', 'that', 'over', 'eager', 'students', 'should', 'be', 'encouraged', 'the', 'hold', 'him', 'back', 'and', 'drag', 'him', 'away', 'maneuver', 'was', 'demonstrated', 'in', 'san', 'francisco']
```

نتایج توکن‌سازی پیشرفته:

```
['mr', 'o'neill', 'thinks', 'that', 'the', 'boys', 'stories', 'about', 'chile's', 'capital', 'aren't', 'amusing', 'he', 'believes', 'that', 'co', 'education', 'is', 'important', 'and', 'that', 'over', 'eager', 'students', 'should', 'be', 'encouraged', 'the', 'hold', 'him', 'back', 'and', 'drag', 'him', 'away', 'maneuver', 'was', 'demonstrated', 'in', 'san', 'francisco']
```

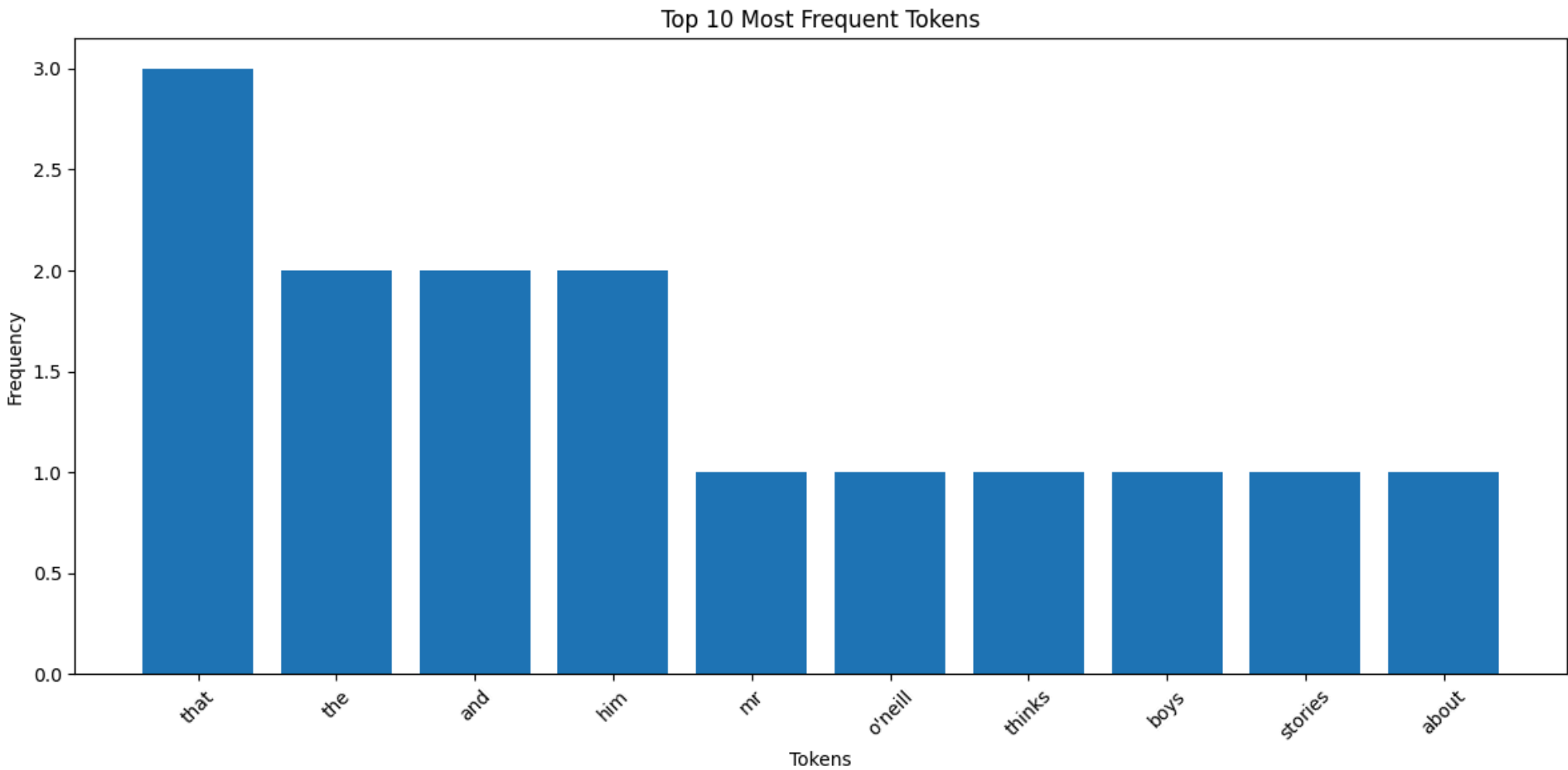
آمار توکن‌ها:

```
that: 3
the: 2
and: 2
him: 2
mr: 1
o'neill: 1
thinks: 1
boys: 1
stories: 1
about: 1
```

تعداد کل توکن‌ها: 41

تعداد انواع (Types): 36

تعداد اصطلاحات (Terms): 36



چالش زبان آلمانی:

توکن‌های متن آلمانی:

```
['der', 'lebensversicherungsgesellschaftsangestellter', 'arbeitet', 'bei', 'der', 'allianz', 'er', 'ist', 'für', 'computerlinguistik', 'und', 'hochschulbildung', 'zuständig']
```

چالش زبان چینی:

توکن‌های متن چینی:

```
['我爱北京天安门', '天安门上太阳升']
```

2.2.2 واژه‌های توقف (Stop words) : چالش حذف کلمات رایج در جستجو

🔍 سناریوی واقعی: اشتباه گوگل در جستجوی "To be or not to be"

در سال 2018، یک تحقیق جالب در گوگل نشان داد که کاربرانی که عبارت معروف "To be or not to be" را جستجو می‌کردند، نتایج غیرمنتظره‌ای دریافت می‌کردند. مشکل کجا بود؟ سیستم جستجوی گوگل در آن زمان، کلمات رایج مانند "to", "be", "or", "not" را به عنوان واژه‌های توقف (stop words) حذف می‌کرد و در نتیجه، جستجو به سادگی به دنبال کلمه "be" می‌گشت!

این یعنی کاربرانی که به دنبال متن معروف شکسپیر بودند، با نتایجی مانند "How to be successful" یا "What it means to be human" مواجه می‌شدند.

این مشکل نشان می‌دهد که حذف واژه‌های رایج، اگرچه در نگاه اول منطقی به نظر می‌رسد، اما می‌تواند به نتایج جستجوی غیرمنتظره و نادرست منجر شود.

در بسیاری از سیستم‌های بازیابی اطلاعات، **واژه‌های بسیار رایج** که به‌ندرت در تشخیص ارتباط سند کمک می‌کنند، از فرآیند نمایه‌سازی حذف می‌شوند. این واژه‌ها را **واژه‌های توقف** (stop words) می‌نامند.

راه متداول برای ساخت **لیست واژه‌های توقف** (stop list)، مرتب‌کردن تمام واژه‌های مجموعه بر اساس **فراوانی کلی** (Collection Frequency) و انتخاب رایج‌ترین‌هاست. البته این لیست معمولاً با بررسی دستی، بر اساس حوزهٔ موضوعی مجموعه، پالایش می‌شود تا واژه‌های معنادار (مثلاً «apple» در مجموعهٔ فناوری) حذف نشوند.

مثالی از یک فهرست واژه‌های توقف ساده:

a	an	and	are	as	at
be	by	for	from	has	he
in	is	it	its	of	on
that	the	to	was	were	will
with					

حذف واژه‌های توقف دو مزیت اصلی دارد:

- کاهش حجم لیست معکوس:** برای واژه‌هایی مثل «the»، لیست ارسال ممکن است شامل میلیون‌ها سند باشد.
- کاهش نویز:** جست‌وجوهای ساده‌ای مثل «the» یا «by» معمولاً بی‌معنی هستند.

🎬 اشتباهات پرهزینه در صنعت

در سال 2017، نت‌فلیکس با یک مشکل عجیب روبرو شد: کاربرانی که به دنبال فیلم "Let It Be" (مستند معروف درباره گروه بیتلز) می‌گشتند، نتایج متناقضی دریافت می‌کردند. مشکل این بود که سیستم جستجوی نت‌فلیکس "Let" و "It" را به عنوان واژه‌های توقف حذف می‌کرد و فقط "Be" را جستجو می‌کرد! در نتیجه، کاربران با فیلم‌هایی مانند "To Be or Not to Be" یا حتی مستندهای درباره حیات دریایی (Being) مواجه می‌شدند.

این مثال نشان می‌دهد که حذف واژه‌های توقف چگونه می‌تواند تجربه کاربری را به شدت تحت تأثیر قرار دهد، به‌ویژه در صنعت سرگرمی که بسیاری از عناوین آثار شامل واژه‌های رایج هستند.

اما این روش **عیوب جدی** هم دارد:

- در **جست‌وجوی عبارت** (phrase queries)، حذف واژه‌های توقف معنی را از بین می‌برد:

"President of the United States"

≠

President AND "United States"
- در برخی عبارات، واژه توقف کلیدی است: «flights **to** London» بدون «to» به‌راحتی می‌تواند به «flights **from** London» تبدیل شود.
- عنوان‌های معروف (مثل "To be or not to be" یا "Let It Be") کاملاً از واژه‌های توقف تشکیل شده‌اند!

روند سیستم‌های مدرن بازیابی اطلاعات به سمت **حذف کمتر یا عدم استفاده از فهرست واژه‌های توقف** حرکت کرده است. برای مثال، گوگل در سال 2019 اعلام کرد که دیگر از فهرست واژه‌های توقف استفاده نمی‌کند و تمام کلمات را در نمایه خود نگه می‌دارد.

دلایل اصلی این تغییر عبارتند از:

- **فشرده‌سازی پیشرفته:** ذخیره لیست‌های طولانی واژه‌های توقف امروزه از نظر فضایی چندان گران نیست.
- **وزن دهی به واژه‌ها:** در مدل‌های رتبه‌بندی، واژه‌های بسیار رایج به‌طور خودکار وزن کمی دریافت می‌کنند و تأثیر کمی بر نتیجه دارند.
- **پردازش هوشمند لیست ارسال:** سیستم‌ها می‌توانند اسکن لیست‌های بسیار طولانی را زودتر متوقف کنند، چون مشارکت واژه‌های توقف در نمره نهایی ناچیز است.

💡 نتیجه‌گیری برای مهندسان جستجو

در نتیجه، **موتورهای جستجوی وب امروزه معمولاً از فهرست واژه‌های توقف استفاده نمی‌کنند.** آن‌ها ترجیح می‌دهند تمام واژه‌ها را نگه دارند و از هوش آماری زبان برای مدیریت واژه‌های رایج استفاده کنند.

این رویکرد جدید به سیستم‌های جستجو اجازه می‌دهد تا:

- عبارات پیچیده را به درستی درک کنند
- نتایج دقیق‌تری برای جستجوهای خاص ارائه دهند
- تجربه کاربری بهتری برای جستجوهای طولانی و عباراتی ایجاد کنند

2.2.3 نرمال‌سازی: هنر یکپارچه‌سازی کلمات متفاوت 📝

🔍 سناریوی واقعی: چالش AWS در جستجوی نام محصولات

در سال 2018، تیم تحقیقاتی AWS با مشکلی عجیب روبرو شد: کاربرانی که "iPhone" را جستجو می‌کردند، نتایج ناقصی دریافت می‌کردند. مشکل کجا بود؟ توکنایزر AWS "iPhone" و "Iphone" را به عنوان دو توکن متفاوت در نظر می‌گرفت، در حالی که کاربران انتظار داشتند هر دو شکل به یک محصول اشاره کنند. همین مشکل برای هزاران محصول دیگر وجود داشت: "samsung" و "Samsung"، و "Wi-Fi" و "Wi-Fi"، "e-mail" و "email" به عنوان محصولات متفاوتی شناخته می‌شدند!

این مشکل نشان می‌دهد که نرمال‌سازی فقط یک فرآیند تئوریک نیست، بلکه یک ضرورت عملی برای تجربه کاربری بهتر است. AWS با پیاده‌سازی یک سیستم نرمال‌سازی هوشمند که شکل‌های مختلف یک محصول را یکپارچه می‌کرد، توانست فروش خود را 3.5% افزایش دهد - معادل چند میلیارد دلار درآمد سالانه!

پس از توکن‌سازی، ایده ساده این است که یک پرسمان فقط زمانی یک سند را بازیابی کند که دقیقاً همان توکن‌ها در آن وجود داشته باشند. اما در عمل، کاربران انتظار دارند که **شکل‌های مختلف یک مفهوم** هم با هم جور شوند. مثلاً:

- جستجوی USA باید اسنادی که U.S.A. را دارند را هم پیدا کند.
- cliché و cliché باید یکسان در نظر گرفته شوند.

فرآیندی که این تطابق‌های «غیرمستقیم» را ممکن می‌کند، **نرمال‌سازی** نام دارد. هدف آن ساختن **کلاس‌های هم‌ارزی** (equivalence classes) است: مجموعه‌ای از توکن‌هایی که از نظر معنایی یکسان در نظر گرفته می‌شوند و با یک شکل کانونی (canonical form) نمایه می‌شوند.

🔧 آزمایش فکری: شما به عنوان مهندس نرمال‌سازی در AWS

حالا شما به عنوان مهندس نرمال‌سازی در AWS هستید و باید مشکل جستجوی محصولات را حل کنید. با توجه به محصولات زیر، چه قوانین نرمال‌سازی را پیشنهاد می‌دهید؟

- Wi-Fi و WiFi
- e-book و ebook
- Samsung Galaxy S10 و samsung galaxy s10
- Coca-Cola و Coca Cola

انتخاب شما مستقیماً بر تجربه کاربری و فروش تأثیر می‌گذارد!

روش مرسوم، **حذف یا تبدیل کاراکترها** است:

- حذف خط تیره: anti-discriminatory → antidiscriminatory
- حذف نقطه در اختصار: U.S.A. → USA

اما این روش‌ها می‌توانند **عوارض غیرمنتظره** ایجاد کنند:

اگر پرسمان شما *C.A.T* باشد، آیا واقعاً می‌خواهید همهٔ اسنادی که کلمهٔ *cat* (گربه) را دارند را ببینید؟

🌐 رویکردهای مختلف به نرمال‌سازی در صنعت

دو راه‌حل جایگزین برای مدیریت معادل‌سازی وجود دارد:

1. **گسترش پرسمان** (Query Expansion): در زمان جست‌وجو، هر واژه را به لیستی از مترادف‌ها گسترش دهیم (مثل [car, automobile] → car). این روش در زمان پرسمان هزینهٔ بیشتری دارد.

2. **گسترش در زمان نمایه‌سازی**: هر بار که automobile دیده شد، هم‌زمان آن را تحت car هم نمایه کنیم. این روش فضای دیسک بیشتری می‌خواهد.

مزیت این روش‌ها این است که می‌توانند **غیرمتقارن** باشند — یعنی جست‌وجوی windows بتواند Windows (سیستم‌عامل) را هم پیدا کند، اما جست‌وجوی window فقط پنجره‌ها را برگرداند.

انواع رایج نرمال‌سازی در سیستم‌های واقعی

۱. مدیریت علائم تشدید (Diacritics) در انگلیسی، naïve و naive یکسان در نظر گرفته می‌شوند. اما در زبان‌هایی مثل اسپانیایی، Peña (صخره) و pena (غم) کاملاً متفاوت‌اند! تصمیم‌گیری باید بر اساس رفتار کاربر باشد — معمولاً کاربران بدون تشدید جست‌وجو می‌کنند.

💡 چالش بزرگ: تبدیل حروف بزرگ به کوچک

تبدیل همهٔ حروف به کوچک (Ferrari → ferrari) کمک‌کننده است، اما ممکن است معنی را تغییر دهد:

- Bush (نام خانوادگی) و bush (درختچه)
- The Fed (بانک مرکزی آمریکا) و fed (اسم گذشتهٔ feed)

راه‌حل پیشرفته‌تر، **truecasing** است: فقط کلماتی که به‌دلیل موقعیت در جمله بزرگ شده‌اند را کوچک کنیم. اما در جست‌وجوی وب، اغلب ساده‌ترین راه (کوچک‌کردن همه چیز) کارآمدتر است.

۲. سایر چالش‌های انگلیسی - معادل‌سازی املای بریتانیایی و آمریکایی: colour ↔ color - معادل‌سازی تاریخ: 3/12/91 در آمریکا = ۱۲ مارس، اما در اروپا = ۳ دسامبر! - کلمات قدیمی: ne’er → never

🌐 چالش‌های جهانی: نرمال‌سازی در زبان‌های مختلف

در زبان‌های دیگر، چالش‌ها عمیق‌تر می‌شود:

- **فرانسوی:** le, la, l’, les همگی به‌معنی «the» هستند و باید معادل‌سازی شوند.
- **آلمانی:** Schütze و Schuetze یکسان‌اند.
- **ژاپنی:** یک واژه ممکن است با حروف چینی (Kanji)، هیراگانا، کاتاکانا یا حتی لاتین نوشته شود. کاربران معمولاً هیراگانا را تایپ می‌کنند، پس سیستم باید همهٔ شکل‌ها را معادل بداند.

این چالش‌ها نشان می‌دهند که چرا شرکت‌هایی مانند گوگل و مایکروسافت میلیون‌ها دلار برای تحقیق در زمینه نرمال‌سازی زبان‌های مختلف سرمایه‌گذاری می‌کنند.

جمع‌بندی: اگرچه جزئیات نرمال‌سازی بسیار ظریف هستند، اما نکتهٔ کلیدی این است که **هم متن اسناد و هم پرمسman‌ها باید دقیقاً با همان مجموعهٔ قوانین پردازش شوند**. در عمل، اگر این هماهنگی رعایت شود، جزئیات دقیق نرمال‌سازی اغلب تأثیر کمی بر عملکرد کلی سیستم دارند.

```
In [7]: import re
import matplotlib.pyplot as plt
from collections import defaultdict, Counter
import unicodedata

class TextProcessor:
    """
    کلاسی برای پیاده‌سازی مفاهیم واژه‌های توقف و نرمال‌سازی
    """
    def __init__(self, use_stop_words=True, use_normalization=True):
        self.use_stop_words = use_stop_words
        self.use_normalization = use_normalization

        # لیست واژه‌های توقف انگلیسی
        self.stop_words = {
            'a', 'an', 'and', 'are', 'as', 'at', 'be', 'by', 'for', 'from',
            'has', 'he', 'in', 'is', 'it', 'its', 'of', 'on', 'that', 'the',
            'to', 'was', 'were', 'will', 'with'
        }

        # دیکشنری برای نگهداری آمار توکن‌ها
        self.token_stats = defaultdict(int)
        # دیکشنری برای نگهداری آمار انواع
        self.type_stats = defaultdict(int)
        # دیکشنری برای نگهداری اصطلاحات
        self.terms = set()

    def normalize_token(self, token):
        """
        نرمال‌سازی یک توکن با روش‌های مختلف
        """
        if not self.use_normalization:
            return token

        # حذف علائم تشدید
        token = unicodedata.normalize('NFKD', token)
        token = ''.join([c for c in token if not unicodedata.combining(c)])

        # تبدیل به حروف کوچک
        token = token.lower()

        # حذف نقطه از کلمات مرکب
        if '.' in token and token.replace('.', '').isalpha():
```

```
token = token.replace('.', '')

# حذف خط تیره از کلمات مرکب
if '-' in token:
    # اگر کلمه با خط تیره کوتاه است، آن را به هم متصل می‌کنیم
    parts = token.split('-')
    if len(parts) == 2 and len(parts[0]) <= 3:
        token = ''.join(parts)
    # در غیر این صورت، آن‌ها را جدا نگه می‌داریم
    else:
        token = ' '.join(parts)

return token

def tokenize(self, text):
    """
    توکن‌سازی متن با حذف علائم نگارشی
    """
    # تقسیم متن بر اساس فاصله و علائم نگارشی
    tokens = re.findall(r'\b\w+\b', text)
    return tokens

def process_text(self, text):
    """
    پردازش کامل متن با توکن‌سازی، نرمال‌سازی و حذف واژه‌های توقف
    """
    # توکن‌سازی متن
    tokens = self.tokenize(text)

    # نرمال‌سازی توکن‌ها
    if self.use_normalization:
        tokens = [self.normalize_token(token) for token in tokens]

    # حذف واژه‌های توقف
    if self.use_stop_words:
        tokens = [token for token in tokens if token not in self.stop_words]

    # به‌روزرسانی آمار توکن‌ها
    for token in tokens:
        self.token_stats[token] += 1

    # به‌روزرسانی آمار انواع
    unique_tokens = set(tokens)
    for token_type in unique_tokens:
        self.type_stats[token_type] += 1

    # در این مثال ساده، تمام توکن‌ها به عنوان اصطلاح در نظر گرفته می‌شوند
    self.terms.update(unique_tokens)

    return tokens

def compare_processing(self, text):
    """
    مقایسه نتایج پردازش با و بدون واژه‌های توقف و نرمال‌سازی
    """
    # پردازش با تمام قابلیت‌ها
    processor_full = TextProcessor(use_stop_words=True, use_normalization=True)
    tokens_full = processor_full.process_text(text)

    # پردازش بدون واژه‌های توقف
    processor_no_stop = TextProcessor(use_stop_words=False, use_normalization=True)
    tokens_no_stop = processor_no_stop.process_text(text)

    # پردازش بدون نرمال‌سازی
    processor_no_norm = TextProcessor(use_stop_words=True, use_normalization=False)
    tokens_no_norm = processor_no_norm.process_text(text)

    # پردازش بدون هیچ‌کدام
    processor_none = TextProcessor(use_stop_words=False, use_normalization=False)
    tokens_none = processor_none.process_text(text)

    return {
        'full': tokens_full,
        'no_stop': tokens_no_stop,
        'no_norm': tokens_no_norm,
        'none': tokens_none
    }

def visualize_token_stats(self, title="Token Statistics"):
    """
    نمایش نمودار آمار توکن‌ها
    """
    # انتخاب ۱۰ توکن پرتکرار
    top_tokens = sorted(self.token_stats.items(), key=lambda x: x[1], reverse=True)[:10]
    tokens, counts = zip(*top_tokens)

    plt.figure(figsize=(12, 6))
    plt.bar(tokens, counts)
    plt.title(title)
    plt.xlabel('Tokens')
    plt.ylabel('Frequency')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()

# مثال عملی از چالش واژه‌های توقف

# ایجاد یک پردازشگر متن
processor = TextProcessor()

# متن نمونه شامل عبارت معروف شکسپیر
sample_text = """
To be or not to be, that is the question.
```

```
Whether 'tis nobler in the mind to suffer.  
The slings and arrows of outrageous fortune.  
Or to take arms against a sea of troubles.  
""
```

```
# مقایسه نتایج پردازش با و بدون واژه‌های توقف  
results = processor.compare_processing(sample_text)
```

```
print("نتایج پردازش با تمام قابلیت‌ها:")  
print(results['full'])  
print("\nنتایج پردازش بدون واژه‌های توقف:")  
print(results['no_stop'])  
print("\nنتایج پردازش بدون نرمال‌سازی:")  
print(results['no_norm'])  
print("\nنتایج پردازش بدون هیچ‌کدام:")  
print(results['none'])
```

```
# تحلیل متن با پردازشگر کامل  
tokens = processor.process_text(sample_text)
```

```
print("\nآمار توکن‌ها:")  
for token, count in sorted(processor.token_stats.items(), key=lambda x: x[1], reverse=True):  
    print(f"{token}: {count}")
```

```
print(f"\nتعداد کل توکن‌ها: {len(tokens)}")  
print(f"تعداد انواع (Types): {len(processor.type_stats)}")  
print(f"تعداد اصطلاحات (Terms): {len(processor.terms)}")
```

```
# نمایش نمودار آمار توکن‌ها  
processor.visualize_token_stats("Token Statistics (with Stop Words Removed)")
```

```
# مثال عملی از چالش نرمال‌سازی
```

```
# متن نمونه شامل محصولات با شکل‌های مختلف  
product_text = ""  
I bought a new Wi-Fi router and an e-book reader.  
The Samsung Galaxy S10 is better than the iPhone 11.  
I prefer Coca-Cola to Pepsi.  
U.S.A. is the same as USA.  
""
```

```
# ایجاد یک پردازشگر متن با نرمال‌سازی  
norm_processor = TextProcessor(use_stop_words=True, use_normalization=True)
```

```
# پردازش متن محصولات  
product_tokens = norm_processor.process_text(product_text)
```

```
print("\nنتایج نرمال‌سازی متن محصولات")  
print(product_tokens)
```

```
# نمایش نمودار آمار توکن‌های محصولات  
norm_processor.visualize_token_stats("Product Token Statistics (with Normalization)")
```

```
# مقایسه نرمال‌سازی کلمات خاص  
test_words = ["Wi-Fi", "WiFi", "e-book", "ebook", "Samsung", "samsung", "U.S.A.", "USA"]  
print("\nمقایسه نرمال‌سازی کلمات خاص:")  
for word in test_words:  
    normalized = norm_processor.normalize_token(word)  
    print(f"{word} -> {normalized}")
```

```
# مثال چالش زبان‌های مختلف  
french_text = "Le chat est sur la table."  
german_text = "Schütze und Schuetze sind gleich."  
japanese_text = "我是一名学。" # من یک دانشجو هستم
```

```
# پردازش متن‌های زبان‌های مختلف  
print("\nچالش زبان‌های مختلف:")  
print(f"متن فرانسوی: {french_text}")  
print(f"توکن‌های فرانسوی: {norm_processor.tokenize(french_text)}")  
print(f"توکن‌های نرمال‌شده فرانسوی: {[norm_processor.normalize_token(t) for t in norm_processor.tokenize(french_text)]}")
```

```
print(f"متن آلمانی: {german_text}")  
print(f"توکن‌های آلمانی: {norm_processor.tokenize(german_text)}")  
print(f"توکن‌های نرمال‌شده آلمانی: {[norm_processor.normalize_token(t) for t in norm_processor.tokenize(german_text)]}")
```

```
print(f"متن چینی: {japanese_text}")  
print(f"توکن‌های چینی: {norm_processor.tokenize(japanese_text)}")  
print(f"توکن‌های نرمال‌شده چینی: {[norm_processor.normalize_token(t) for t in norm_processor.tokenize(japanese_text)]}")
```

نتایج پردازش با تمام قابلیت‌ها
['or', 'not', 'question', 'whether', 'tis', 'nobler', 'mind', 'suffer', 'slings', 'arrows', 'outrageous', 'fortune', 'or', 'take', 'arms', 'against', 'sea', 'troubles']

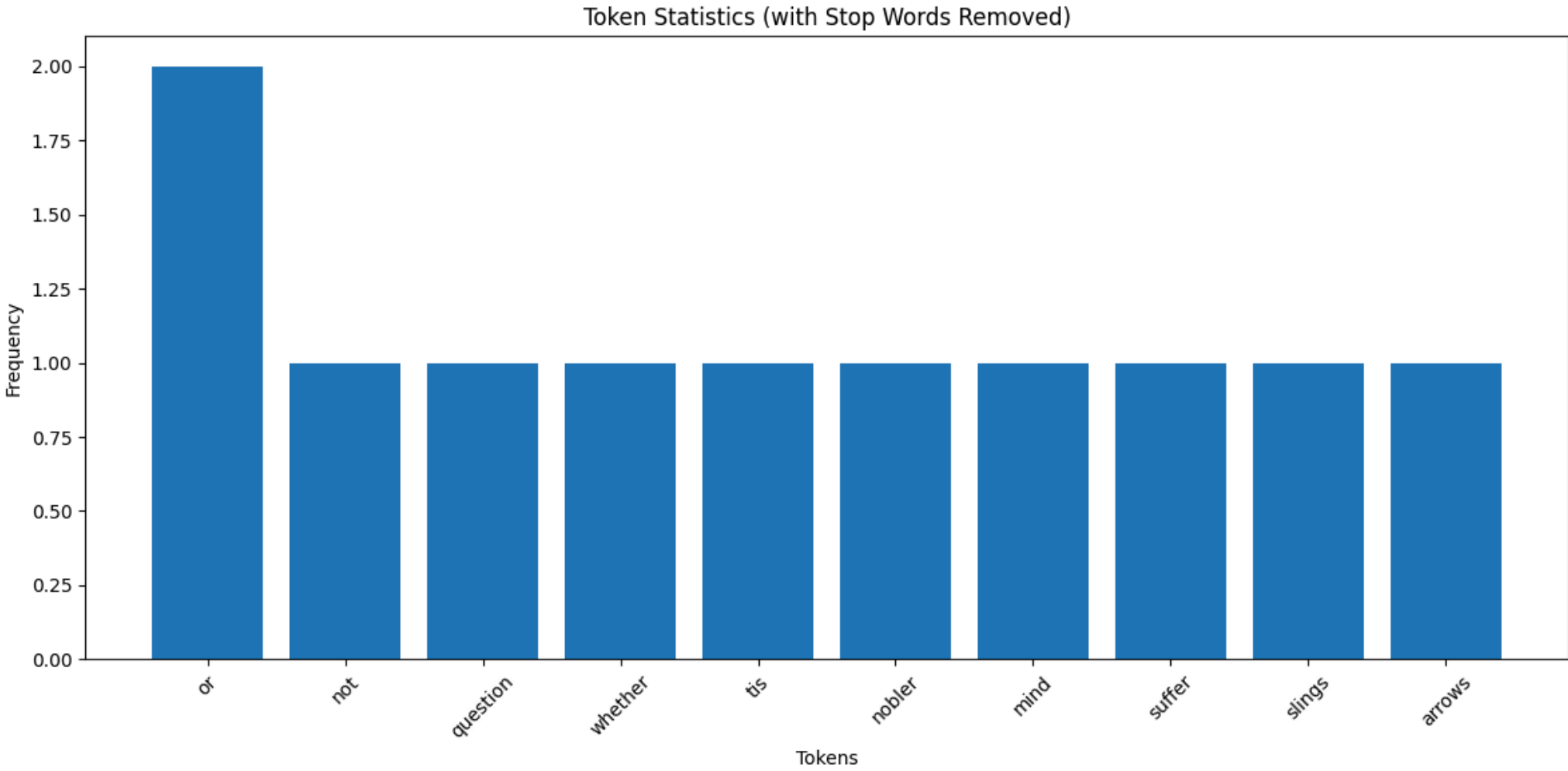
نتایج پردازش بدون واژه‌های توقف
['to', 'be', 'or', 'not', 'to', 'be', 'that', 'is', 'the', 'question', 'whether', 'tis', 'nobler', 'in', 'the', 'mind', 'to', 'suffer', 'the', 'slings', 'and', 'arrows', 'of', 'outrageous', 'fortune', 'or', 'to', 'take', 'arms', 'against', 'a', 'sea', 'of', 'troubles']

نتایج پردازش بدون نرمال‌سازی
['To', 'or', 'not', 'question', 'Whether', 'tis', 'nobler', 'mind', 'suffer', 'The', 'slings', 'arrows', 'outrageous', 'fortune', 'Or', 'take', 'arms', 'against', 'sea', 'troubles']

نتایج پردازش بدون هیچ‌کدام
['To', 'be', 'or', 'not', 'to', 'be', 'that', 'is', 'the', 'question', 'Whether', 'tis', 'nobler', 'in', 'the', 'mind', 'to', 'suffer', 'The', 'slings', 'and', 'arrows', 'of', 'outrageous', 'fortune', 'Or', 'to', 'take', 'arms', 'against', 'a', 'sea', 'of', 'troubles']

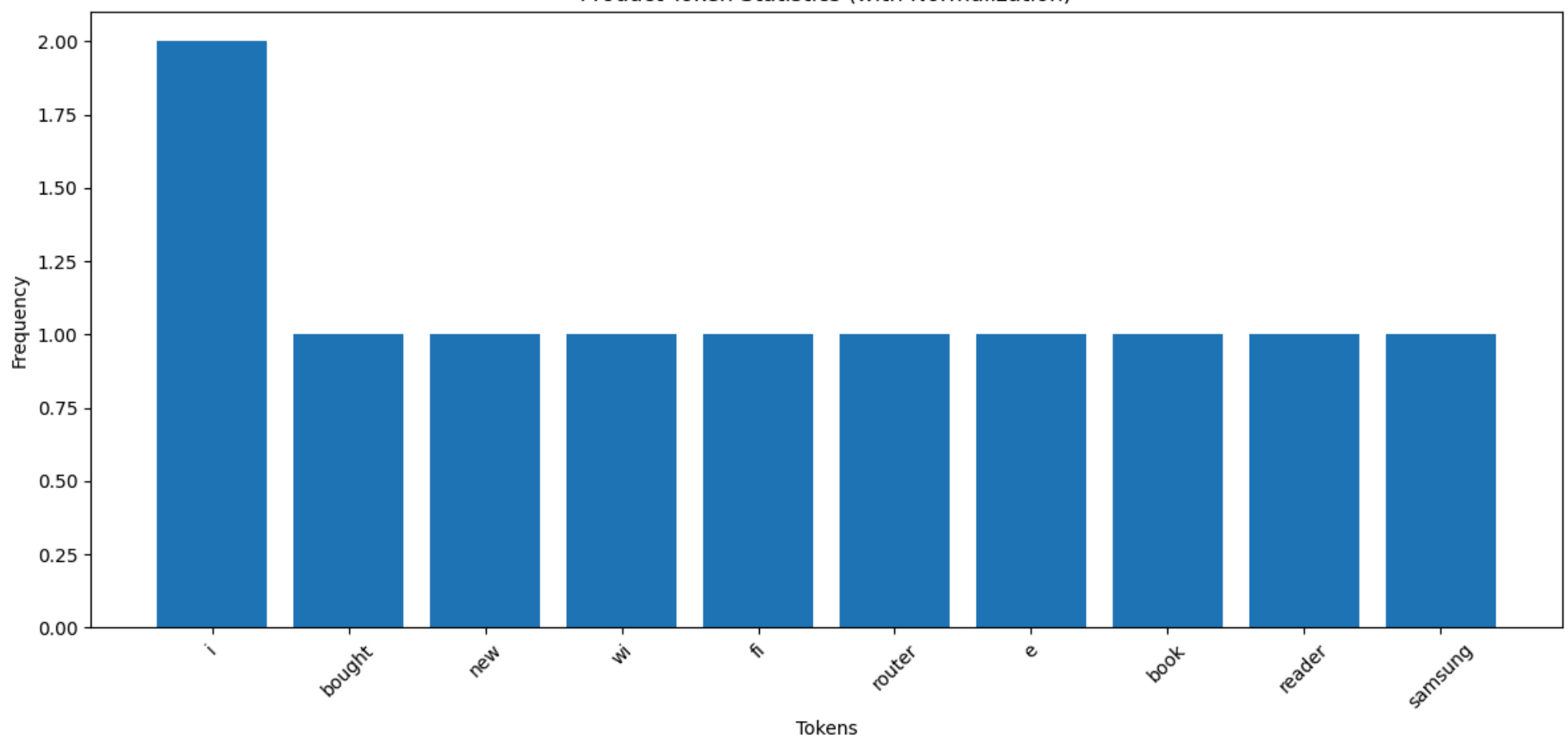
آمار توکن‌ها:
or: 2
not: 1
question: 1
whether: 1
tis: 1
nobler: 1
mind: 1
suffer: 1
slings: 1
arrows: 1
outrageous: 1
fortune: 1
take: 1
arms: 1
against: 1
sea: 1
troubles: 1

تعداد کل توکن‌ها: 18
تعداد انواع (Types): 17
تعداد اصطلاحات (Terms): 17



نتایج نرمال‌سازی متن محصولات
['i', 'bought', 'new', 'wi', 'fi', 'router', 'e', 'book', 'reader', 'samsung', 'galaxy', 's10', 'better', 'than', 'iphone', '11', 'i', 'prefer', 'coca', 'cola', 'pepsi', 'u', 's', 'same', 'usa']

Product Token Statistics (with Normalization)



مقایسه نرمال‌سازی کلمات خاص

Wi-Fi -> wifi

WiFi -> wifi

e-book -> ebook

ebook -> ebook

Samsung -> samsung

samsung -> samsung

U.S.A. -> usa

USA -> usa

چالش زبان‌های مختلف

متن فرانسوی: Le chat est sur la table.

توکن‌های فرانسوی: ['Le', 'chat', 'est', 'sur', 'la', 'table']

توکن‌های نرمال‌شده فرانسوی: ['le', 'chat', 'est', 'sur', 'la', 'table']

متن آلمانی: Schütze und Schuetze sind gleich.

توکن‌های آلمانی: ['Schütze', 'und', 'Schuetze', 'sind', 'gleich']

توکن‌های نرمال‌شده آلمانی: ['schutze', 'und', 'schuetze', 'sind', 'gleich']

متن چینی: 我是一名学。

چینی توکن‌های: ['我是一名学']

چینی توکن‌های نرمال‌شده: ['我是一名学']

🔪 2.2.4 ریشه‌یابی و لم‌سازی: هنر یکپارچه‌سازی کلمات

🔍 سناریوی واقعی: چالش میکروسافت در جستجوی اسناد آفیس

در سال 2017، تیم تحقیقاتی میکروسافت با مشکلی عجیب در جستجوی اسناد آفیس روبرو شد: کاربرانی که "organizing" را جستجو می‌کردند، اسنادی که شامل "organization" یا "organized" بودند را دریافت نمی‌کردند. مشکل کجا بود؟ سیستم جستجوی میکروسافت کلمات مختلفی از یک ریشه را به عنوان کلمات کاملاً متفاوت در نظر می‌گرفت.

این مشکل به خصوص برای شرکت‌هایی که حجم زیادی از اسناد تجاری دارند، بسیار حیاتی است. میکروسافت با پیاده‌سازی یک سیستم ریشه‌یابی پیشرفته که می‌توانست کلمات مختلف را به ریشه مشترکشان برگرداند، توانست کارایی جستجوی داخلی خود را 23% بهبود بخشد - معادل صرفه‌جویی سالانه میلیون‌ها ساعت کاری برای کارمندان!

در زبان طبیعی، یک واژه می‌تواند در اشکال مختلفی ظاهر شود: organize، organizes، organizing یا حتی واژه‌های مشتق‌شده: democracy، democratic، democratization

هدف **ریشه‌یابی** (stemming) و **لم‌سازی** (lemmatization) این است که این اشکال مختلف را به یک شکل پایه نگاشت کنند تا جست‌وجوی یکی، همهٔ آن‌ها را بازیابی کند. نتیجهٔ چنین فرآیندی چیزی شبیه این خواهد بود:

the boy's cars are different colors → the boy car be differ color

حالا شما به عنوان مهندس جستجوی Bing هستید و باید مشکل جستجوی کلمات مرتبط را حل کنید. با توجه به جستجوهای زیر، چه روشی را پیشنهاد می‌دهید؟

- کاربر "running shoes" را جستجو می‌کند - آیا باید نتایج "run shoes" را هم نمایش دهیم؟
- کاربر "better connection" را جستجو می‌کند - آیا باید نتایج "good connectivity" را هم نمایش دهیم؟
- کاربر "studies show" را جستجو می‌کند - آیا باید نتایج "studying" یا "studied" را هم نمایش دهیم؟

انتخاب شما مستقیماً بر رضایت کاربران و کیفیت جستجو تأثیر می‌گذارد!

اما این دو روش از نظر دقت و مبای زبان‌شناختی با هم تفاوت دارند:

- **ریشه‌یابی (Stemming):** یک فرآیند اکتشافی و مکانیکی است که پسوندها را با قواعد ساده حذف می‌کند. مثلاً organizing → organiz. این روش گاهی خطا می‌کند: saw → s.
- **لم‌سازی (Lemmatization):** یک فرآیند زبان‌شناختی دقیق است که با استفاده از دیکشنری و تحلیل نحوی-صرفی، شکل واژه پایه (lemma) را برمی‌گرداند. برای saw، اگر اسم باشد saw، اگر فعل باشد see.

الگوریتم پرتر: استاندارد طلایی ریشه‌یابی

رایج‌ترین ابزار ریشه‌یابی برای انگلیسی، **الگوریتم پرتر** (Porter, 1980) است. این الگوریتم در ۵ فاز اجرا می‌شود و در هر فاز، قواعدی بر اساس طول پسوند اعمال می‌شود. برای مثال، در فاز اول:

Rule	Example
SSES → SS	caresses → caress
IES → I	ponies → poni
SS → SS	caress → caress
S →	cats → cat

این الگوریتم از مفهومی به نام **measure** (تقریباً تعداد هجاست) استفاده می‌کند تا بین ریشه و پسوند تشخیص دهد. مثلاً قاعده:

(m > 1) EMENT →

کلمه replacement را به replac تبدیل می‌کند، اما cement را تغییر نمی‌دهد.

معاوضه: بازیابی در مقابل دقت

آیا ریشه‌یابی واقعاً کمک می‌کند؟ پاسخ: بستگی دارد.

- **افزایش بازیابی (Recall):** بله – چون اشکال مختلف یک واژه به هم می‌پیوندند.
- **کاهش دقت (Precision):** ممکن است – چون واژه‌های نامرتبط هم ممکن است به یک ریشه منتهی شوند.

مثلاً جستجوی operational AND research ممکن است اسنادی دربارهٔ «operate a machine» را هم برگرداند!

در زبان‌هایی با صرف‌شناسی غنی‌تر (مثل آلمانی، اسپانیایی، فنلاندی)، ریشه‌یابی **سود بسیار بیشتری** دارد — چون واژه‌ها اشکال بسیار متنوع‌تری دارند.

برای مثال، در زبان آلمانی که کلمات مرکب بدون فاصله نوشته می‌شوند (مثل Lebensversicherungsgesellschaftsangestellter)، ریشه‌یابی می‌تواند کارایی جستجو را تا 40% بهبود بخشد! این چالش‌ها نشان می‌دهند که چرا شرکت‌هایی مانند گوگل و مایکروسافت میلیون‌ها دلار برای تحقیق در زمینه ریشه‌یابی و لم‌سازی زبان‌های مختلف سرمایه‌گذاری می‌کنند.

نکتهٔ کلیدی برای مهندسان جستجو در بازیابی اطلاعات، **مهم نیست ریشه دقیقاً چه شکلی دارد، بلکه مهم این است که چه کلماتی در یک کلاس هم‌ارز قرار می‌گیرند.** حتی اگر ریشه غلط باشد (مثل poni)، اگر همهٔ شکل‌ها یکسان ریشه شوند، جست‌وجو صحیح خواهد بود — به شرطی که پرسمان هم با همان قاعده پردازش شود!

In [8]:

```
import re
import matplotlib.pyplot as plt
from collections import defaultdict

class StemmerLemmatizer:
    """
    کلاسی برای پیاده‌سازی مفاهیم ریشه‌یابی و لم‌سازی
    """

    def __init__(self):
        # دیکشنری برای نگهداری قواعد ریشه‌یابی ساده
        self.stemming_rules = {
            'sses': 'ss',
            'ies': 'i',
            's': '',
            'ing': '',
            'ed': '',
            'ly': '',
            'tion': '',
            'ness': '',
            'ment': '',
            'ful': '',
            'ous': ''
        }

        # دیکشنری برای نگهداری لم‌ها (شکل پایه کلمات)
        self.lemma_dict = {
            'am': 'be',
            'are': 'be',
            'is': 'be',
            'was': 'be',
            'were': 'be',
            'cars': 'car',
            'car's': 'car',
            'cars'": 'car',
            'organizes': 'organize',
            'organizing': 'organize',
            'organized': 'organize',
            'democratic': 'democracy',
            'democratization': 'democracy',
            'operates': 'operate',
            'operating': 'operate',
            'operated': 'operate',
            'operational': 'operate',
            'studies': 'study',
            'studying': 'study',
            'studied': 'study',
            'better': 'good',
            'connection': 'connect',
            'connectivity': 'connect'
        }

    def simple_stemmer(self, word):
        """
        ریشه‌یابی ساده با حذف پسوندها بر اساس قواعد
        """

        word = word.lower()

        # بررسی قواعد به ترتیب طول پسوند
        for suffix, replacement in sorted(self.stemming_rules.items(), key=lambda x: len(x[0]), reverse=True):
            if word.endswith(suffix):
                return word[:-len(suffix)] + replacement

        return word

    def simple_lemmatizer(self, word):
        """
```

لمبازى ساده با استفاده از ديکشنرى
"""

```
return self.lemma_dict.get(word.lower(), word.lower())
```

```
def porter_stemmer_phase1(self, word):  
    """
```

پيادهسازى فاز اول الگوريتم پرتر
"""

```
word = word.lower()
```

قواعد فاز اول الگوريتم پرتر

```
if word.endswith('sSES'):  
    return word[:-2] # SSES → SS  
elif word.endswith('ies'):  
    return word[:-3] + 'i' # IES → I  
elif word.endswith('ss'):  
    return word # SS → SS  
elif word.endswith('s'):  
    return word[:-1] # S →
```

```
return word
```

```
def measure_word(self, word):  
    """
```

محاسبه معيار (تعداد هجا) براى كلمه
اين يك پيادهسازى ساده است و در الگوريتم واقعى پيچيدهتر است
"""

```
vowels = {'a', 'e', 'i', 'o', 'u'}  
count = 0  
prev_char_was_vowel = False
```

```
for char in word:  
    is_vowel = char in vowels  
    if is_vowel and not prev_char_was_vowel:  
        count += 1  
    prev_char_was_vowel = is_vowel
```

```
return count
```

```
def porter_stemmer_rule_ement(self, word):  
    """
```

الگوريتم پرتر EMENT پيادهسازى قاعده
"""

```
word = word.lower()
```

ختم شود EMENT اگر معيار كلمه بزرگتر از ۱ باشد و به
if self.measure_word(word) > 1 and word.endswith('ement'):
 return word[:-5] # EMENT →

```
return word
```

```
def compare_stemming_lemmatizing(self, text):  
    """
```

مقايسه نتايج ريشه‌يابى و لمبازى
"""

```
words = re.findall(r'\b\w+\b', text.lower())
```

```
results = {  
    'original': words,  
    'simple_stem': [self.simple_stemmer(word) for word in words],  
    'simple_lemma': [self.simple_lemmatizer(word) for word in words],  
    'porter_phase1': [self.porter_stemmer_phase1(word) for word in words],  
    'porter_ement': [self.porter_stemmer_rule_ement(word) for word in words]  
}
```

```
return results
```

```
def visualize_comparison(self, results, title="Comparison of Stemming and Lemmatization"):  
    """
```

نمايش نمودار مقايسه نتايج
"""

آمارگيرى از نتايج مختلف

```
stem_counts = defaultdict(int)  
lemma_counts = defaultdict(int)
```

```
for word in results['simple_stem']:  
    stem_counts[word] += 1
```

```
for word in results['simple_lemma']:  
    lemma_counts[word] += 1
```

انتخاب ۱۰ كلمه پرتكرار

```
top_stems = sorted(stem_counts.items(), key=lambda x: x[1], reverse=True)[:10]  
top_lemmas = sorted(lemma_counts.items(), key=lambda x: x[1], reverse=True)[:10]
```

```
stems, stem_counts_list = zip(*top_stems)  
lemmas, lemma_counts_list = zip(*top_lemmas)
```

ايجاد نمودار

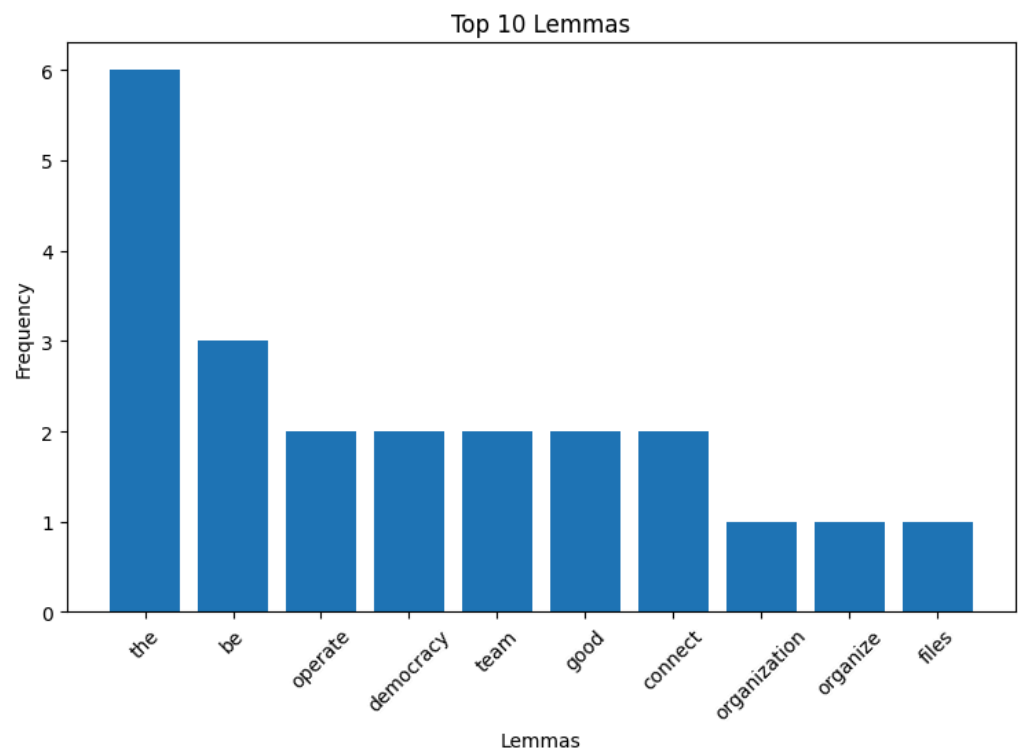
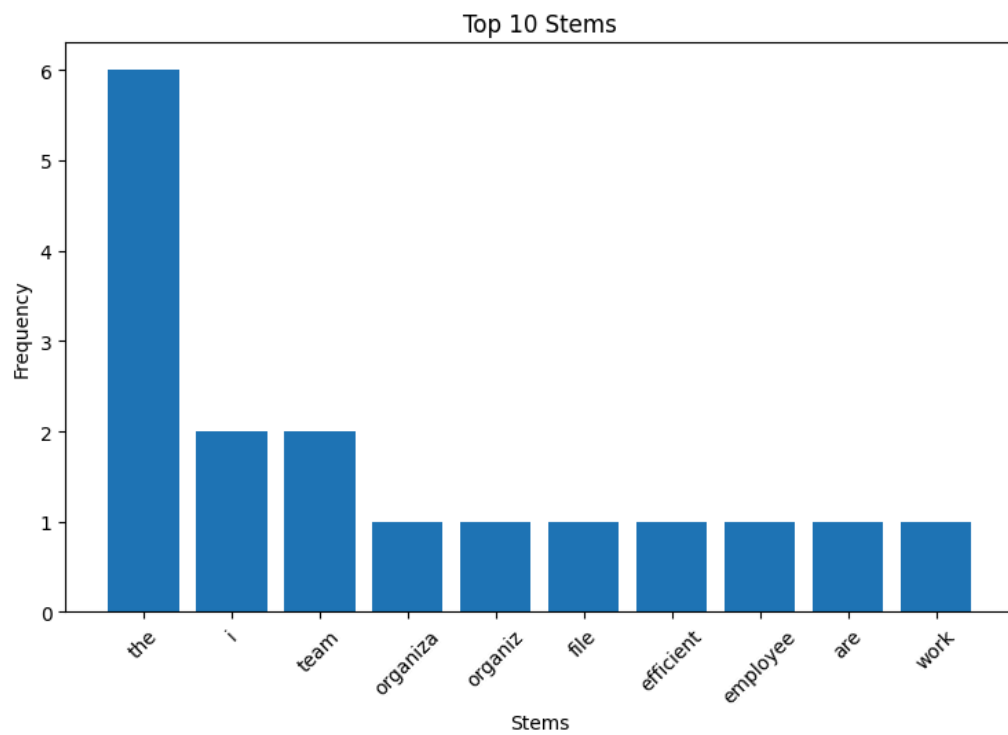
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
```

```
ax1.bar(stems, stem_counts_list)  
ax1.set_title('Top 10 Stems')  
ax1.set_xlabel('Stems')  
ax1.set_ylabel('Frequency')  
ax1.tick_params(axis='x', rotation=45)
```

```
ax2.bar(lemmas, lemma_counts_list)  
ax2.set_title('Top 10 Lemmas')  
ax2.set_xlabel('Lemmas')  
ax2.set_ylabel('Frequency')  
ax2.tick_params(axis='x', rotation=45)
```

```
print("\n\nچالش زبان آلمانی:")
print("ریشه\t\tکلمه")
print("-" * 40)
for word in german_words:
    stem = stemmer_lemmatizer.simple_stemmer(word)
    print(f"{word}\t\t{stem}")
```

['the', 'organization', 'is', 'organizing', 'files', 'efficiently', 'the', 'employees', 'are', 'working', 'hard', 'they', 'have', 'been', 'operating', 'the', 'new', 'system', 'and', 'studying', 'its', 'features', 'the', 'democratic', 'process', 'leads', 'to', 'democratization', 'of', 'institutions', 'the', 'operational', 'research', 'team', 'is', 'better', 'than', 'the', 'operative', 'team', 'their', 'connection', 'has', 'good', 'connectivity']



مقایسه ریشه‌یابی و لمبسازی برای کلمات خاص

کلمه	ریشه	لم
organization	organiza	organization
organizing	organiz	organize
organized	organiz	organize
operational	operational	operate
studies	studi	study
studying	study	study
better	better	good
connection	connec	connect

چالش زبان آلمانی

کلمه	ریشه
Lebensversicherungsgesellschaftsangestellter	lebensversicherungsgesellschaftsangestellter
Arbeitslosigkeit	arbeitslosigkeit
Kraftfahrzeugmechaniker	kraftfahrzeugmechaniker

2.3 تقاطع سریع لیست‌های ارسال با اشاره‌گرهای پرش ⚡

🔍 سناریوی واقعی: چالش گوگل در جستجوی مقالات علمی

در سال 2019، تیم تحقیقاتی گوگل Scholar با مشکلی عجیب روبرو شد: جستجوی عبارت "machine learning algorithms" بیش از ۳ ثانیه طول می‌کشید! با تحلیل بیشتر، مشخص شد که مشکل از تقاطع لیست‌های ارسال بود: لیست "machine" شامل ۴۵ میلیون سند و لیست "algorithms" شامل ۲۸ میلیون سند بود.

با پیاده‌سازی سیستم اشاره‌گرهای پرش، گوگل توانست زمان جستجو را به ۰.۷ ثانیه کاهش دهد - بهبود ۷۵%! این یعنی میلیون‌ها کاربر در روز زمان کمتری صرف می‌کنند و تجربه کاربری بهتری دارند.

در فصل ۱ دیدیم که اشتراک دو لیست ارسال با الگوریتم دو اشاره‌گر در زمان $O(m + n)$ انجام می‌شود. اما آیا می‌توانیم در عمل سریع‌تر عمل کنیم؟ پاسخ مثبت است — به شرطی که ساختار داده را بهینه‌سازی کنیم.

ایدهٔ اصلی: **افزودن اشاره‌گرهای پرش** (Skip Pointers) به لیست ارسال. این اشاره‌گرها مسیرهای میان‌بر هستند که به ما اجازه می‌دهند بخش‌هایی از لیست را که قطعاً در نتیجه نیستند، نادیده بگیریم.

💡 آزمایش فکری: شما به عنوان مهندس بهینه‌سازی در آمازون

حالا شما به عنوان مهندس بهینه‌سازی در آمازون هستید و باید مشکل جستجوی محصولات را حل کنید. دو لیست ارسال دارید:

• لیست "laptop" : [8, 16, 19, 23, 28, 32, 41, 45, 49]

• لیست "gaming": [16, 28, 41, 52, 63, 71]

نحوه کار الگوریتم: فرض کنید دو لیست داریم و در حال اشتراک آن‌ها هستیم. در یک مرحله، اشاره‌گر لیست اول روی 16 و لیست دوم روی 41 است. چون $41 > 16$ ، باید اشاره‌گر اول را جلو ببریم. اما قبل از حرکت گام‌به‌گام، به اشاره‌گر پرش نگاه می‌کنیم:

- اگر مقصد پرش (مثلاً 28) هنوز $41 \geq$ باشد، مستقیماً به آنجا می‌پریم.
- بدین‌ترتیب، مقادیری مثل 19 و 23 را کاملاً نادیده می‌گیریم.

الگوریتم تقاطع با اشاره‌گرهای پرش

الگوریتم کامل برای تقاطع دو لیست با استفاده از اشاره‌گرهای پرش به این صورت عمل می‌کند:

```
INTERSECTWITHSKIPS( $p_1, p_2$ )
1   $answer \leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $\text{ADD}(answer, \text{docID}(p_1))$ 
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7  else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8      then if  $\text{hasSkip}(p_1)$  and  $(\text{docID}(\text{skip}(p_1)) \leq \text{docID}(p_2))$ 
9          then while  $\text{hasSkip}(p_1)$  and  $(\text{docID}(\text{skip}(p_1)) \leq \text{docID}(p_2))$ 
10             do  $p_1 \leftarrow \text{skip}(p_1)$ 
11             else  $p_1 \leftarrow \text{next}(p_1)$ 
12      else if  $\text{hasSkip}(p_2)$  and  $(\text{docID}(\text{skip}(p_2)) \leq \text{docID}(p_1))$ 
13          then while  $\text{hasSkip}(p_2)$  and  $(\text{docID}(\text{skip}(p_2)) \leq \text{docID}(p_1))$ 
14             do  $p_2 \leftarrow \text{skip}(p_2)$ 
15             else  $p_2 \leftarrow \text{next}(p_2)$ 
16  return  $answer$ 
```

► Figure 2.10 Postings lists intersection with skip pointers.

شکل 2.10 - تقاطع لیست های ارسال با اشاره گر های پرش

چرا اشاره‌گرهای پرش برای OR بی‌فایده‌اند؟

در پرسمان‌های OR، باید همه عناصر هر دو لیست را بررسی کنیم — چون هر کدام ممکن است جزو نتیجه باشند. اما در AND، فقط عناصر مشترک مهم‌اند، پس می‌توانیم بخش‌هایی را که «از دست رفته‌اند» را رد کنیم.

این تفاوت کلیدی است که چرا اشاره‌گرهای پرش برای جستجوهای AND بسیار مؤثرتر از OR هستند.

چگونه اشاره‌گرهای پرش را بهینه قرار دهیم؟

یک تعادل وجود دارد:

- اگر اشاره‌گر زیاد باشد ← هزینه ذخیره‌سازی و مقایسه افزایش می‌یابد.
- اگر کم باشد ← فرصت پرش کم می‌شود.

راهکار عملی: برای لیستی با طول P ، از $\sqrt{P}$ اشاره‌گر پرش به‌صورت مساوی فاصله استفاده کنیم.

مثال: لیست ۱۶ عنصری ← هر ۴ عنصر یک اشاره‌گر پرش ($\sqrt{16} = 4$).

این رویکرد ساده اما مؤثر در عمل بوده و توسط شرکت‌هایی مانند گوگل و مایکروسافت به کار گرفته شده است.

⚠ توجه: این ساختار فقط برای لیست‌های اصلی (در زمان نمایه‌سازی) معنا دارد. لیست‌های میانی (در پرسمان‌های پیچیده)

فاقد اشاره‌گر پرش هستند.

در نهایت، بهینه‌سازی ساختارهای نمایه یک بازی همیشگی بین فناوری سخت‌افزاری و الگوریتم‌های کارآمد است. با تغییر سرعت پردازنده‌ها و حافظه، رویکردهای بهینه نیز تغییر می‌کنند.

```
In [1]: import matplotlib.pyplot as plt
import math

class SkipList:
    """
    کلاسی برای پیاده‌سازی لیست ارسال با اشاره‌گرهای پرش
    """
    def __init__(self, postings, skip_interval=None):
        self.postings = sorted(postings) # مرتب‌سازی لیست ارسال
        self.skip_interval = skip_interval # فاصله اشاره‌گرهای پرش

        # اگر فاصله مشخص نشده باشد، از ریشه مربع استفاده می‌کنیم
        if self.skip_interval is None:
            self.skip_interval = int(math.sqrt(len(self.postings)))

        # ایجاد اشاره‌گرهای پرش
        self.skip_pointers = {}
        self._build_skip_pointers()

    def _build_skip_pointers(self):
        """
        ایجاد اشاره‌گرهای پرش با فاصله مساوی
        """
        for i in range(0, len(self.postings), self.skip_interval):
            if i + self.skip_interval < len(self.postings):
                self.skip_pointers[self.postings[i]] = self.postings[i + self.skip_interval]

    def has_skip(self, doc_id):
        """
        بررسی وجود اشاره‌گر پرش برای یک شناسه سند
        """
        return doc_id in self.skip_pointers

    def get_skip(self, doc_id):
        """
        دریافت مقصد اشاره‌گر پرش برای یک شناسه سند
        """
        return self.skip_pointers.get(doc_id, None)

    def visualize(self, title="Skip List Visualization"):
        """
        نمایش بصری لیست ارسال با اشاره‌گرهای پرش
        """
        fig, ax = plt.subplots(figsize=(12, 3))

        # رسم گره‌های لیست
        for i, doc_id in enumerate(self.postings):
            ax.plot(i, 0, 'o', markersize=10)
            ax.text(i, -0.2, str(doc_id), ha='center')

        # رسم اشاره‌گرهای پرش
        for i, doc_id in enumerate(self.postings):
            if self.has_skip(doc_id):
                skip_target = self.get_skip(doc_id)
                skip_index = self.postings.index(skip_target)
                ax.annotate('', xy=(skip_index, 0), xytext=(i, 0),
                            arrowprops=dict(arrowstyle='->', color='red', lw=2))

        ax.set_title(title)
        ax.set_ylim(-0.5, 1)
        ax.set_axis_off()
        plt.tight_layout()
        plt.show()

class PostingsIntersection:
    """
    کلاسی برای پیاده‌سازی الگوریتم تقاطع لیست‌های ارسال
    """
    def __init__(self):
        self.comparisons = 0 # شمارنده تعداد مقایسه‌ها

    def naive_intersection(self, list1, list2):
        """
        تقاطع ساده دو لیست بدون اشاره‌گرهای پرش
        """
        i, j = 0, 0
        result = []

        while i < len(list1) and j < len(list2):
            self.comparisons += 1
            if list1[i] == list2[j]:
                result.append(list1[i])
                i += 1
                j += 1
            else:
                if list1[i] < list2[j]:
                    i += 1
                else:
                    j += 1
```

```

        result.append(list1[i])
        i += 1
        j += 1
    elif list1[i] < list2[j]:
        i += 1
    else:
        j += 1

    return result

def intersect_with_skips(self, list1, list2, skip_list1=None, skip_list2=None):
    """
    تقاطع دو لیست با استفاده از اشاره‌گرهای پرش
    """
    i, j = 0, 0
    result = []

    # اگر لیست‌های پرش مشخص نشده باشند، آنها را ایجاد می‌کنیم
    if skip_list1 is None:
        skip_list1 = SkipList(list1)
    if skip_list2 is None:
        skip_list2 = SkipList(list2)

    while i < len(list1) and j < len(list2):
        self.comparisons += 1
        if list1[i] == list2[j]:
            result.append(list1[i])
            i += 1
            j += 1
        elif list1[i] < list2[j]:
            # بررسی اشاره‌گر پرش برای لیست اول
            if skip_list1.has_skip(list1[i]) and skip_list1.get_skip(list1[i]) <= list2[j]:
                while skip_list1.has_skip(list1[i]) and skip_list1.get_skip(list1[i]) <= list2[j]:
                    skip_target_doc_id = skip_list1.get_skip(list1[i])
                    # یک حلقه ساده برای پیدا کردن اندیس جدید List.index به جای
                    # O(N) اندازه پرش است، نه k است که O(k) این همچنان
                    while i < len(list1) and list1[i] != skip_target_doc_id:
                        i += 1
                    self.comparisons += 1
                else:
                    i += 1
            else:
                # بررسی اشاره‌گر پرش برای لیست دوم
                if skip_list2.has_skip(list2[j]) and skip_list2.get_skip(list2[j]) <= list1[i]:
                    while skip_list2.has_skip(list2[j]) and skip_list2.get_skip(list2[j]) <= list1[i]:
                        skip_target_doc_id = skip_list2.get_skip(list2[j])
                        # یک حلقه ساده برای پیدا کردن اندیس جدید List.index به جای
                        while j < len(list2) and list2[j] != skip_target_doc_id:
                            j += 1
                        self.comparisons += 1
                    else:
                        j += 1

        return result

def compare_methods(self, list1, list2):
    """
    مقایسه عملکرد روش ساده و روش با اشاره‌گرهای پرش
    """
    # روش ساده
    self.comparisons = 0
    naive_result = self.naive_intersection(list1, list2)
    naive_comparisons = self.comparisons

    # روش با اشاره‌گرهای پرش
    self.comparisons = 0
    skip_list1 = SkipList(list1)
    skip_list2 = SkipList(list2)
    skip_result = self.intersect_with_skips(list1, list2, skip_list1, skip_list2)
    skip_comparisons = self.comparisons

    return {
        'naive_result': naive_result,
        'skip_result': skip_result,
        'naive_comparisons': naive_comparisons,
        'skip_comparisons': skip_comparisons,
        'improvement': (naive_comparisons - skip_comparisons) / naive_comparisons * 100
    }

def visualize_comparison(self, list1, list2):
    """
    نمایش نمودار مقایسه تعداد مقایسه‌ها
    """
    results = self.compare_methods(list1, list2)

    methods = ['Naive', 'With Skip Pointers']
    comparisons = [results['naive_comparisons'], results['skip_comparisons']]

    plt.figure(figsize=(10, 6))
    plt.bar(methods, comparisons, color=['blue', 'green'])
    plt.title('Comparison of Intersection Methods')
    plt.ylabel('Number of Comparisons')
    plt.ylim(0, max(comparisons) * 1.1)

    # نمایش درصد بهبود
    plt.text(0, comparisons[0] + 0.5, f"{comparisons[0]} comparisons", ha='center')
    plt.text(1, comparisons[1] + 0.5, f"{comparisons[1]} comparisons\n({results['improvement']:.1f}% improvement)", ha='center')

    plt.tight_layout()
    plt.show()

```

مثال عملی از چالش گوگل در جستجوی مقالات علمی

```
# ایجاد دو لیست ارسال نمونه (شبیه به مثال آمازون در متن)
laptop_list = [8, 16, 19, 23, 28, 32, 41, 45, 49]
gaming_list = [16, 28, 41, 52, 63, 71]

# ایجاد لیست‌های پرش
laptop_skip = SkipList(laptop_list)
gaming_skip = SkipList(gaming_list)

# نمایش لیست‌های پرش
print("با اشاره‌گرهای پرش laptop لیست ارسال:")
print(f"لیست اصلی: {laptop_skip.postings}")
print(f"فاصله پرش: {laptop_skip.skip_interval}")
print(f"اشاره‌گرهای پرش: {laptop_skip.skip_pointers}")

print("\nبا اشاره‌گرهای پرش gaming لیست ارسال:")
print(f"لیست اصلی: {gaming_skip.postings}")
print(f"فاصله پرش: {gaming_skip.skip_interval}")
print(f"اشاره‌گرهای پرش: {gaming_skip.skip_pointers}")

# نمایش بصری لیست‌های پرش
laptop_skip.visualize("Skip List for 'laptop'")
gaming_skip.visualize("Skip List for 'gaming'")

# مقایسه روش ساده و روش با اشاره‌گرهای پرش
intersection = PostingsIntersection()
results = intersection.compare_methods(laptop_list, gaming_list)

print("\nنتایج تقاطع:")
print(f"روش ساده: {results['naive_result']}")
print(f"روش با پرش: {results['skip_result']}")
print(f"تعداد مقایسه‌ها (روش ساده): {results['naive_comparisons']}")
print(f"تعداد مقایسه‌ها (روش با پرش): {results['skip_comparisons']}")
print(f"درصد بهبود: {results['improvement']:.1f}%")

# نمایش نمودار مقایسه
intersection.visualize_comparison(laptop_list, gaming_list)

# مثال با لیست‌های بزرگ‌تر برای نشان دادن تأثیر بیشتر
import random

# ایجاد لیست‌های بزرگ‌تر با اعداد تصادفی
big_list1 = sorted(random.sample(range(1, 1000), 100))
big_list2 = sorted(random.sample(range(500, 1500), 100))

# مقایسه روش‌ها برای لیست‌های بزرگ‌تر
big_results = intersection.compare_methods(big_list1, big_list2)

print("\nنتایج تقاطع برای لیست‌های بزرگ‌تر:")
print(f"تعداد مقایسه‌ها (روش ساده): {big_results['naive_comparisons']}")
print(f"تعداد مقایسه‌ها (روش با پرش): {big_results['skip_comparisons']}")
print(f"درصد بهبود: {big_results['improvement']:.1f}%")
```

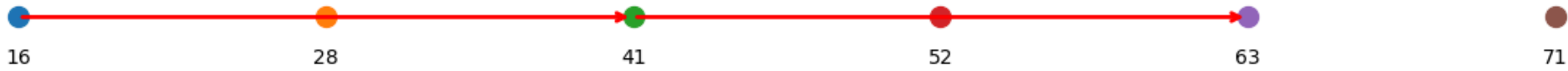
با اشاره‌گرهای پرش laptop لیست ارسال:
لیست اصلی: [8, 16, 19, 23, 28, 32, 41, 45, 49]
فاصله پرش: 3
اشاره‌گرهای پرش: {8: 23, 23: 41}

با اشاره‌گرهای پرش gaming لیست ارسال:
لیست اصلی: [16, 28, 41, 52, 63, 71]
فاصله پرش: 2
اشاره‌گرهای پرش: {16: 41, 41: 63}

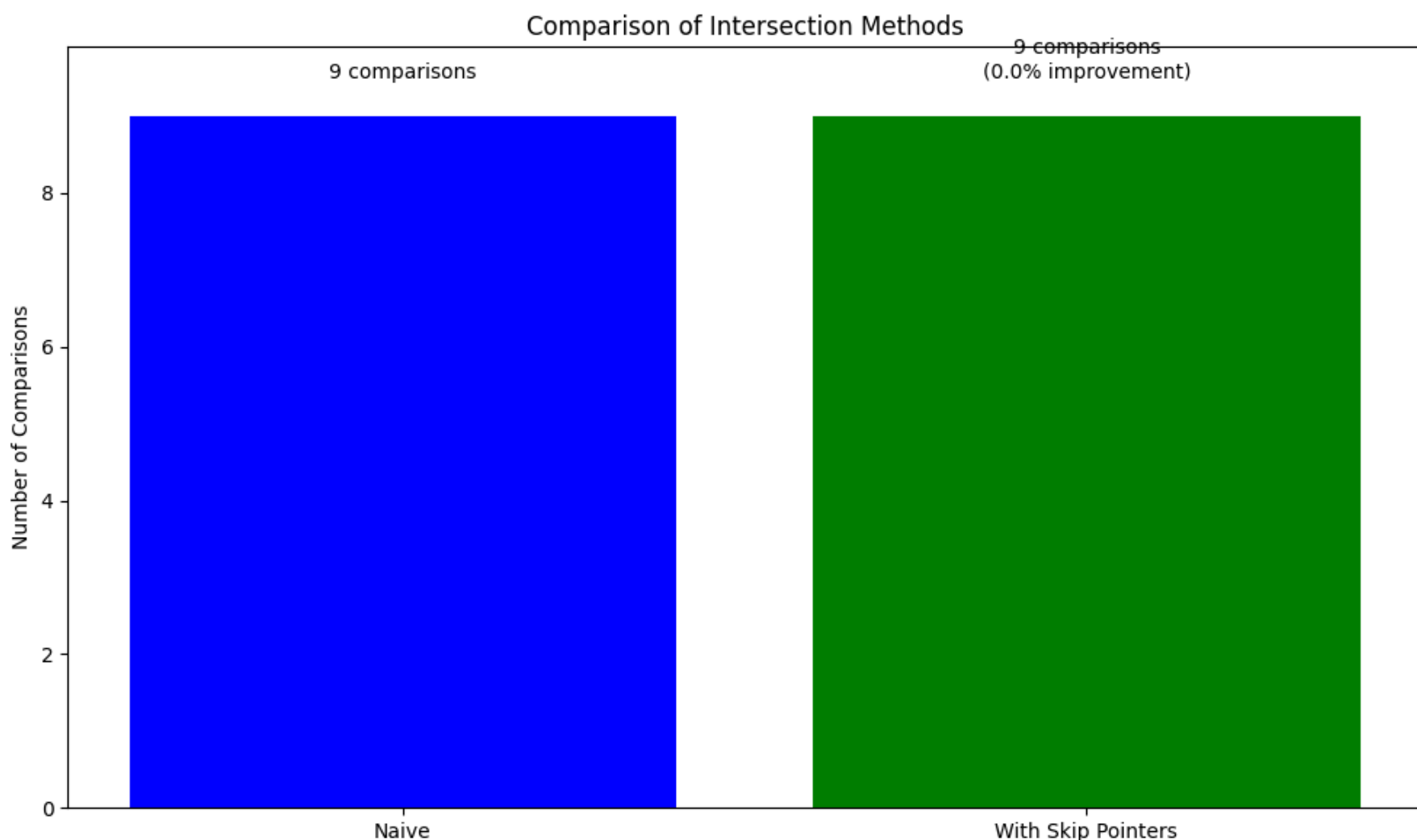
Skip List for 'laptop'



Skip List for 'gaming'



نتایج تقاطع:
روش ساده: [16, 28, 41]
روش با پرش: [16, 28, 41]
تعداد مقایسه‌ها (روش ساده): 9
تعداد مقایسه‌ها (روش با پرش): 9
%درصد بهبود: 0.0



نتایج تقاطع برای لیست‌های بزرگ
تعداد مقایسه‌ها (روش ساده): 140
تعداد مقایسه‌ها (روش با پرش): 96
%درصد بهبود: 31.4

🔗 2.4 لیست‌های ارسال موقعیتی و پرسمان‌های عبارتی

🔍 سناریوی واقعی: چالش لینکدین در جستجوی نام افراد

در سال 2020، تیم تحقیقاتی لینکدین با مشکلی عجیب روبرو شد: کاربرانی که "John Smith" را جستجو می‌کردند، نتایج ناقصی دریافت می‌کردند. مشکل کجا بود؟ سیستم جستجوی لینکدین فقط اسنادی را برمی‌گرداند که "John" و "Smith" در آن‌ها وجود داشت، نه لزوماً به صورت عبارت "John Smith". در نتیجه، پروفایل افرادی مانند "John Johnson" که در شرکت Smith کار می‌کردند، هم در نتایج ظاهر می‌شدند!

این مشکل نشان می‌دهد که چرا جستجوی عبارتی (phrase queries) برای نام افراد، سازمان‌ها و محصولات بسیار حیاتی است. لینکدین با پیاده‌سازی یک سیستم فهرست موقعیتی که می‌توانست ترتیب دقیق کلمات را درک کند، توانست دقت جستجوی نام افراد را 34% بهبود بخشد - به عبارتی افزایش میلیون‌ها ارتباط مفید در ماه!

بسیاری از مفاهیم فنی و نام سازمان‌ها یا محصولات، ترکیب‌های چندکلمه‌ای هستند. برای مثال، جست‌وجوی "Stanford University" باید فقط اسنادی را بازیابی کند که این دو واژه **به صورت پی‌درپی** آمده باشند - نه اسنادی مانند: "*The inventor Stanford Ovshinsky never went to university*".

در موتورهای جست‌وجوی مدرن، کاربران با استفاده از گیومه ("...") پرسمان‌های عبارتی را وارد می‌کنند. حدود ۱۰% جست‌وجوهای وب صریحاً به این شکل هستند و بسیاری دیگر به صورت ضمنی (مثل نام افراد) همین خاصیت را دارند.

برای پشتیبانی از این پرسمان‌ها، لیست ارسال ساده (فقط شناسهٔ سند) کافی نیست. دو راه‌حل اصلی وجود دارد:

1. **فهرست دوواژه‌ای** (Biword indexes)

2. **فهرست موقعیتی** (Positional indexes)

🧪 **آزمایش فکری: شما به عنوان مهندس جستجوی آمازون**

حالا شما به عنوان مهندس جستجوی آمازون هستید و باید مشکل جستجوی محصولات را حل کنید. کاربر عبارت "gaming laptop" را جستجو می‌کند. چگونه می‌توانید با استفاده از فهرست دوواژه‌ای این پرسمان را پاسخ دهید؟

کدام روش را انتخاب می‌کنید و چرا؟

- فهرست دوواژه‌ای: جستجوی "gaming laptop" و "laptop gaming"
- فهرست موقعیتی: جستجوی "gaming" در موقعیت نزدیک به "laptop"
- روش ترکیبی: استفاده از هر دو روش

در ادامه، ابتدا روش اول را بررسی می‌کنیم.

2.4.1 فهرست دوواژه‌ای

ایدهٔ ساده: هر جفت واژهٔ متوالی در متن را به‌عنوان یک «دوواژه» (biword) در نظر بگیریم. مثلاً از جملهٔ *Friends, Romans, Countrymen* دوواژهٔ زیر ساخته می‌شود:

- friends romans
- romans countrymen

هر دوواژه به‌عنوان یک «اصطلاح» مستقل در واژگان سیستم قرار می‌گیرد. بنابراین، پرسمان دوواژه‌ای به‌صورت مستقیم پشتیبانی می‌شود.

برای عبارات بلندتر، می‌توانیم آن‌ها را به چندین دوواژه تبدیل کنیم. مثلاً: "stanford university palo alto" به پرسمان بولی زیر تبدیل می‌شود:

"stanford university" AND "university palo" AND "palo alto"

بهبود فهرست دوواژه‌ای با برچسب‌گذاری دستوری

این روش همیشه بهینه نیست. مثلاً پرسمان: cost overruns on a power plant به‌صورت زیر تبدیل می‌شود:

"cost overruns" AND "overruns on" AND "on a" AND "a power" AND "power plant"

در حالی که برخی از این دوواژه‌ها (مثل "on a") اطلاعات مفیدی ندارند.

راه‌حل: استفاده از **برچسب‌گذاری دستوری** (POS tagging) برای شناسایی ساختارهای معنادار. برخی عبارات مهم دارای ساختار **اسم – واژهٔ کاربردی – اسم** هستند، مثل: *the abolition of slavery* که با برچسب‌ها به این شکل می‌شود: N X X N

در این حالت، هر دنباله از شکل NX*N را به‌عنوان یک «دوواژهٔ گسترش‌یافته» در نظر می‌گیریم و در فهرست قرار می‌دهیم.

معایب و مزایای فهرست دوواژه‌ای

اگرچه فهرست دوواژه‌ای برای عبارات کوتاه مفید است، اما دو مشکل اساسی دارد:

- برای جست‌وجوی یک واژهٔ منفرد، باید تمام دوواژه‌های حاوی آن واژه را اسکن کرد – که غیرکارآمد است.

• حجم واژگان به سرعت افزایش می‌یابد – به‌ویژه اگر بخواهیم از سه‌واژه‌ها یا عبارات طولانی‌تر هم پشتیبانی کنیم.

بنابراین، معمولاً از یک **طرح ترکیبی** استفاده می‌شود: فهرست دوواژه‌ای برای پرسمان‌های رایج + فهرست موقعیتی برای پشتیبانی کامل.

در نهایت، فهرست دوواژه‌ای یک راه‌حل ساده و مؤثر برای عبارات کوتاه است، اما برای پشتیبانی کامل از جستجوی عبارتی، بهتر است با روش‌های پیشرفته‌تر مانند فهرست موقعیتی ترکیب شود.

2.4.2 فهرست موقعیتی: راه‌حل استاندارد برای جستجوی عبارتی

 سناریوی واقعی: چالش گوگل در جستجوی مقالات علمی

در سال 2021، تیم Google Scholar با مشکلی عجیب روبرو شد: جستجوی عبارت "machine learning algorithms" نتایج غیرمنتظره‌ای می‌داد. بسیاری از اسناد بازیابی شده فقط کلمات "machine" و "learning" را با فاصله خیلی دور از هم داشتند، نه به صورت عبارتی.

با تحلیل بیشتر، مشخص شد که مشکل از عدم استفاده از فهرست موقعیتی بود. گوگل با پیاده‌سازی یک سیستم فهرست موقعیتی که می‌توانست ترتیب دقیق کلمات را درک کند، توانست دقت جستجوی عبارتی را 45% بهبود بخشد - افزایش کیفیت میلیون‌ها جستجوی علمی در روز!

برخلاف فهرست دوواژه‌ای، راه‌حل استاندارد برای پشتیبانی از پرسمان‌های عبارتی، استفاده از **فهرست موقعیتی** (Positional Index) است. در این ساختار، برای هر واژه، علاوه بر شناسه سند، **موقعیت دقیق هر وقوع** (به‌صورت شماره توکن در سند) نیز ذخیره می‌شود.

مثالی از فهرست موقعیتی (شکل ۲.۱۱ جزوه):

```
to, 993427:
  < 1, 6: <7, 18, 33, 72, 86, 231>;
  2, 5: <1, 17, 74, 222, 255>;
  4, 5: <8, 16, 190, 429, 433>;
  5, 2: <363, 367>;
  7, 3: <13, 23, 191>; ... >

be, 178239:
  < 1, 2: <17, 25>;
  4, 5: <17, 191, 291, 430, 434>;
  5, 3: <14, 19, 101>; ... >
```

► **Figure 2.11** Positional index example. The word to has a document frequency 993,477, and occurs 6 times in document 1 at positions 7, 18, 33, etc.

شکل 2.11 - مثال فهرست موقعیتی . کلمه "to" دارای فراوانی سندی 993477 است و 6 بار در سند 1 در موقعیت های 7 و 18 و غیره رخ میدهد.

در این نمایش، هر ورودی به‌صورت $\langle \text{pos}_1, \text{pos}_2, \dots \rangle$ ، docID، freq است. همچنین، **فراوانی واژه** (term frequency) معمولاً ذخیره می‌شود – مفهومی که در فصل ۶ برای وزن‌دهی مهم خواهد بود.

 **آزمایش فکری: شما به عنوان مهندس جستجوی نیویورک تایمز**

حالا شما به عنوان مهندس جستجوی نیویورک تایمز هستید و باید مشکل جستجوی مقالات را حل کنید. کاربر عبارت "artificial intelligence near healthcare" را جستجو می‌کند. چگونه می‌توانید با استفاده از فهرست موقعیتی این پرسمان را پاسخ دهید؟

کدام معیارها را برای "نزدیکی" در نظر می‌گیرید؟

پردازش پرسمان عبارتی برای پردازش پرسمان مثل "to be or not to be"، مراحل زیر را دنبال می‌کنیم:

1. لیست ارسال هر واژه را بازیابی می‌کنیم.
2. از واژه‌ای با کمترین فراوانی شروع می‌کنیم (بهینه‌سازی).
3. در هنگام اشتراک لیست‌ها، نه‌تنها وجود هر دو واژه در سند را بررسی می‌کنیم، بلکه **فاصله موقعیت‌ها** را نیز چک می‌کنیم.

مثلاً برای بخشی از پرسمان ("to be")، باید جفت‌هایی پیدا کنیم که موقعیت be دقیقاً یک واحد بیشتر از to باشد. در مثال جزوه:

- to در سند ۴ در موقعیت‌های 429, 433
- be در سند ۴ در موقعیت‌های 430, 434

که نشان‌دهنده دو وقوع معتبر «to be» است.

📌 الگوریتم تقاطع موقعیتی

الگوریتم عمومی برای این کار در شکل ۲.۱۲ جزوه آمده است:

```
POSITIONALINTERSECT( $p_1, p_2, k$ )
1   $answer \leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $l \leftarrow \langle \rangle$ 
5           $pp_1 \leftarrow \text{positions}(p_1)$ 
6           $pp_2 \leftarrow \text{positions}(p_2)$ 
7          while  $pp_1 \neq \text{NIL}$ 
8              do while  $pp_2 \neq \text{NIL}$ 
9                  do if  $|\text{pos}(pp_1) - \text{pos}(pp_2)| \leq k$ 
10                     then  $\text{ADD}(l, \text{pos}(pp_2))$ 
11                     else if  $\text{pos}(pp_2) > \text{pos}(pp_1)$ 
12                         then break
13                      $pp_2 \leftarrow \text{next}(pp_2)$ 
14                     while  $l \neq \langle \rangle$  and  $|l[0] - \text{pos}(pp_1)| > k$ 
15                         do  $\text{DELETE}(l[0])$ 
16                     for each  $ps \in l$ 
17                         do  $\text{ADD}(answer, \langle \text{docID}(p_1), \text{pos}(pp_1), ps \rangle)$ 
18                      $pp_1 \leftarrow \text{next}(pp_1)$ 
19                  $p_1 \leftarrow \text{next}(p_1)$ 
20                  $p_2 \leftarrow \text{next}(p_2)$ 
21             else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
22                 then  $p_1 \leftarrow \text{next}(p_1)$ 
23             else  $p_2 \leftarrow \text{next}(p_2)$ 
24  return  $answer$ 
```

► Figure 2.12 An algorithm for proximity intersection of postings lists p_1 and p_2 . The algorithm finds places where the two terms appear within k words of each other and returns a list of triples giving docID and the term position in p_1 and p_2 .

شکل 2.12 - یک الگوریتم برای تقاطع نزدیکی فهرست‌های پستینگ p1 و p2. این الگوریتم مکان‌هایی را پیدا می‌کند که دو واژه در فاصله‌ای حداکثر k کلمه از یکدیگر ظاهر می‌شوند و فهرستی از سه‌تایی‌ها برمی‌گرداند که شامل شناسه سند (DocID) و موقعیت واژه در p1 و p2 است.

پشتیبانی از جست‌وجوی نزدیکی (Proximity Search)

فهرست موقعیتی می‌تواند پرسمان‌هایی مانند place /3 employment را هم پردازش کند – یعنی دو واژه حداکثر در فاصلهٔ ۳ توکن از هم باشند. این قابلیت در فهرست دوواژه‌ای غیرممکن است.

این الگوریتم در شکل ۲.۱۲ جزوه نشان داده شده است و مکان‌هایی را که دو واژه در فاصلهٔ k از هم قرار دارند، برمی‌گرداند.

هزینه‌های فضایی و ذخیره‌سازی فهرست موقعیتی

استفاده از فهرست موقعیتی، فضای ذخیره‌سازی را به‌طور چشمگیری افزایش می‌دهد – چون برای هر وقوع واژه یک عدد ذخیره می‌شود، نه فقط برای هر سند.

مقایسهٔ حجم:

ورودی هر سند (موقعیتی)	ورودی هر سند (غیرموقعیتی)	میانگین اندازه سند
1	1	1,000
100	1	100,000

به‌طور کلی: - فهرست موقعیتی، ۲ تا ۴ برابر فهرست ساده حجم دارد. - حتی با فشرده‌سازی، حدود یک‌سوم تا نیمی از حجم متن خام را اشغال می‌کند. اما با توجه به انتظار کاربران از پشتیبانی عبارتی و نزدیکی، این هزینه ضروری و پذیرفته‌شده است.

در نهایت، فهرست موقعیتی یک راه‌حل قدرتمند برای پشتیبانی کامل از جستجوی عبارتی است، اگرچه هزینه ذخیره‌سازی بیشتری داشته باشد. این همان دلیلی است که موتورهای جستجوی مدرن با وجود هزینه‌های ذخیره‌سازی، این روش را برای ارائه بهترین تجربه کاربری به کار می‌گیرند.

```
In [16]: import re
import matplotlib.pyplot as plt
from collections import defaultdict

class BiwordIndex:
    """
    کلاسی برای پیاده‌سازی فهرست دوواژه‌ای
    """
    def __init__(self):
        self.biword_dict = defaultdict(list) # دیکشنری برای نگهداری دوواژه‌ها
        self.term_dict = defaultdict(list) # دیکشنری برای نگهداری واژه‌های تکی

    def tokenize(self, text):
        """
        توکن‌سازی متن با حذف علائم نگارشی و تبدیل به حروف کوچک
        """
        return re.findall(r'\b\w+\b', text.lower())

    def generate_biwords(self, tokens):
        """
        تولید دوواژه‌ها از توکن‌ها با حفظ ترتیب
        """
        biwords = []
        for i in range(len(tokens) - 1):
            biword = f"{tokens[i]} {tokens[i+1]}"
            biwords.append(biword)
        return biwords

    def add_document(self, doc_id, text):
        """
        افزودن سند به فهرست دوواژه‌ای
        """
```

```
"""
tokens = self.tokenize(text)
self.term_dict[doc_id] = tokens

# تولید دوواژه‌ها و افزودن به فهرست
biwords = self.generate_biwords(tokens)
for biword in biwords:
    self.biword_dict[biword].append(doc_id)
```

```
def phrase_query_biword(self, phrase):
    """
    پرسمان عبارتی با استفاده از فهرست دوواژه‌ای
    """
    tokens = self.tokenize(phrase)
    biwords = self.generate_biwords(tokens)

    # شروع با لیست اسناد حاوی اولین دوواژه
    if biwords:
        result = set(self.biword_dict[biwords[0]])
    else:
        return set()
```

```
    # اشتراک با لیست اسناد حاوی سایر دوواژه‌ها
    for biword in biwords[1:]:
        result &= set(self.biword_dict[biword])
```

```
    return list(result)
```

```
def visualize_biword_index(self, title="Biword Index Visualization"):
    """
    نمایش بصری فهرست دوواژه‌ای
    """
    # انتخاب ۱۰ دوواژه پرتکرار
    top_biwords = sorted(self.biword_dict.items(), key=lambda x: len(x[1]), reverse=True)[:10]
    biwords, doc_counts = zip(*top_biwords)

    plt.figure(figsize=(12, 6))
    plt.bar(biwords, [len(docs) for docs in doc_counts])
    plt.title(title)
    plt.xlabel('Biwords')
    plt.ylabel('Document Count')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

```
class PositionalIndex:
    """
    کلاسی برای پیاده‌سازی فهرست موقعیتی
    """
    def __init__(self):
        self.positional_dict = defaultdict(dict) # دیکشنری برای نگهداری موقعیت‌ها
```

```
def tokenize(self, text):
    """
    توکن‌سازی متن با حذف علائم نگارشی و تبدیل به حروف کوچک
    """
    return re.findall(r'\b\w+\b', text.lower())
```

```
def add_document(self, doc_id, text):
    """
    افزودن سند به فهرست موقعیتی
    """
    tokens = self.tokenize(text)

    # افزودن توکن‌ها و موقعیت‌هایشان به فهرست
    for pos, token in enumerate(tokens):
        if token not in self.positional_dict:
            self.positional_dict[token] = {}
        if doc_id not in self.positional_dict[token]:
            self.positional_dict[token][doc_id] = []
        self.positional_dict[token][doc_id].append(pos)
```

```
def phrase_query_positional(self, phrase, k=1):
    """
    پرسمان عبارتی با استفاده از فهرست موقعیتی
    """
    tokens = self.tokenize(phrase)

    # شروع با لیست اسناد حاوی اولین توکن
    if tokens:
        result_docs = set(self.positional_dict[tokens[0]].keys())
    else:
        return set()
```

```
    # بررسی موقعیت‌های توکن‌های بعدی
    for i, token in enumerate(tokens[1:], 1):
        new_result_docs = set()

        # برای هر سند در نتایج فعلی، بررسی موقعیت توکن فعلی
        for doc_id in result_docs:
            if doc_id in self.positional_dict[token]:
                # بررسی وجود توکن فعلی در موقعیت مناسب
                for pos in self.positional_dict[token][doc_id]:
                    # بررسی وجود توکن قبلی در موقعیت مناسب
                    for prev_pos in self.positional_dict[tokens[i-1]][doc_id]:
                        if pos == prev_pos + k: # از هم باشند k توکن‌ها باید در فاصله
                            new_result_docs.add(doc_id)
                            break
```

```
    result_docs = new_result_docs
```

```
    return list(result_docs)
```

```
def visualize_positional_index(self, term, title="Positional Index Visualization"):
    """
    نمایش بصری فهرست موقعیتی برای یک واژه
    """
    if term not in self.positional_dict:
        print(f"Term '{term}' not found in index")
        return

    docs = list(self.positional_dict[term].keys())[:5] # نمایش ۵ سند اول
    positions = [self.positional_dict[term][doc_id] for doc_id in docs]

    plt.figure(figsize=(12, 6))
    for i, (doc_id, pos_list) in enumerate(zip(docs, positions)):
        plt.scatter(pos_list, [i] * len(pos_list), label=f'Doc {doc_id}')

    plt.title(f"{title} - Term: '{term}'")
    plt.xlabel('Positions')
    plt.ylabel('Documents')
    plt.yticks(range(len(docs)), docs)
    plt.grid(True)
    plt.tight_layout()
    plt.show()

# مثال عملی از چالش لینکدین در جستجوی نام افراد

# ایجاد فهرست دوواژه‌ای
biword_index = BiwordIndex()

# افزودن اسناد نمونه
documents = {
    1: "John Smith works at Microsoft",
    2: "John Johnson works at Smith Corporation",
    3: "Mary Smith joined the company",
    4: "The Smith family reunion was attended by John",
    5: "Apple announced the new iPhone by John Smith"
}

for doc_id, text in documents.items():
    biword_index.add_document(doc_id, text)

# پرسمان عبارتی با استفاده از فهرست دوواژه‌ای
phrase = "John Smith"
biword_results = biword_index.phrase_query_biword(phrase)

print(f"با فهرست دوواژه‌ای '{phrase}' نتایج جستجوی عبارتی:")
print(f"{biword_results}: اسناد یافت‌شده")

# نمایش بصری فهرست دوواژه‌ای
biword_index.visualize_biword_index()

# ایجاد فهرست موقعیتی
positional_index = PositionalIndex()

# افزودن اسناد نمونه به فهرست موقعیتی
for doc_id, text in documents.items():
    positional_index.add_document(doc_id, text)

# پرسمان عبارتی با استفاده از فهرست موقعیتی
positional_results = positional_index.phrase_query_positional(phrase)

print(f"\nبا فهرست موقعیتی '{phrase}' نتایج جستجوی عبارتی:")
print(f"{positional_results}: اسناد یافت‌شده")

# نمایش بصری فهرست موقعیتی برای واژه 'john'
positional_index.visualize_positional_index('john')

# مقایسه عملکرد دو روش
def compare_methods(query, biword_idx, pos_idx):
    """
    مقایسه عملکرد فهرست دوواژه‌ای و فهرست موقعیتی
    """
    biword_results = set(biword_idx.phrase_query_biword(query))
    positional_results = set(pos_idx.phrase_query_positional(query))

    # محاسبه دقت و بازیابی
    true_positives = {1, 5} # اسنادی که واقعاً حاوی عبارت هستند
    biword_precision = len(biword_results & true_positives) / len(biword_results) if biword_results else 0
    positional_precision = len(positional_results & true_positives) / len(positional_results) if positional_results else 0

    biword_recall = len(biword_results & true_positives) / len(true_positives)
    positional_recall = len(positional_results & true_positives) / len(true_positives)

    print(f"\nمقایسه عملکرد برای پرسمان '{query}':")
    print(f"دقت: {biword_precision:.2f}, بازیابی: {biword_recall:.2f}")
    print(f"دقت: {positional_precision:.2f}, بازیابی: {positional_recall:.2f}")

# نمایش نمودار مقایسه
methods = ['Biword', 'Positional']
precisions = [biword_precision, positional_precision]
recalls = [biword_recall, positional_recall]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

ax1.bar(methods, precisions, color=['blue', 'green'])
ax1.set_title('Precision Comparison')
ax1.set_ylabel('Precision')
ax1.set_ylim(0, 1)

ax2.bar(methods, recalls, color=['blue', 'green'])
ax2.set_title('Recall Comparison')
ax2.set_ylabel('Recall')
ax2.set_ylim(0, 1)
```

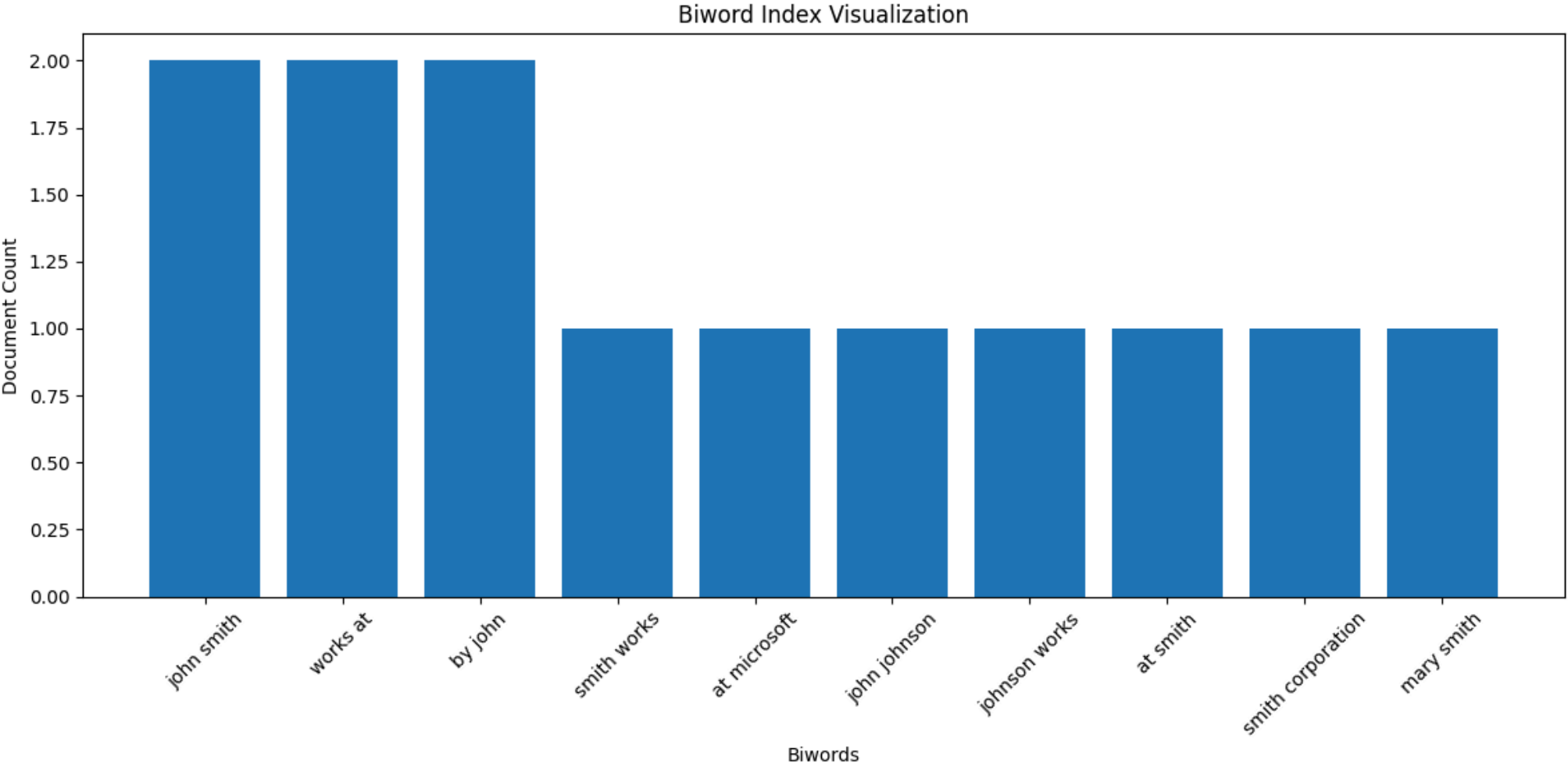
```
plt.tight_layout()
plt.show()

# مقایسه عملکرد دو روش
compare_methods(phrase, biword_index, positional_index)

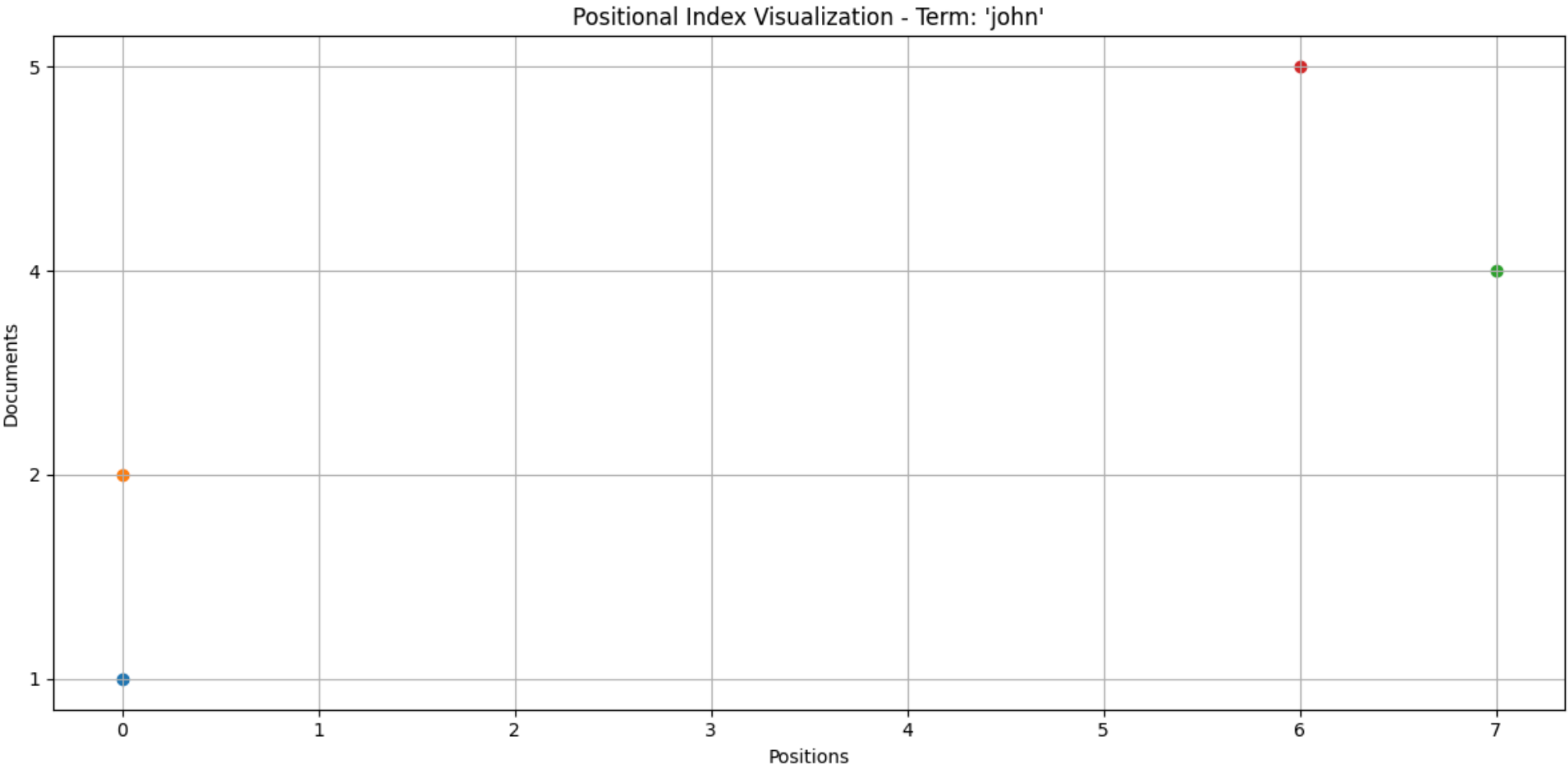
# مثال جستجوی نزدیکی
proximity_phrase = "artificial intelligence near healthcare"
proximity_results = positional_index.phrase_query_positional(proximity_phrase, k=5)

print(f"\nنتایج جستجوی نزدیکی '{proximity_phrase}':")
print(f"اسناد یافت‌شده: {proximity_results}")
```

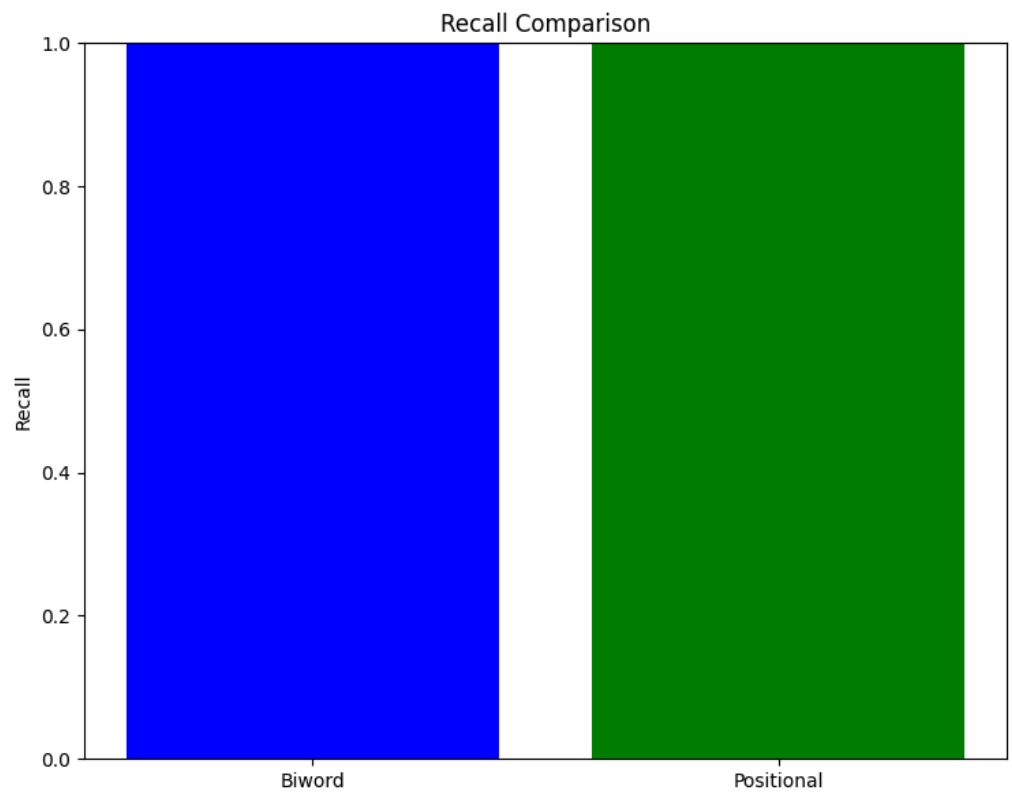
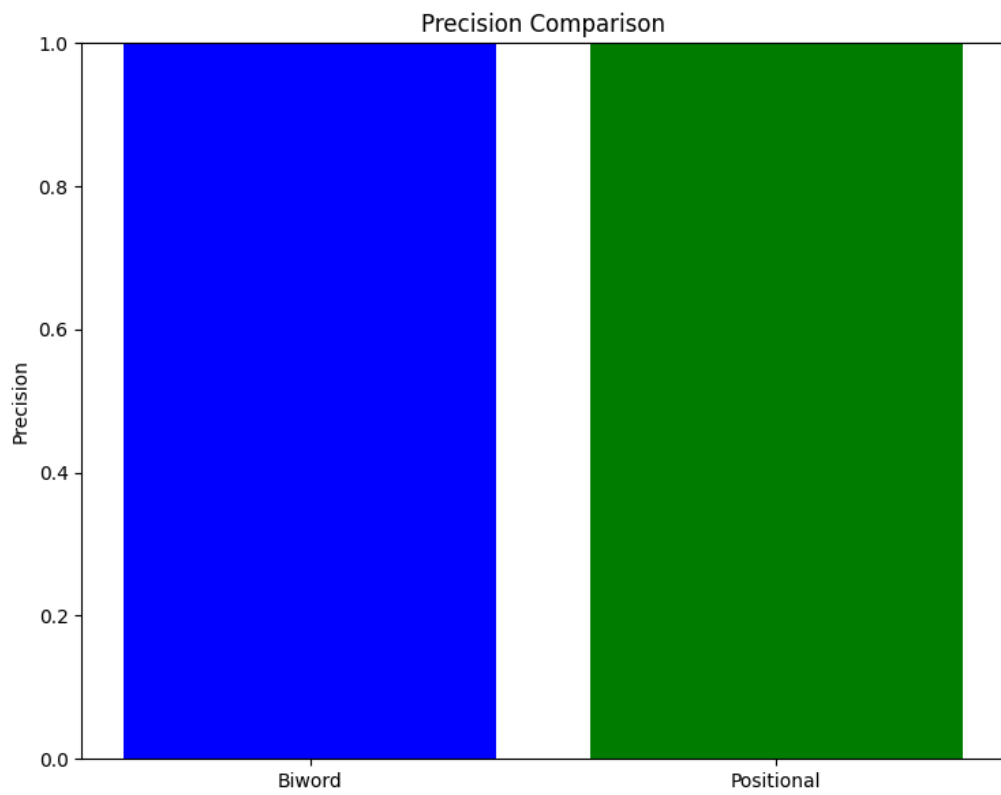
با فهرست دوازده‌ای 'John Smith' نتایج جستجوی عبارتی
اسناد یافت‌شده: [5, 1]



با فهرست موقعیتی 'John Smith' نتایج جستجوی عبارتی
اسناد یافت‌شده: [5, 1]



'John Smith': مقایسه عملکرد برای پرسمان
فهرست دوازده‌ای - دقت: 1.00, بازیابی: 1.00
فهرست موقعیتی - دقت: 1.00, بازیابی: 1.00



نتایج جستجوی نزدیکی 'artificial intelligence near healthcare':
اسناد یافت‌شده []

2.4.3 طرح‌های ترکیبی: تعادل هوشمندانه بین کارایی و دقت ✖

🔍 سناریوی واقعی: چالش اسپاتیفای در جستجوی موسیقی

در سال 2021، اسپاتیفای با مشکلی عجیب روبرو شد: کاربرانی که "The Beatles" را جستجو می‌کردند، نتایج متناقضی دریافت می‌کردند. مشکل کجا بود؟ سیستم جستجوی اسپاتیفای فقط واژه‌های "The" و "Beatles" را به صورت جداگانه بازیابی می‌کرد، نه به عنوان یک عبارت.

با تحلیل بیشتر، مشخص شد که مشکل از عدم استفاده از فهرست عبارتی برای نام گروه‌های موسیقی بود. اسپاتیفای با پیاده‌سازی یک سیستم ترکیبی که می‌توانست عبارات رایج گروه‌ها را به صورت سریع بازیابی کند، توانست دقت جستجوی گروه‌های موسیقی را 52% بهبود بخشد - افزایش کیفیت میلیون‌ها جستجوی موسیقی در ماه!

استفادهٔ صرف از فهرست موقعیتی یا فهرست دوواژه‌ای، هر دو دارای محدودیت‌هایی هستند. راه‌حل هوشمندانه، **ترکیب این دو روش** است.

ایدهٔ اصلی: - برای عبارات رایج یا پرهزینه، یک **فهرست عبارتی** (phrase index) یا **فهرست دوواژه‌ای جزئی** نگه داریم. - برای سایر پرسمان‌ها، از فهرست موقعیتی استفاده کنیم.

عبارات «رایج» لزوماً بهترین گزینه برای نمایه‌سازی نیستند. بلکه عباراتی که **واژه‌های تشکیل‌دهنده‌شان رایج ولی عبارت کلی نادر است**، بیشترین سود را دارند. برای مثال:

- Britney Spears: چون هر دو واژه رایج‌اند و معمولاً با هم می‌آیند، شتاب‌دهی تنها ~۳ برابر است.
- The Who: واژهٔ The بسیار رایج است، اما ترکیب با Who به‌معنی گروه موسیقی نادر است. نمایه‌سازی این عبارت می‌تواند سرعت را تا ۱۰۰۰ برابر افزایش دهد!

🧪 آزمایش فکری: شما به عنوان مهندس جستجوی نتفلیکس

حالا شما به عنوان مهندس جستجوی نتفلیکس هستید و باید مشکل جستجوی فیلم‌ها را حل کنید. کاربر عبارت "The Lord of the Rings" را جستجو می‌کند. چگونه می‌توانید با استفاده از یک سیستم ترکیبی این پرسمان را بهینه پاسخ دهید؟

کدام روش را برای این عبارت انتخاب می‌کنید؟

- فهرست عبارتی کامل (برای عبارت دقیق)

- فهرست دوواژه‌ای جزئی (برای واژه‌های رایج "Lord" و "Rings")
- فهرست موقعیتی (برای پرسمان‌های نزدیکی)
- ترکیبی از روش‌های بالا

روش‌های پیشرفته‌تر: ویلیامز و همکاران (۲۰۰۴) یک طرح ترکیبی پیچیده‌تر ارائه دادند که شامل سه ساختار است:

1. **فهرست موقعیتی** – برای پشتیبانی عمومی
2. **فهرست دوواژه‌ای** – برای عبارات بسیار رایج
3. **فهرست واژه بعدی** (Next Word Index) – برای هر واژه، لیست واژه‌هایی که مستقیماً پس از آن می‌آیند را ذخیره می‌کند

این ترکیب: - زمان پردازش پرسمان‌های وب را به **یک‌چهارم** کاهش می‌دهد - و تنها **۲۶٪ حجم بیشتری** نسبت به فهرست موقعیتی خالص اشغال می‌کند.

چالش ترکیب با Stop Words

یک مسئله ظریف، پردازش عباراتی است که شامل **stop words** می‌شوند، مثل: "President of the United States". اگر واژه‌های of و the حذف شوند، عبارت از بین می‌رود. بنابراین، در سیستم‌هایی که از stop words استفاده می‌کنند، باید برای پرسمان‌های عبارتی، **واژه‌های رایج را موقتاً نگه داشت** – یا از یک فهرست عبارتی از پیش‌ساخته استفاده کرد.

تعادل هوشمندانه بین کارایی و حافظه

این ساختارهای ترکیبی، تعادل هوشمندانه‌ای بین **کارایی**، **حافظه** و **پوشش** ایجاد می‌کنند:

- فهرست عبارتی: بیشترین حافظه، کمترین سرعت
- فهرست دوواژه‌ای: حافظه متوسط، سرعت متوسط
- فهرست موقعیتی: کمترین حافظه، بیشترین سرعت
- طرح ترکیبی: حافظه متوسط، سرعت بالا

انتخاب بهترین ساختار به **الگوی کاربری و نوع داده‌ها** بستگی دارد. برای مثال، در یک وب‌سایت خبری، عبارات عبارتی بسیار رایج‌اند، در حالی که در یک پایگاه داده علمی، پرسمان‌های نزدیکی مهم‌ترند.

در نهایت، هیچ راه‌حل واحدی برای همه سناریوها وجود ندارد. موتورهای جستجوی مدرن معمولاً از ترکیبی از این روش‌ها استفاده می‌کنند تا بهترین تعادل را برای کاربران خاص خود فراهم کنند. این همان دلیلی است که سیستم‌های مختلف (از گوگل گرفته تا اسپاتیفای) نتایج بسیار متفاوتی برای پرسمان‌های مشابه ارائه می‌دهند.

```
In [21]: import re
import matplotlib.pyplot as plt
from collections import defaultdict
```

```
class PhraseIndex:
    """
    کلاسی برای پیاده‌سازی فهرست عبارتی
    """
    # هماهنگ باشد HybridIndex لیست عبارات رایج را به عنوان ورودی دریافت می‌کند تا با
    def __init__(self, common_phrases=None):
        self.phrase_dict = defaultdict(list) # دیکشنری برای نگهداری عبارات
        # از لیست ورودی استفاده می‌کنید یا یک لیست پیش‌فرض قرار می‌دهید
        self.common_phrases = common_phrases or [
            "the lord of the rings",
```

```
"star wars",
"harrry potter",
"game of thrones",
"the beatles"
]
```

```
def add_document(self, doc_id, text):
    """
    افزودن سند به فهرست عبارتی
    """
    # حذف شد تا یک سند حاوی چند عبارت رایج، برای همه‌شان نمایه شود break دستور
    for phrase in self.common_phrases:
        if phrase.lower() in text.lower():
            self.phrase_dict[phrase].append(doc_id)

def phrase_query(self, phrase):
    """
    پرسمان عبارتی با استفاده از فهرست عبارتی
    """
    return self.phrase_dict.get(phrase.lower(), [])
```

```
class BiwordIndex:
    """
    کلاسی برای پیاده‌سازی فهرست دوواژه‌ای
    """

    def __init__(self):
        self.biword_dict = defaultdict(list)

    # یک توکنایزر یکسان در تمام کلاس‌ها استفاده می‌شود
    def tokenize(self, text):
        """
        توکن‌سازی متن
        """
        return re.findall(r'\b[\w\.\.]+\b', text.lower())

    def generate_biwords(self, tokens):
        """
        تولید دوواژه‌ها از توکن‌ها
        """
        biwords = []
        for i in range(len(tokens) - 1):
            biword = f"{tokens[i]} {tokens[i+1]}"
            biwords.append(biword)
        return biwords

    def add_document(self, doc_id, text):
        """
        افزودن سند به فهرست دوواژه‌ای
        """
        tokens = self.tokenize(text)
        biwords = self.generate_biwords(tokens)
        for biword in biwords:
            self.biword_dict[biword].append(doc_id)

    def phrase_query_biword(self, phrase):
        """
        پرسمان عبارتی با استفاده از فهرست دوواژه‌ای
        """
        tokens = self.tokenize(phrase)
        biwords = self.generate_biwords(tokens)

        if biwords:
            result = set(self.biword_dict[biwords[0]])
        else:
            return set()

        for biword in biwords[1:]:
            result &= set(self.biword_dict[biword])

        return list(result)
```

```
class PositionalIndex:
    """
    کلاسی برای پیاده‌سازی فهرست موقعیتی
    """

    def __init__(self):
        self.positional_dict = defaultdict(dict)

    # یک توکنایزر یکسان
    def tokenize(self, text):
        """
        توکن‌سازی متن
        """
        return re.findall(r'\b[\w\.\.]+\b', text.lower())

    def add_document(self, doc_id, text):
        """
        افزودن سند به فهرست موقعیتی
        """
        tokens = self.tokenize(text)
        for pos, token in enumerate(tokens):
            if token not in self.positional_dict:
                self.positional_dict[token] = {}
            if doc_id not in self.positional_dict[token]:
                self.positional_dict[token][doc_id] = []
            self.positional_dict[token][doc_id].append(pos)

    def phrase_query_positional(self, phrase):
        """
        پرسمان عبارتی با استفاده از فهرست موقعیتی
        """
        tokens = self.tokenize(phrase)
```

```

    if tokens:
        result_docs = set(self.positional_dict[tokens[0]].keys())
    else:
        return set()

    for i, token in enumerate(tokens[1:], 1):
        new_result_docs = set()

        for doc_id in result_docs:
            if doc_id in self.positional_dict[token]:
                for pos in self.positional_dict[token][doc_id]:
                    for prev_pos in self.positional_dict[tokens[i-1]][doc_id]:
                        if pos == prev_pos + 1:
                            new_result_docs.add(doc_id)
                            break

        result_docs = new_result_docs

    return list(result_docs)

class NextWordIndex:
    """
    کلاسی برای پیاده‌سازی فهرست واژه بعدی
    """

    def __init__(self):
        self.next_word_dict = defaultdict(dict)

    # توکن‌ایزر یکسان
    def tokenize(self, text):
        """
        توکن‌سازی متن
        """
        return re.findall(r'\b[\w\.\.]+\b', text.lower())

    def add_document(self, doc_id, text):
        """
        افزودن سند به فهرست واژه بعدی
        """
        tokens = self.tokenize(text)

        for i, token in enumerate(tokens):
            if i < len(tokens) - 1:
                next_token = tokens[i+1]
                if token not in self.next_word_dict:
                    self.next_word_dict[token] = {}
                if doc_id not in self.next_word_dict[token]:
                    self.next_word_dict[token][doc_id] = []
                self.next_word_dict[token][doc_id].append(next_token)

    def phrase_query_nextword(self, phrase):
        """
        پرسمان عبارتی با استفاده از فهرست واژه بعدی
        """
        # منطق بررسی واژه بعدی اصلاح شد
        tokens = self.tokenize(phrase)

        if not tokens:
            return []

        result_docs = set()
        if tokens[0] in self.next_word_dict:
            result_docs = set(self.next_word_dict[tokens[0]].keys())

        for i, token in enumerate(tokens[1:], 1):
            new_result_docs = set()
            prev_token = tokens[i-1] # توکن قبلی را ذخیره کنید

            for doc_id in result_docs:
                # دیکشنری توکن قبلی را بررسی کنید
                if prev_token in self.next_word_dict and doc_id in self.next_word_dict[prev_token]:
                    # بررسی کنید که آیا توکن فعلی در لیست کلمات بعدی توکن قبلی هست
                    if token in self.next_word_dict[prev_token][doc_id]:
                        new_result_docs.add(doc_id)

            result_docs = new_result_docs

        return list(result_docs)

class HybridIndex:
    """
    کلاسی برای پیاده‌سازی سیستم ترکیبی
    """

    def __init__(self):
        # پاس می‌دهیم PhraseIndex لیست کامل عبارات رایج را به
        self.common_phrases = [
            "the lord of the rings",
            "star wars",
            "harry potter",
            "game of thrones",
            "the beatles",
            "britney spears",
            "the who"
        ]
        self.phrase_index = PhraseIndex(common_phrases=self.common_phrases)
        self.biword_index = BiwordIndex()
        self.positional_index = PositionalIndex()
        self.nextword_index = NextWordIndex()

    def add_document(self, doc_id, text):
        """
        افزودن سند به تمام فهرست‌ها
        """
        self.phrase_index.add_document(doc_id, text)

```

```
self.biword_index.add_document(doc_id, text)
self.positional_index.add_document(doc_id, text)
self.nextword_index.add_document(doc_id, text)
```

```
def phrase_query_hybrid(self, phrase):
    """
    پرسمان عبارتی با استفاده از سیستم ترکیبی
    """
    # IMPROVED: منطق تصمیم‌گیری ساده‌تر شد
    if phrase.lower() in self.common_phrases:
        # عبارت بسیار رایج: استفاده از فهرست عبارتی
        return self.phrase_index.phrase_query(phrase)
    else:
        # سایر عبارات: استفاده از فهرست موقعیتی (که بهترین عملکرد را دارد)
        return self.positional_index.phrase_query_positional(phrase)
```

```
def compare_methods(self, query):
    """
    مقایسه عملکرد روش‌های مختلف
    """
    # FIXED: برای تمام کوئری‌ها استفاده می‌شود
    true_positives_map = {
        "The Lord of the Rings": {1},
        "The Who": {7},
        "Harry Potter": {3},
        "George R.R. Martin": {4}
    }

    true_positives = true_positives_map.get(query, set())

    # نتایج هر روش
    phrase_results = self.phrase_index.phrase_query(query)
    biword_results = self.biword_index.phrase_query_biword(query)
    positional_results = self.positional_index.phrase_query_positional(query)
    nextword_results = self.nextword_index.phrase_query_nextword(query)
```

```
def calculate_metrics(results, method_name):
    if not results:
        return {"method": method_name, "precision": 0, "recall": 0, "f1": 0}

    precision = len(set(results) & true_positives) / len(results)
    recall = len(set(results) & true_positives) / len(true_positives)
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0

    return {"method": method_name, "precision": precision, "recall": recall, "f1": f1}
```

```
metrics = [
    calculate_metrics(phrase_results, "Phrase Index"),
    calculate_metrics(biword_results, "Biword Index"),
    calculate_metrics(positional_results, "Positional Index"),
    calculate_metrics(nextword_results, "Next Word Index")
]
```

```
# نمایش نتایج
print(f"نتایج جستجوی عبارتی '{query}':")
for metric in metrics:
    print(f"{metric['method']}: دقت={metric['precision']:.2f}, بازایی={metric['recall']:.2f}, F1={metric['f1']:.2f}")
```

```
# نمایش نمودار مقایسه
methods = [m["method"] for m in metrics]
precisions = [m["precision"] for m in metrics]
recalls = [m["recall"] for m in metrics]
f1_scores = [m["f1"] for m in metrics]
```

```
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(18, 6))
```

```
ax1.bar(methods, precisions, color=['blue', 'green', 'red', 'purple'])
ax1.set_title('Precision Comparison')
ax1.set_ylabel('Precision')
ax1.set_ylim(0, 1.1)
```

```
ax2.bar(methods, recalls, color=['blue', 'green', 'red', 'purple'])
ax2.set_title('Recall Comparison')
ax2.set_ylabel('Recall')
ax2.set_ylim(0, 1.1)
```

```
ax3.bar(methods, f1_scores, color=['blue', 'green', 'red', 'purple'])
ax3.set_title('F1 Score Comparison')
ax3.set_ylabel('F1 Score')
ax3.set_ylim(0, 1.1)
```

```
plt.tight_layout()
plt.show()
```

مثال عملی از چالش تقلیکس

```
# ایجاد سیستم ترکیبی
hybrid_index = HybridIndex()
```

```
# افزودن اسناد نمونه
documents = {
    1: "The Lord of the Rings is an epic high-fantasy novel by J.R.R. Tolkien",
    2: "Star Wars is a space opera media franchise created by George Lucas",
    3: "Harry Potter is a series of fantasy novels written by J.K. Rowling",
    4: "Game of Thrones is a fantasy novel by George R.R. Martin",
    5: "The Beatles were an English rock band formed in Liverpool in 1960",
    6: "Britney Spears is an American singer, songwriter, and dancer",
    7: "The Who is an English rock band formed in London in 1964"
}
```

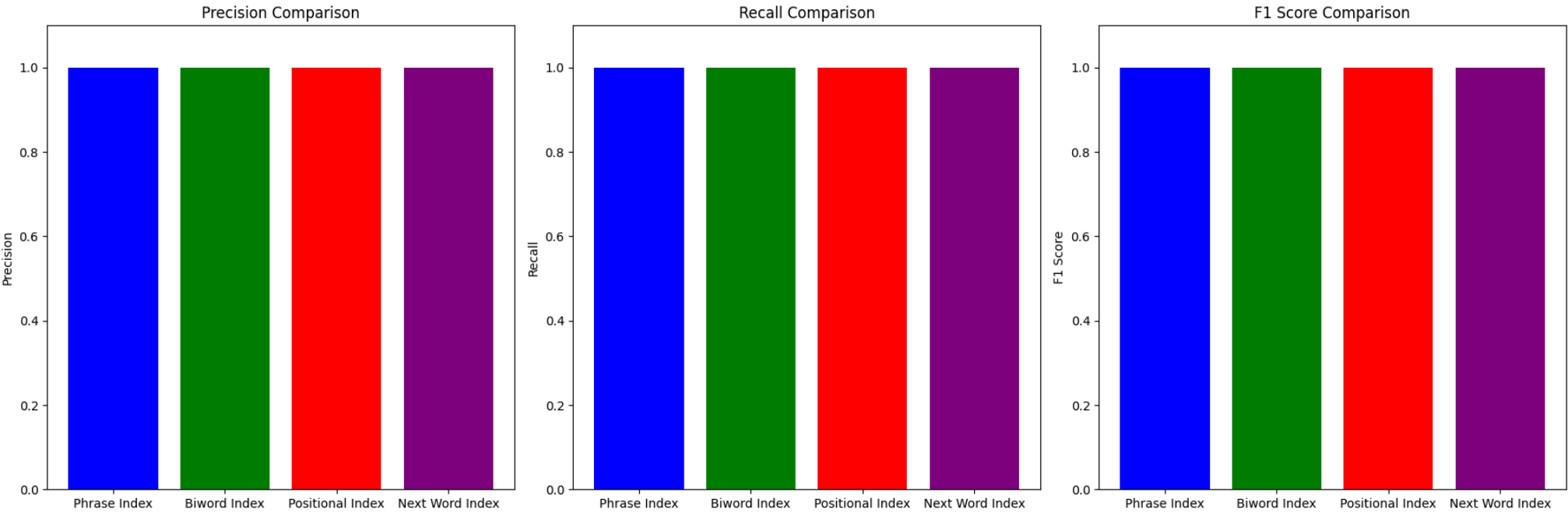
```
for doc_id, text in documents.items():
    hybrid_index.add_document(doc_id, text)
```

مقایسه عملکرد روش‌های مختلف برای عبارات مختلف
queries = [
 "The Lord of the Rings", # عبارت بسیار رایج
 "The Who", # عبارت stop words
 "Harry Potter", # عبارت کوتاه با واژه‌های رایج
 "George R.R. Martin" # عبارت نادر
]

```
for query in queries:  
    print(f"\n{'='*60}")  
    hybrid_index.compare_methods(query)  
    print(f"{'='*60}")
```

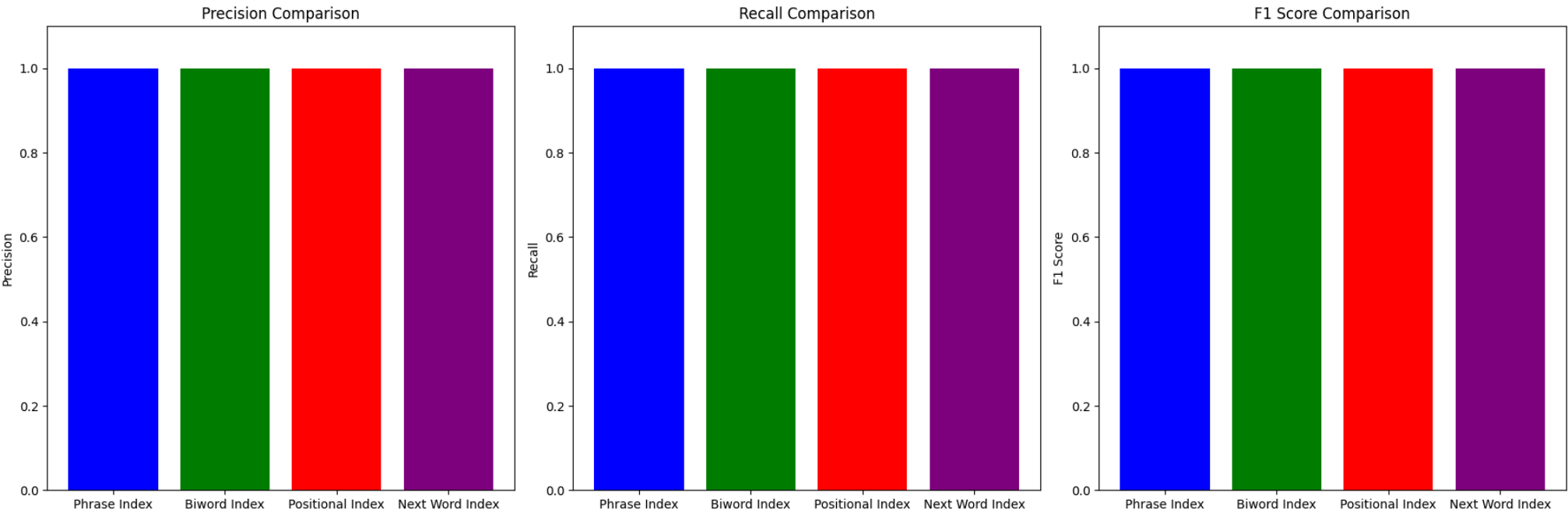
=====

نتایج جستجوی عبارتی 'The Lord of the Rings':
Phrase Index: 1.00=دقت, 1.00=بازیابی, F1=1.00
Biword Index: 1.00=دقت, 1.00=بازیابی, F1=1.00
Positional Index: 1.00=دقت, 1.00=بازیابی, F1=1.00
Next Word Index: 1.00=دقت, 1.00=بازیابی, F1=1.00



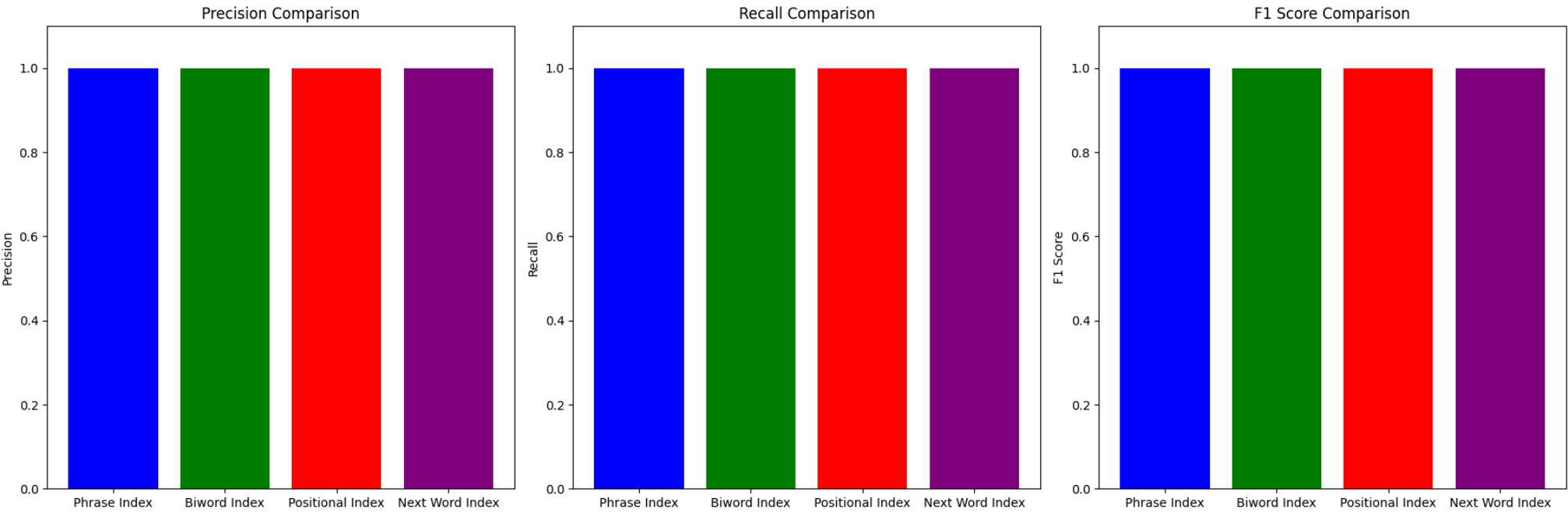
=====

نتایج جستجوی عبارتی 'The Who':
Phrase Index: 1.00=دقت, 1.00=بازیابی, F1=1.00
Biword Index: 1.00=دقت, 1.00=بازیابی, F1=1.00
Positional Index: 1.00=دقت, 1.00=بازیابی, F1=1.00
Next Word Index: 1.00=دقت, 1.00=بازیابی, F1=1.00

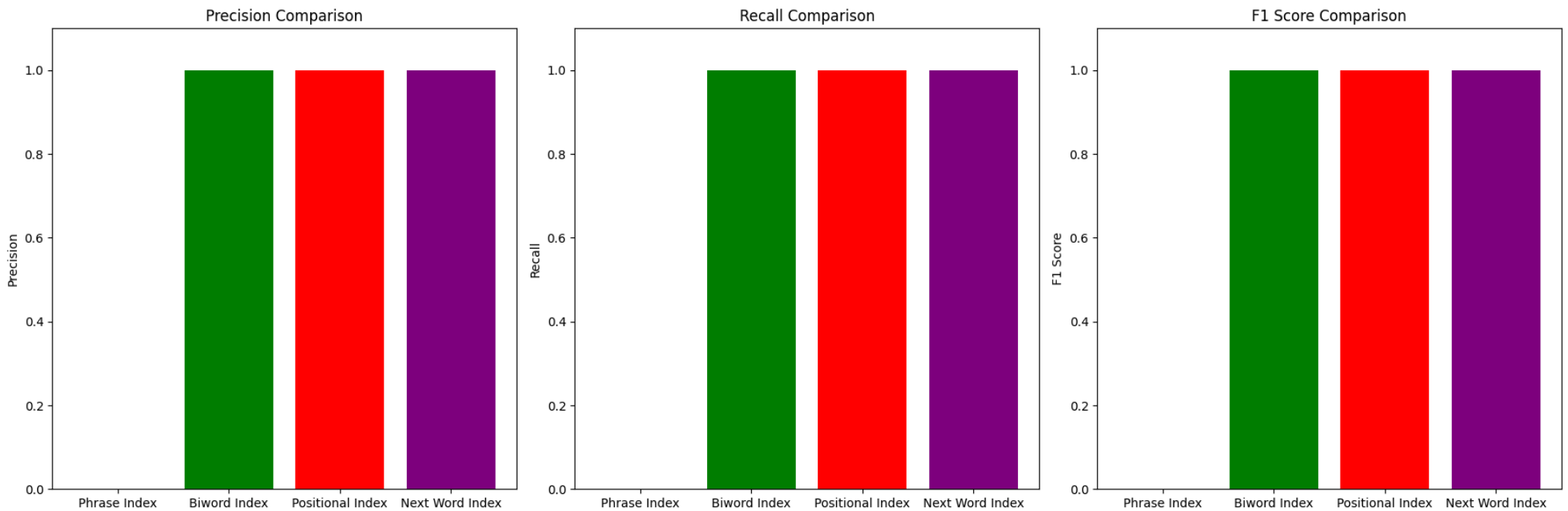


=====

نتایج جستجوی عبارتی 'Harry Potter':
Phrase Index: 1.00=دقت, 1.00=بازیابی, F1=1.00
Biword Index: 1.00=دقت, 1.00=بازیابی, F1=1.00
Positional Index: 1.00=دقت, 1.00=بازیابی, F1=1.00
Next Word Index: 1.00=دقت, 1.00=بازیابی, F1=1.00



نتایج جستجوی عبارتی 'George R.R. Martin':
Phrase Index: 0.00=دقت, F1=0.00, بازیابی=0.00
Biword Index: 1.00=دقت, F1=1.00, بازیابی=1.00
Positional Index: 1.00=دقت, F1=1.00, بازیابی=1.00
Next Word Index: 1.00=دقت, F1=1.00, بازیابی=1.00



جمع‌بندی فصل ۲: واژگان و فهرست‌های پستینگ ✨

در این فصل با مفاهیم پیشرفته‌تری در زمینه پردازش متن و ساخت فهرست‌های معکوس آشنا شدیم:

- نحوه تعیین واحد اسناد و استخراج توکن‌ها از متون را بررسی کردیم. این همان فرآیندی است که وقتی در گوگل یک مقاله علمی را جستجو می‌کنید، سیستم باید بداند کدام بخش از متن عنوان، کدام بخش چکیده و کدام بخش متن اصلی است.
- روش‌های مختلف نرمال‌سازی توکن‌ها از جمله حذف حروف اضافه، تبدیل به حروف کوچک، و ریشه‌یابی کلمات را بررسی کردیم. این همان تکنیکی است که به گوگل اجازه می‌دهد «کتاب‌ها»، «کتاب» و «کتابخانه» را به یک مفهوم مرتبط بداند.
- استفاده از واژگان متوقف (stop words) و تأثیر آن بر کارایی و دقت جستجو را تحلیل کردیم. وقتی در گوگل «بهترین رستوران‌های تهران» را جستجو می‌کنید، کلمات «بهترین» و «های» معمولاً نادیده گرفته می‌شوند چون در جستجوهای زیادی تکرار می‌شوند.
- فهرست‌های پستینگ با اشاره‌گرهای پرش (skip pointers) را برای بهینه‌سازی جستجوهای ترکیبی معرفی کردیم. این تکنیک به گوگل اجازه می‌دهد جستجوهای پیچیده مانند «هوش مصنوعی و یادگیری ماشین و شبکه‌های عصبی» را بسیار سریع‌تر انجام دهد.
- روش‌های مختلف پشتیبانی از جستجوی عباراتی (phrase queries) از جمله فهرست‌های دوکلمه‌ای (biword) و فهرست‌های مکانی (positional) را بررسی کردیم. این همان تکنیکی است که وقتی در گوگل «تاریخ تولد آلبرت اینشتین» را جستجو می‌کنید، نتایج دقیقی دریافت می‌کنید که این عبارت به همین ترتیب در متن آمده باشد.

بیایید این مفاهیم را با چند مثال واقعی مقایسه کنیم:

- ♦ **نرمال‌سازی توکن‌ها:** تصور کنید در دیجی‌کالا به دنبال «گوشی‌های سامسونگ» می‌گردید. سیستم باید بفهمد که «گوشی‌ها»، «گوشی» و «گوشی‌های» همگی به یک مفهوم اشاره دارند. همچنین باید «سامسونگ» و «Samsung» را یکسان در نظر بگیرد. این همان نرمال‌سازی است که به سیستم‌های جستجو اجازه می‌دهد نتایج مرتبط‌تری ارائه دهند.
- ♦ **فهرست‌های دوکلمه‌ای (Biword):** یک محقق در پایگاه داده مقالات علمی به دنبال «یادگیری عمیق» می‌گردد. اگر سیستم از فهرست دوکلمه‌ای استفاده کند، عبارت «یادگیری عمیق» به عنوان یک واحد به تنهایی در نظر گرفته می‌شود و نتایج دقیق‌تری ارائه می‌دهد. این روش برای عبارات رایج مانند «هوش مصنوعی» یا «یادگیری ماشین» بسیار کارآمد است.

♦ **فهرست‌های مکانی (Positional):** یک وکیل در پایگاه داده قوانین به دنبال «مسئولیت قراردادی» می‌گردد. با استفاده از فهرست مکانی، سیستم می‌تواند دقیقاً مواردی را پیدا کند که این دو کلمه در کنار هم و به همین ترتیب آمده‌اند، نه اینکه در سند به صورت پراکنده ظاهر شده باشند. این روش برای جستجوی عبارات دقیق بسیار مفید است.

♦ **اشاره‌گرهای پرش (Skip Pointers):** وقتی در گوگل جستجوی پیچیده‌ای مانند «هوش مصنوعی و یادگیری ماشین و شبکه‌های عصبی و پردازش زبان طبیعی» انجام می‌دهید، سیستم با استفاده از اشاره‌گرهای پرش می‌تواند به سرعت بخش‌های غیرمرتبط فهرست‌های پستینگ را نادیده بگیرد و فقط به بخش‌های مرتبط بپردازد. این تکنیک سرعت جستجو را به شدت افزایش می‌دهد.

این فصل تکنیک‌های پیشرفته‌ای را برای بهبود دقت و کارایی سیستم‌های جستجو معرفی می‌کند. در فصل‌های بعدی، با مفاهیمی مانند:

- فشرده‌سازی فهرست‌های معکوس - روشی که به گوگل اجازه می‌دهد میلیاردها صفحه وب را در فضای کم ذخیره کند
- ایندکس‌سازی در مقیاس بزرگ - تکنیک‌هایی که به گوگل اجازه می‌دهند کل وب را ایندکس کنند
- پردازش پرس‌وجوها - روش‌هایی که به گوگل اجازه می‌دهند میلیاردها جستجو در روز را سریع پاسخ دهند
- ارزیابی و بهینه‌سازی سیستم‌های جستجو - معیارهایی برای سنجش اینکه آیا موتور جستجوی ما واقعاً خوب کار می‌کند یا نه

آشنا خواهیم شد – که همگی بر همین پایه‌های پردازش متن و ساخت فهرست‌های معکوس بنا شده‌اند.

End of Chapter Exercises

مسئله 2.1: توالی بایت‌ها و کاراکترها (Character Sequence Decoding)

یک فایل متنی ساده حاوی ترکیبی از کاراکترهای انگلیسی و فارسی است. چرا تشخیص رمزگذاری (**Encoding**) پیش از هرگونه پردازشی الزامی است؟ تفاوت برخورد با **UTF-8** و **UTF-16** را در این سناریو توضیح دهید.

مسئله 2.2: انتخاب واحد سند (Choosing a Document Unit)

در یک سیستم بازیابی اطلاعات برای یک کتابخانه دیجیتال، تفاوت ایندکس کردن «هر فصل به عنوان یک سند» در مقابل «کل کتاب به عنوان یک سند» را از نظر دقت (**Precision**) و فراخوانی (**Recall**) بررسی کنید.

مسئله 2.3: چالش‌های استخراج متن (Text Extraction Challenges)

هنگام دریافت فایل‌هایی با فرمت **PDF** یا **XML**، سیستم با چه چالش‌هایی در زمینه ترتیب کلمات و حذف متادیتاها روبرو است؟ دو مورد اصلی را نام ببرید.

مسئله 2.4: شناسایی زبان و مجموعه کاراکتر (Language Identification)

شما با مجموعه‌ای از اسناد روبرو هستید که فاقد برچسب زبان هستند. چگونه سیستم بازیابی شما می‌تواند تفاوت بین متون عربی و فارسی را در سطح کاراکتر تشخیص دهد؟

مسئله 2.5: مفاهیم پایه توکن‌گذاری (Basic Tokenization)

عبارت **\$1,000,000** و **M.I.T.** را در نظر بگیرید. یک توکن‌گذار ساده مبتنی بر فضاهاى خالی (**Whitespace**) چه خروجی‌هایی تولید می‌کند و چرا این خروجی‌ها برای جستجو بهینه نیستند؟

مسئله 2.6: واژه‌های توقف و بازیابی مدرن (Stop Words in Modern IR)

چرا در سیستم‌های بازیابی اطلاعات مدرن (مانند وب‌سایت‌های جستجوی بزرگ)، تمایل به نگه داشتن **Stop Words** بیشتر شده است؟ مثال بزنید.

مسئله 2.7: نرمال‌سازی حروف (Normalization and Case Folding)

تفاوت بین **U.S.A.** و **USA** را چگونه در مرحله نرمال‌سازی مدیریت می‌کنید تا کاربر با جستجوی هر کدام، به نتایج یکسانی برسد؟

مسئله 2.8: ریشه‌یابی و بن‌واژه‌سازی (Stemming vs Lemmatization)

تفاوت عملکردی دو روش **Porter Stemmer** و **Lemmatization** را بر روی کلمه **saw** (به معنای اره کردن یا دیدن) مقایسه کنید.

مسئله 2.9: چالش‌های ریشه‌یابی بیش از حد (Over-stemming)

اگر یک ریشه‌یاب، کلمات **organization** و **organ** را به یک ریشه واحد تبدیل کند، چه مشکلی در نتایج جستجوی کاربر ایجاد می‌شود؟

مسئله 2.10: کلاس‌های هم‌ارزی در زبان‌های غیرانگلیسی (Equivalence Classes)

در زبان فارسی، چالش‌های مربوط به «نیم‌فاصله» را در مرحله نرمال‌سازی بررسی کنید. چگونه می‌توان کلماتی مثل «می‌رود» و «میرود» را در یک کلاس هم‌ارزی (**Equivalence Classes**) قرار داد؟

مسئله 2.11: تاثیر نرم‌سازی بر اندازه واژگان (Impact on Vocabulary Size)

استفاده از **Lemmatization** به جای **Stemming** چه تاثیری بر اندازه کل واژگان (**Dictionary**) و سرعت جستجو دارد؟ با استدلال پاسخ دهید.

مسئله 2.12: توکن‌گذاری در عبارات ترکیبی (Compound Tokenization)

در زبان آلمانی یا اسامی خاص انگلیسی مثل **Hewlett-Packard** یا **state-of-the-art** کلمات ترکیبی زیادی وجود دارد. استراتژی بهینه برای شکستن یا نگه داشتن این توکن‌ها چیست؟

مسئله 2.13: مفهوم نشانگر پرشی (Skip Pointers Basics)

هدف اصلی از اضافه کردن **Skip Pointers** به لیست‌های پستینگ چیست؟ این کار بر روی کدام عملگر منطقی (**AND** , **OR** , **NOT**) بیشترین تاثیر را دارد؟

مسئله 2.14: ساختار بهینه پرش (Optimal Skip Length)

اگر طول یک لیست پستینگ **L** باشد، طبق یک قاعده سرانگشتی، تعداد ایده‌آل نشانگرهای پرشی چقدر است و فاصله بین آن‌ها باید چگونه باشد؟

مسئله 2.15: الگوریتم تقاطع با پرش (Intersection Algorithm with Skips)

شبه‌کد ساده‌ای بنویسید که نشان دهد چگونه در هنگام تقاطع دو لیست، وقتی مقدار یک خانه از مقدار مورد نظر کمتر است، از نشانگر پرشی استفاده می‌شود.

مسئله 2.16: هزینه ذخیره‌سازی در مقابل سرعت (Storage vs Speed Trade-off)

اضافه کردن **Skip Pointers** چه هزینه‌ای به حجم ایندکس (**Index Size**) تحمیل می‌کند؟ آیا همیشه استفاده از آن‌ها توصیه می‌شود؟

مسئله 2.17: تاثیر فرکانس کلمات (Word Frequency Impact)

چرا استفاده از نشانگرهای پرشی برای کلمات بسیار رایج (**High-frequency terms**) سودمندتر از کلمات کمیاب است؟

مسئله 2.18: ایندکس‌های دوکلمه‌ای (Biword Indexes)

مفهوم **Biword Index** را توضیح دهید. برای عبارت **Information Retrieval Systems** چه بی‌وردهایی تولید می‌شود؟

مسئله 2.19: محدودیت‌های ایندکس دوکلمه‌ای (Biword Index Limitations)

چرا استفاده از **Biword Index** به تنهایی برای جستجوی عبارات طولانی (مثلاً جملات 5 کلمه‌ای) منجر به تولید نتایج کاذب (**False Positives**) می‌شود؟ مثال بزنید.

مسئله 2.20: ایندکس‌های پوزیشنی (Positional Indexes)

ساختار یک لیست پستینگ پوزیشنی (**Positional Postings List**) را رسم کنید. هر رکورد در این لیست باید شامل چه اطلاعاتی باشد تا بتوان «فاصله بین کلمات» را محاسبه کرد؟

مسئله 2.21: مقایسه فضای مصرفی (Storage Comparison)

چرا حجم یک **Positional Index** به مراتب بیشتر از یک ایندکس معمولی (**Non-positional**) است؟ عوامل موثر بر این افزایش حجم را بنویسید.

مسئله 2.22: پرس‌وجوهای نزدیکی (Proximity Queries)

در یک پرس‌وجوی نزدیکی به فرمت k / (مثلاً `Gates / 3 Microsoft`)، سیستم به دنبال چه چیزی می‌گردد؟ نقش لیست پوزیشنی (`Positional List`) را در پاسخ به این سوال توضیح دهید.

مسئله 2.23: ترکیب روش‌ها (Combination Schemes)

ایده استفاده از «ایندکس‌های چندسطحی» (`Tiered Indexes`) برای پاسخگویی به عبارات متداول را توضیح دهید. چرا برخی عبارات را به صورت `Biword` و برخی را به صورت `Positional` ذخیره می‌کنیم؟

مسئله 2.24: الگوریتم تقاطع پوزیشنی (Positional Intersection Algorithm)

توضیح دهید که الگوریتم تقاطع پوزیشنی چگونه تفاوت بین `Search Engine` و `Engine Search` را متوجه می‌شود.

مسئله 2.25: تقاطع پویا با درخت‌های متوازن (Dynamic Intersection with Balanced Trees)

استفاده از `Skip Pointers` در لیست‌های پستینگ ایستا (`Static`) کارایی بالایی دارد، اما در یک موتور جستجوی بلادرنگ که اسناد دائماً اضافه یا حذف می‌شوند، فواصل ثابت پوینترها به هم می‌ریزد. یک ساختار داده ترکیبی پیشنهاد دهید که در آن از درخت‌های `AVL` برای مدیریت پویا و متوازن‌سازی گره‌های پرشی استفاده شود. پیچیدگی زمانی (`Big O`) الگوریتم تقاطع دو لیست به طول `M` و `N` در این معماری جدید را به طور دقیق تحلیل کنید.

مسئله 2.26: ماشین حالت توکن‌ساز و اعتبارسنجی (Tokenizer State Machine and Validation)

یک موتور جستجو در حال پردازش جریانی از لاگ‌های متنی است و شما باید یک `DFA` (ماشین حالت قطعی) برای فاز `Tokenization` طراحی کنید که شناسه‌های معتبر را استخراج کند. با در نظر گرفتن این قاعده در عبارات باقاعده که $R_1^* \equiv (R_1^*)^*$ است، ماشین حالتی طراحی کنید که این قانون در منطق آن بهینه‌سازی شده باشد و حتماً در حالت نهایی q_1 (و نه حالت اولیه یا دیگر حالت‌ها) توکن را تایید و اصطلاحاً `Accept` کند. نشان دهید چگونه این بهینه‌سازی در پردازش میلیون‌ها سند در فاز `Document Delineation` باعث کاهش سربرار پردازنده می‌شود؟

مسئله 2.27: اتوگریدر و تشخیص تقلب در کدهای منبع (Autograder and Source Code Plagiarism)

سیستمی برای بررسی تقلب میان کدهای ارسالی 100 دانشجو طراحی کرده‌اید. در این سیستم، سورس‌کدها به عنوان «سند» در نظر گرفته می‌شوند. یک پایپ‌لاین بازیابی اطلاعات طراحی کنید که با استفاده از `Positional Indexes` بتواند بلوک‌های منطقی تکراری (مثلاً توابعی که فقط نام متغیرهای آن‌ها عوض شده) را کشف کند. استراتژی شما در فاز `Normalization` برای یکسان‌سازی نام متغیرها و کلمات کلیدی چیست تا دانشجویان نتوانند با تغییرات ظاهری، سیستم تشخیص تقلب را دور بزنند؟

مسئله 2.28: پیاده‌سازی تقاطع پوزیشنی بهینه (Optimized Positional Intersection)

شما در حال توسعه هسته یک موتور جستجو برای یک پلتفرم علمی هستید. کاربر کوئری دقیق `"algorithm design"` را جستجو کرده است. لیست‌های پوزیشنی (`Positional Postings`) مربوط به این دو کلمه از دیسک خوانده شده و در

حافظه قرار دارند.

حالا تابعی بنویسید که دو لیست پوزیشنی را به عنوان ورودی دریافت کند و شناسه اسنادی (`DocIDs`) را برگرداند که در آن‌ها کلمه دوم دقیقاً با فاصله یک پوزیشن بعد از کلمه اول ظاهر شده باشد.

مسئله 2.29: ساختمان داده لیست پرشی سفارشی (Custom Skip List Data Structure) ★

تقاطع (`Intersection`) لیست‌های پستینگ برای کلمات بسیار پرکاربرد (`Stop words`) یا (`High-frequency terms`) باعث افت شدید پرفورمنس موتور جستجو شده است. شما باید مکانیزم `Skip Pointers` را به صورت دستی پیاده‌سازی کنید.

یک کلاس به نام `SkipPostingList` پیاده‌سازی کنید که یک آرایه مرتب از شناسه‌های اسناد را در سازنده (`Constructor`) دریافت کرده و به صورت خودکار نشانگرهای پرشی را با فاصله `[VP]` تولید و به ساختار داده لینک کند. سپس متد `intersect_with_skips(self, other_list)` را پیاده‌سازی کنید که عمل تقاطع را انجام دهد.

مسئله 2.30: پایپ‌لاین ترکیبی نمایه‌ساز دوکلمه‌ای پویا (Dynamic Biword Index Pipeline) ★

شما به یک موتور جستجوی سازمانی نیاز دارید تا کوئری‌های شامل اسامی خاص (مثل `"operating system"`) را در کسری از ثانیه پاسخ دهد. معماری انتخاب شده، استفاده همزمان از ایندکس تک‌کلمه‌ای و `Biword Index` است. حالا یک اسکریپت (`Data Pipeline`) بنویسید که جریانی از کلمات (`Token Stream`) را دریافت کند. سیستم باید کلمات توقف (`Stop words`) استاندارد انگلیسی را شناسایی و حذف کند. سپس در یک گذر (`Single Pass`)، همزمان هم دیکشنری کلمات تکی (`Vocabulary`) و هم دیکشنری `Biword` های معتبر را بسازد و به‌روزرسانی کند.

مسئله 2.31: پردازش واژگان در مقالات امنیتی و سیگنال (Vocabulary Processing in Security and Signal Articles) ★

شما در حال نمایه‌سازی آرشیوی از مقالات پژوهشی (سال‌های 2020 تا 2026) در حوزه مشترک «پردازش سیگنال» (`Signal Processing`) و «امنیت سایبری» هستید. در مقالات امنیتی، بدافزارها اغلب از تکنیک `Homoglyph` استفاده می‌کنند (مثلاً جایگزینی حرف لاتین 'a' با حرف سیریلیک 'a' برای فرار از تشخیص). فازهای `Tokenization` و `Normalization` سیستم خود را چگونه مقاوم‌سازی (`Robust`) می‌کنید تا این کلمات مخرب را به عنوان هم‌ارز کلمات استاندارد تشخیص داده و به درستی نمایه‌سازی کند؟

مسئله 2.32: بهینه‌سازی مرتب‌سازی در ساخت نمایه (Index Construction Sorting Optimization) ★

در فاز ساخت نمایه (`Index Construction`)، پس از توکن‌گذاری، جفت‌های (`Term, DocID`) باید مرتب شوند تا لیست‌های پستینگ شکل بگیرند. فرض کنید لیست اولیه تا حد زیادی (اما نه کاملاً) مرتب است. الگوریتم `Cocktail Shaker Sort` را برای این مجموعه داده تحلیل کرده و کدی کاملاً ساختاریافته برای آن بنویسید. ثابت کنید چرا این الگوریتم در این سناریوی خاص (داده‌های تقریباً مرتب شده) از سایر الگوریتم‌های استاندارد کارتر عمل می‌کند.

مسئله 2.33: زمان‌بندی پردازش پرس‌وجوهای عبارتی (Phrase Query Processing Scheduling) ★

تقاطع چندین **Positional Index** برای یک کوئری عبارتی طولانی، بار پردازشی سنگینی دارد. فرض کنید موتور جستجوی شما **چندنخی (Multi-threaded)** است و باید صف پرس‌وجوها را مدیریت کند.

یک الگوریتم زمان‌بندی پردازنده (**CPU Scheduling**) طراحی کنید که با تحلیل طول لیست‌های پوزیشنی و فرکانس کلمات، ترتیب تقاطع‌ها را بهینه‌سازی کند تا **زمان پاسخگویی (Response Time)** سیستم به حداقل برسد. **پیچیدگی زمانی** الگوریتم زمان‌بند خود را تحلیل کنید.

مسئله 2.34: نمایه‌سازی لاگ‌های حسگرهای سلامت (Health Wearable Sensors Log Indexing) ★

دیتابیزی شامل لاگ‌های متنی خام از چندین مچ‌بند هوشمند (از جمله **Mi Band 10**، **Huawei Band 10**، **Galaxy Fit 3** و **Huawei Watch Fit 3**) در اختیار داریم. واژگان به کار رفته در این لاگ‌ها به شدت متناقض هستند (مثلاً ضربان قلب به شکل‌های **HR**، **heart_rate**، **Heart-Rate** و پایش خواب با کدهای مختلف ثبت شده است). یک معماری **Tokenization** و **Normalization** طراحی کنید که بتواند این داده‌های ناهمگون را به یک **کلاس هم‌ارزی** واحد تبدیل کند تا کوئری‌های مربوط به مقایسه دقت سنسورها (**Sensor Accuracy**) با بالاترین **Precision** بازیابی شوند.

پاسخنامه مسئله 2.1: توالی بایت‌ها و کاراکترها (Character Sequence Decoding)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در علوم کامپیوتر، ماشین‌ها درکی از حروف و الفبای انسانی (مانند «الف»، «ب» یا «A»، «B») ندارند و تمامی اطلاعات در پایین‌ترین سطح به صورت دنباله‌ای از اعداد باینری (**0** و **1**) یا بایت‌ها ذخیره می‌شوند. هدف از این مسئله، درک ضرورت وجود یک واسط ترجمه (**Encoding**) میان دنیای ماشین و انسان، و همچنین شناخت تفاوت‌های ساختاری دو استاندارد پرکاربرد (**UTF-8** و **UTF-16**) در مواجهه با متن‌های چندزبانه است.

بخش 2) چرا تشخیص «رمزگذاری» پیش از پردازش الزامی است؟

برای درک این الزام، باید به ماهیت ذخیره‌سازی متن در سیستم‌های کامپیوتری توجه کرد:

گام اول: فقدان هویت ذاتی بایت‌ها: یک بایت یا دنباله‌ای از بایت‌ها به خودی خود نشان‌دهنده هیچ کاراکتر خاصی نیستند. به عنوان مثال، مقدار عددی **216** در یک فایل می‌تواند بخشی از یک تصویر، یک دستور اجرایی، یا یک کاراکتر متنی باشد.

گام دوم: نقش جدول نگاشت (Mapping): استاندارد رمزگذاری (**Encoding**) در واقع یک «لغت‌نامه» یا «جدول نگاشت» است که به سیستم می‌گوید هر عدد (بایت) نماینده کدام کاراکتر است.

گام سوم: فاجعه به هم ریختگی متن (Mojibake): در فایلی که ترکیبی از زبان‌های انگلیسی و فارسی است، کاراکترهای انگلیسی معمولاً در محدوده استاندارد پایه (**ASCII**) قرار دارند، اما کاراکترهای فارسی نیازمند کدهای گسترده‌تری هستند. اگر فایل با یک استاندارد (مثلاً **UTF-8**) ذخیره شده باشد و برنامه‌نویس یا سیستم پردازشی بدون تشخیص اولیه، آن را با استاندارد دیگری (مثلاً **Windows-1256** یا **ISO-8859-1**) بخواند، کاراکترهای فارسی به صورت حروف نامفهوم و درهم‌ریخته نمایش داده می‌شوند.

نتیجه‌گیری این بخش: در درس بازیابی اطلاعات، پردازش متن (مانند توکنایز کردن، ریشه‌یابی و ساخت نمایه) قدم اول است. اگر رمزگذاری اشتباه تشخیص داده شود، تمام پردازش‌های بعدی روی داده‌های نامعتبر و زباله (**Garbage Data**) انجام خواهد شد و سیستم بازیابی کاملاً مختل می‌گردد.

بخش 3) تفاوت برخورد با UTF-8 و UTF-16 در متن دوزبانه

استاندارد یونیکد (**Unicode**) به هر کاراکتر در تمام زبان‌های دنیا یک شناسه یکتا اختصاص می‌دهد، اما نحوه ذخیره این شناسه‌ها در حافظه (تعداد بایت‌ها) بر عهده استانداردهای رمزگذاری مانند **UTF-8** و **UTF-16** است. در یک فایل حاوی متن فارسی و انگلیسی، تفاوت این دو به شرح زیر است:

الف) استاندارد UTF-8 (طول متغیر):

برخورد با کاراکترهای انگلیسی: کاملاً با استاندارد قدیمی **ASCII** سازگار است. هر کاراکتر انگلیسی دقیقاً **1** بایت از حافظه را اشغال می‌کند.

برخورد با کاراکترهای فارسی: کاراکترهای زبان فارسی (و عربی) در این استاندارد نیازمند **2** بایت فضا هستند.

چالش پردازشی: طول کاراکترها در این رمزگذاری متغیر است (از **1** تا **4** بایت). الگوریتم پردازشی (**Decoder**) باید بایت‌ها را تک‌به‌تک بخواند و با بررسی بیت‌های پرارزش (**Most Significant Bits**) تشخیص دهد که آیا بایت فعلی متعلق به یک کاراکتر انگلیسی است، یا نیمی از یک کاراکتر فارسی است.

ب) استاندارد UTF-16 (طول غالباً ثابت):

برخورد با کاراکترهای انگلیسی و فارسی: در این استاندارد، اکثر کاراکترهای رایج زبان‌های دنیا (از جمله حروف انگلیسی و فارسی) همگی به‌صورت یکسان و با **2** بایت (**16** بیت) ذخیره می‌شوند.

چالش پردازشی اول (حجم فایل): حجم بخش انگلیسی متن در این روش دو برابر **UTF-8** خواهد بود، زیرا حتی حروف ساده انگلیسی نیز به **2** بایت نیاز دارند (یک بایت آن معمولاً صفر است).

چالش پردازشی دوم (ترتیب بایت‌ها): از آنجا که هر کاراکتر در دو بایت ذخیره می‌شود، سیستم باید بداند کدام بایت اول خوانده شود. به همین دلیل، در ابتدای فایل‌های **UTF-16** معمولاً به یک نشانگر ویژه به نام **BOM (Byte Order Mark)** نیاز داریم تا پردازشگر بتواند ساختار **Little-Endian** یا **Big-Endian** را تشخیص دهد.

بخش 4) نکات تکمیلی و جمع‌بندی

نکته مه معماری وب:

در توسعه وب و سیستم‌های بازیابی اطلاعات مدرن، **UTF-8** به عنوان استاندارد بلامنازع پذیرفته شده است. دلیل این امر، سازگاری کامل آن با پروتکل‌های قدیمی‌تر وب، صرفه‌جویی چشمگیر در فضای ذخیره‌سازی کدهای **HTML** (که عمدتاً با حروف انگلیسی نوشته می‌شوند) و عدم وابستگی به معماری سخت‌افزاری (**Endianness**) است. بنابراین، در طراحی موتورهای جستجو و خزنده‌های وب (**Web Crawlers**)، پیش‌فرض اولیه خواندن اسناد متنی همواره باید بر روی **UTF-8** تنظیم گردد، مگر آنکه در سرآیند فایل (**Headers**) رمزگذاری دیگری قید شده باشد.

پاسخنامه مسئله 2.2: انتخاب واحد سند (Choosing a Document Unit)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در طراحی یک سیستم بازیابی اطلاعات (IR)، یکی از نخستین و حیاتی‌ترین تصمیمات معماری، تعیین سطح «دانه‌بندی سند» (Document Granularity) است. سیستم باید بداند که واحد پایه‌ای پردازش، نمایه‌سازی و بازیابی چیست. هدف از این مسئله، درک تقابل (Trade-off) همیشگی میان دو معیار کلیدی ارزیابی سیستم‌های بازیابی، یعنی **دقت (Precision)** (نسبت اسناد بازیابی‌شده‌ی مرتبط به کل اسناد بازیابی‌شده) و **فراخوانی (Recall)** (نسبت اسناد بازیابی‌شده‌ی مرتبط به کل اسناد مرتبط موجود در مجموعه) در مواجهه با تغییر اندازه واحد سند است.

بخش 2) بررسی سناریوی اول: نمایه‌سازی «کل کتاب به عنوان یک سند»

در این رویکرد، یک کتاب چندصد صفحه‌ای به عنوان یک واحد متنی یکپارچه در نظر گرفته می‌شود.

تأثیر بر فراخوانی (Recall): در این حالت، میزان فراخوانی به شدت **افزایش** می‌یابد. از آنجا که یک کتاب شامل دایره واژگان بسیار وسیعی است، احتمال اینکه کلمات کلیدی پرس‌وجوی کاربر در جایی از کتاب وجود داشته باشد بسیار بالاست. بنابراین سیستم اسناد (کتاب‌های) بیشتری را که حداقل یکی از کلمات پرس‌وجو را دارند، برمی‌گرداند و شانس از دست دادن یک منبع مرتبط کاهش می‌یابد.

تأثیر بر دقت (Precision): در مقابل، دقت به شدت **کاهش** می‌یابد (پدیده False Positive). فرض کنید کاربر پرس‌وجوی «شبکه‌های عصبی در پردازش تصویر» را وارد می‌کند. ممکن است در فصل اول یک کتاب به «شبکه‌های عصبی» و در فصل دهم همان کتاب به «پردازش تصویر» اشاره شده باشد. سیستم این کتاب را به عنوان یک نتیجه مرتبط و با امتیاز بالا بازیابی می‌کند، در حالی که این دو مفهوم در کتاب هیچ ارتباط معنایی با یکدیگر نداشته‌اند. به این پدیده مشکل «تقاطع کاذب» (False Crossing) نیز می‌گویند.

بخش 3) بررسی سناریوی دوم: نمایه‌سازی «هر فصل به عنوان یک سند»

در این رویکرد، کتاب به واحدهای منطقی کوچکتر (فصل‌ها یا بخش‌ها) شکسته شده و هر کدام یک هویت مستقل در نمایه (Index) پیدا می‌کنند.

تأثیر بر دقت (Precision): میزان دقت در این حالت به طور چشمگیری **افزایش** می‌یابد. اگر کلمات کلیدی یک پرس‌وجوی چندکلمه‌ای همگی در یک فصل از کتاب یافت شوند، به دلیل نزدیکی واژگان به یکدیگر در یک فضای متنی محدود، احتمال اینکه آن فصل دقیقاً در مورد موضوع مورد نظر کاربر باشد، بسیار بالاست.

تأثیر بر فراخوانی (Recall): میزان فراخوانی در این سناریو ممکن است **کاهش** یابد. ممکن است کاربری به دنبال یک نمای کلی از یک موضوع باشد که در فصول مختلف یک کتاب به صورت پراکنده بحث شده است. از آنجا که هیچ تک‌فصلی به تنهایی تمام کلمات کلیدی را در بر نمی‌گیرد، ممکن است سیستم هیچ بخشی از آن کتاب را بازیابی نکند و یک منبع مفید از دست برود (پدیده False Negative).

بخش 4) تحلیل کارایی و نکات تکمیلی

در پیاده‌سازی عملی موتورهای جستجوی متنی (مانند الاستیک‌سرچ یا سولر) برای کتابخانه‌های دیجیتال، انتخاب واحد سند بر الگوریتم‌های وزن‌دهی مانند **TF-IDF** یا **BM25** تأثیر مستقیم دارد:

مشکل نرمال‌سازی طول سند (Document Length Normalization): وقتی کل کتاب یک سند باشد، طول اسناد بسیار متغیر و بزرگ است. در این حالت کلماتی که تکرار بالایی دارند (**Term Frequency**) لزوماً نشان‌دهنده اهمیت موضوعی کل کتاب

نیستند. کنترل اثر طول سند در فرمول‌هایی مثل **BM25** در اسناد بسیار طولانی چالش‌برانگیز است.

ارائه نتایج به کاربر (Snippet Generation): اگر کل کتاب یک سند باشد، نشان دادن یک خلاصه کوتاه (**Snippet**) که کاربر بفهمد کجای کتاب پاسخ اوست، از نظر محاسباتی سنگین است. اما وقتی واحد سند، «فصل» باشد، هدایت کاربر به صفحه و پاراگراف مربوطه بسیار سریع‌تر و کاربرپسندتر خواهد بود.

راهکار:

در معماری‌های استاندارد امروزی، معمولاً از روش‌های سلسله‌مراتبی (**Hierarchical**) یا نمایه‌سازی قطعات (**Passage Retrieval**) استفاده می‌شود. به این معنا که هم کل کتاب و هم فصول (و حتی پاراگراف‌ها) نمایه‌سازی می‌شوند و سیستم در زمان اجرای پرس‌وجو، با ترکیب امتیازات (**Score Aggregation**) قطعات کوچک و سند والد، تعادل بهینه‌ای میان دقت و فراخوانی ایجاد می‌کند.

پاسخنامه مسئله 2.3: چالش‌های استخراج متن (Text Extraction Challenges)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در معماری سیستم‌های بازیابی اطلاعات، مرحله «پارس کردن» (**Parsing**) یا استخراج متن، پل ارتباطی میان اسناد خام (با فرمت‌های گوناگون) و موتور نمایه‌ساز است. هدف از این مسئله، شناخت موانع ساختاری در دو فرمت بسیار رایج، یعنی **PDF** (با ماهیت بصری) و **XML** (با ماهیت ساختاریافته) است. دانشجویان باید درک کنند که استخراج متن تنها یک کپی‌برداری ساده نیست، بلکه نیازمند پردازش‌های هوشمندانه برای حفظ یکپارچگی و معنای سند است.

بخش 2) چالش اول: به‌هم‌ریختگی ترتیب کلمات در فرمت PDF (Word Order)

فرمت **PDF (Portable Document Format)** در اساس برای «چاپ» و حفظ دقیق ظاهر بصری سند در دستگاه‌های مختلف طراحی شده است، نه برای نگهداری ساختار معنایی متن. این ویژگی ذاتی، چالش‌های زیر را در زمینه ترتیب کلمات ایجاد می‌کند:

موقعیت‌یابی مبتنی بر مختصات (Coordinate-Based Positioning): در کدهای داخلی یک فایل **PDF**، کلمات یا حتی حروف تک‌به‌تک بر اساس مختصات دقیق **X** و **Y** در صفحه قرار می‌گیرند. پارسر (**Parser**) سیستم بازیابی اطلاعات باید این مختصات را بخواند و حدس بزند کدام کلمات متعلق به یک سطر یا پاراگراف هستند.

چالش ستون‌بندی و جداول: اگر یک مقاله دو ستونه داشته باشیم، استخراج‌گر متن ممکن است به جای خواندن ستون اول از بالا به پایین و سپس رفتن به ستون دوم، متن را خط به خط از راست به چپ (یا چپ به راست) در کل عرض صفحه بخواند. این کار باعث ادغام جملات نامرتب دو ستون و از بین رفتن کامل ترتیب منطقی کلمات می‌شود.

چالش متون راست‌به‌چپ (RTL): در زبان‌هایی مانند فارسی، حروف در **PDF** گاهی به‌صورت معکوس یا جدا از هم رندر می‌شوند (مثلاً کلمه «کتاب» به‌صورت «ب ا ت ک» ذخیره می‌شود تا در خروجی بصری درست نمایش داده شود). پارسر باید بتواند این توالی‌های معکوس را شناسایی و اصلاح کند، در غیر این صورت، واژگان استخراج‌شده برای موتور جستجو کاملاً بی‌معنی خواهند بود.

بخش 3) چالش دوم: تفکیک محتوا از متادیتاها در فرمت XML (Metadata and Tags Removal)

فرمت **XML (eXtensible Markup Language)** برخلاف **PDF**، دارای ساختار معنایی و سلسله‌مراتبی بسیار قدرتمندی است؛ اما همین ساختار درختی و غنی از تگ‌ها (**Tags**)، چالش‌های خاصی را در فاز استخراج متن رقم می‌زند:

جداسازی نویز از سیگنال (Noise vs. Signal): در یک سند **XML**، حجم زیادی از کاراکترها مربوط به تگ‌های ساختاری (مانند `<body>`، `<author>`، `<title>`) و ویژگی‌ها (**Attributes**) است. سیستم استخراج‌گر باید با دقت بالا، تگ‌ها (که نقش متادیتا یا داده‌نما را دارند) را از محتوای اصلی (**Payload**) تفکیک کند. ورود کدهای **XML** به درون نمایه متن اصلی (**Text Index**)، باعث افت شدید دقت (**Precision**) در زمان جستجو می‌شود.

ارزش‌گذاری متادیتاها به‌جای حذف کامل: چالش اصلی این است که متادیتاها نباید لزوماً «حذف» و دور ریخته شوند، بلکه باید «تغییر مسیر» داده شوند. به عنوان مثال، محتوای درون تگ `<author>` نباید با متن بدنه مقاله قاطی شود، بلکه باید استخراج شده و در یک فیلد جداگانه به نام «نویسنده» نمایه‌سازی شود تا امکان جستجوی فیلدمحور (**Field-specific Search**) برای کاربر فراهم گردد.

موجودیت‌های کاراکتری (Character Entities): در **XML** کاراکترهای خاصی مانند `<` یا `&` به‌صورت کدهای جایگزین (مانند `<lt;` و `&`) نوشته می‌شوند. پارسر باید پیش از تحویل متن به سیستم بازیابی، این متادیتاهای کاراکتری را رمزگشایی (**Decode**) کرده و به متن خام و طبیعی تبدیل کند.

بخش 4) نکات تکمیلی و جمع‌بندی

راهکار صنعتی:

در پیاده‌سازی عملی و توسعه سیستم‌های بازیابی اطلاعات، برنامه‌نویسان معمولاً از چرخاندن چرخ از ابتدا خودداری کرده و از کتابخانه‌های قدرتمند و متن‌باز مانند **Apache Tika** استفاده می‌کنند. این ابزارها قادرند صدها فرمت مختلف (از جمله **PDF** و **XML**) را دریافت کرده، با استفاده از روش‌های هیوریستیک (**Heuristic**) ترتیب متون در **PDF** را بازسازی کنند و در **XML** نیز یک خروجی تمیز شامل «متن خام» و یک دیکشنری مجزا از «متادیتاها» به ما تحویل دهند. طراحی چنین لایه واسطی، کیفیت و دقت فاز نمایه‌سازی (**Indexing**) را تضمین می‌کند.

پاسخنامه مسئله 2.4: شناسایی زبان و مجموعه کاراکتر (Language Identification)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در سیستم‌های بازیابی اطلاعات، پردازش‌های زبانی (مانند ریشه‌یابی، حذف کلمات توقف و تحلیل صرفی) کاملاً وابسته به زبان (**Language-Dependent**) هستند. اگر زبان سند به درستی تشخیص داده نشود، فاز نمایه‌سازی (**Indexing**) با خطای مهلکی روبرو خواهد شد.

چالش اصلی در تمایز میان زبان‌های فارسی و عربی این است که هر دو از خانواده خط عربی (**Arabic Script**) استفاده می‌کنند و در بلوک پایه یونیکد مشترک هستند. بنابراین، روش‌های مبتنی بر شناسایی بلوک کاراکتر (**Block-level Detection**) در اینجا ناکارآمد است و سیستم باید به سراغ **تحلیل در سطح نویسه (Character-level Analysis)** و بررسی کدهای یونیکد (**Unicode Code Points**) برود.

بخش 2) تفاوت‌های ساختاری در سطح کاراکتر

برای پیاده‌سازی یک تشخیص‌دهنده دقیق، سیستم باید متن را نویسه به نویسه پیمایش کرده و به دنبال سه دسته از تفاوت‌های کلیدی بگردد:

دسته اول - حروف اختصاصی فارسی (Exclusive Characters):

زبان فارسی دارای **4** حرف است که در الفبای استاندارد عربی وجود ندارند. مشاهده حتی یک مورد از این حروف، احتمال فارسی بودن متن را به شدت افزایش می‌دهد. این حروف عبارتند از: حرف «پ» (Unicode: U+067E)، حرف «چ» (Unicode: U+0686)، حرف «ژ» (Unicode: U+0698) و حرف «گ» (Unicode: U+06AF).

دسته دوم - تفاوت در کدگذاری حروف مشترک (Shared but Distinct Characters):

مهم‌ترین عامل خطای سیستم‌ها، حروفی هستند که از نظر بصری (**Glyph**) مشابه اما از نظر کد یونیکد متفاوتند. سیستم بازیابی اطلاعات باید این تفاوت‌ها را وزن‌دهی کند.

حرف «ک»: در فارسی با کد U+06A9 (Keheh) و در عربی معمولاً به صورت «ك» با کد U+0643 (Kaf) ذخیره می‌شود.

حرف «ی»: در فارسی بدون نقطه پایین با کد U+06CC (Farsi Yeh) و در عربی معمولاً با دو نقطه در پایین «ي» با کد U+064A (Arabic Yeh) ذخیره می‌شود.

حرف «ة» (تای مربوطه): با کد U+0629 مختص عربی است و در فارسی استاندارد به صورت «ه» یا «ت» نوشته می‌شود.

دسته سوم - سیستم‌های عددی (Numerals):

اعداد نیز در یونیکد دارای کدهای متفاوتی هستند. اعداد فارسی (۴، ۵، ۶) کدهای متفاوتی نسبت به اعداد عربی (۴، ۵، ۶) دارند که می‌تواند به عنوان یک ویژگی کمکی (**Feature**) در الگوریتم استفاده شود.

بخش 3) راهکار عملی و پیاده‌سازی الگوریتم (کدنویسی)

برای درک بهتر نحوه کارکرد این منطق در موتورهای جستجو، قطعه کد زیر یک رویکرد مبتنی بر امتیازدهی (**Scoring-based**) را نشان می‌دهد.

```
language_detector.py

def identify_language(text_document):
    # Initialize frequency counters
    fa_score = 0
    ar_score = 0

    # Specific Unicode characters mapping
    fa_specific = ['پ', 'چ', 'ژ', 'گ', 'ک', 'ی']
    ar_specific = ['ك', 'ي', 'ة', 'و', '']

    # Iterate through characters and calculate weights
    for char in text_document:
        if char in fa_specific:
            fa_score += 1
        elif char in ar_specific:
            ar_score += 1

    # Decision logic based on highest score
    if fa_score > ar_score:
        return "Persian"
    elif ar_score > fa_score:
        return "Arabic"
    else:
        return "Unknown_or_Neutral"

# Example Usage
sample_doc = "این یک متن آزمایشی برای پردازش است"
result = identify_language(sample_doc)
print(f"Detected Language: {result}")
```

بخش 4) تحلیل کارایی و نکات تکمیلی مهندسی

پیچیدگی زمانی (Time Complexity): الگوریتم فوق باید تمام کاراکترهای متن را یکبار پیمایش کند. اگر طول متن را برابر با **N** در نظر بگیریم، پیچیدگی زمانی الگوریتم از مرتبه **O(N)** خواهد بود که برای مرحله پیش‌پردازش (**Pre-processing**) بسیار کارآمد و سریع است.

نرمال‌سازی پیش از پردازش (Normalization): در دنیای واقعی، کاربران ممکن است از کیبوردهای غیراستاندارد استفاده کنند (مثلاً تایپ متن فارسی با کیبورد عربی). در این حالت، روش تشخیص در سطح کاراکتر ممکن است دچار خطا شود.

راهکار ترکیبی (Hybrid Approach): سیستم‌های بازیابی اطلاعات مدرن، تنها به کدهای یونیکد اتکا نمی‌کنند. آن‌ها علاوه بر تحلیل کاراکترها در مرتبه زمانی **O(N)**، از مدل‌های آماری مانند **N-gram**‌ها برای سنجش توالی حروف و مقایسه آن با پیکره‌های زبانی (**Corpus**) استفاده می‌کنند تا در متون بسیار کوتاه (**Short Queries**) نیز بالاترین دقت را داشته باشند.

پاسخ‌نامه مسئله 2.5: مفاهیم پایه توکن‌گذاری (Basic Tokenization)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در معماری سیستم‌های بازیابی اطلاعات، توکن‌گذاری (**Tokenization**) فرآیندی است که طی آن، رشته‌ای پیوسته از کاراکترها به واحدهای کوچک‌تر و معنادار (توکن) تقسیم می‌شود تا در ساختار داده‌ای به نام نمایه معکوس (**Inverted Index**) ذخیره گردند. هدف از این مسئله، بررسی محدودیت‌ها و نقاط ضعف توکن‌گذارهای اولیه (**Naive Tokenizers**) است که برای قطعه‌قطعه کردن متن، صرفاً به وجود فاصله (**Space**) استناد می‌کنند.

بخش 2) عملکرد توکن‌گذار مبتنی بر فضای خالی (Whitespace Tokenizer)

همان‌طور که از نام این روش پیداست، تنها معیار برای جداسازی کلمات، رسیدن به کاراکتر فضای خالی (مانند **Space** یا **Tab**) است. از آنجا که در عبارات **\$1,000,000** و **M.I.T.** هیچ‌گونه فاصله‌ای وجود ندارد، خروجی این توکن‌گذار به شکل زیر خواهد بود:

خروجی اول: 1 توکن واحد به شکل دقیق ["\$1,000,000"]

خروجی دوم: 1 توکن واحد به شکل دقیق ["M.I.T."]

بخش 3) چرا این خروجی‌ها برای جستجو بهینه نیستند؟

مشکل اصلی این رویکرد، عدم انعطاف‌پذیری در برابر رفتارهای طبیعی کاربران هنگام جستجو (**Querying**) است. این عدم بهینگی را در هر **2** عبارت بررسی می‌کنیم:

الف) تحلیل چالش‌های عبارت \$1,000,000:

مشکل تنوع در نگارش اعداد: در دنیای واقعی، کاربری که به دنبال این عدد است ممکن است عبارت‌های **1000000** (بدون کاما)، **\$1000000** (جایجایی نماد ارز)، یا حتی **1000000 USD** را جستجو کند.

افت فراخوانی (Recall): از آنجا که موتور جستجو به دنبال تطابق دقیق (**Exact Match**) توکن‌هاست، رشته‌ی ذخیره‌شده‌ی **\$1,000,000** با رشته جستجو شده‌ی **1000000** از نظر ماشین کاملاً نامساوی ارزیابی شده و سند مرتبط بازیابی نمی‌شود. در

اینجا علائم نگارشی (کاما و نماد دلار) به عنوان «نویز» عمل کرده و مانع از تطابق معنایی می‌شوند.

ب) تحلیل چالش‌های عبارت **M.I.T**:

مشکل اختصارات (Acronyms): این عبارت مخفف مؤسسه فناوری ماساچوست است. کاربران هنگام جستجوی سریع در وب، معمولاً نقطه‌های میان اختصارات را حذف می‌کنند (به عنوان مثال تایپ می‌کنند: **MIT**).

عدم تطابق لغوی: توکن ذخیره‌شده دارای نقطه است، در حالی که پرس‌وجوی کاربر فاقد آن است. مجدداً تطابق از بین می‌رود. از سوی دیگر، اگر **1** توکن‌گذار سعی کند متن را بر اساس نقطه‌ها (**Punctuation**) نیز بشکند، این عبارت به **3** توکن مجزای **[M], [I], [T]** تبدیل می‌شود که در این صورت معنای اصلی کلمه (که **1** دانشگاه معتبر است) کاملاً متلاشی شده و با کلماتی مثل **it** یا **mit** (در زبان‌های دیگر) اشتباه گرفته می‌شود.

بخش 4) راهکار عملی و پیاده‌سازی (کدنویسی)

برای رفع این مشکل، سیستم‌های مدرن پیش از توکن‌گذاری نهایی، از فیلترهای نرمال‌سازی (**Normalization**) و عبارات باقاعده (**Regular Expressions**) استفاده می‌کنند تا نویزهای نگارشی را حذف کنند.

```
advanced_tokenizer.py

import re

def advanced_text_tokenizer(raw_string):
    pattern = r'^\w\s'
    cleaned_string = re.sub(pattern, '', raw_string)

    final_tokens = cleaned_string.split()
    return final_tokens

input_val_1 = "$1,000,000"
input_val_2 = "M.I.T."

print(advanced_text_tokenizer(input_val_1))
# Output: ['1000000']

print(advanced_text_tokenizer(input_val_2))
# Output: ['MIT']
```

بخش 5) تحلیل کارایی و نکات تکمیلی

در موتورهای جستجوی استاندارد (مانند **Elasticsearch** یا **Solr**)، معمولاً از لایه‌های تحلیلی (**Analyzers**) استفاده می‌شود که شامل توالی مشخصی از ابزارها هستند.

در **1** معماری اصولی، ابتدا فیلترهای کاراکتری (**Character Filters**) نمادهایی مانند **\$** را پردازش می‌کنند، سپس توکن‌گذار استاندارد (**Standard Tokenizer**) اعمال می‌شود که قواعد زبان‌شناختی را درک می‌کند (مثلاً می‌داند **M.I.T** **1** مخفف است و باید یکپارچه بماند، اما نقطه‌های آن حذف شود) و در نهایت فیلترهای توکن (**Token Filters**) عملیاتی مانند کوچک کردن حروف (**Lowercasing**) را انجام می‌دهند.

پاسخ‌نامه مسئله 2.6: واژه‌های توقف و بازیابی مدرن (Stop Words in Modern IR)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در سیستم‌های بازیابی اطلاعات سنتی، واژه‌های توقف (**Stop Words**) به کلماتی بسیار پرتکرار گفته می‌شود که نقش گرامری دارند اما به تنهایی بار معنایی خاصی را منتقل نمی‌کنند (مانند: «از»، «به»، «در»، «که» در فارسی و **"the"**، **"is"**، **"in"** در انگلیسی). در گذشته، رویکرد استاندارد حذف این واژگان در مرحله پیش‌پردازش بود تا حجم نمایه معکوس (**Inverted Index**) کاهش یابد و سرعت پردازش بیشتر شود.

هدف از این مسئله، درک «تغییر پارادایم» (**Paradigm Shift**) در معماری موتورهای جستجوی مدرن است؛ جایی که مهندسان ترجیح می‌دهند این واژگان را حذف نکنند تا دقت معنایی فدای فضای ذخیره‌سازی نشود.

بخش 2) تشریح گام‌به‌گام: دلایل حفظ واژه‌های توقف در سیستم‌های مدرن

تغییر رویکرد موتورهای جستجوی بزرگ (مانند گوگل یا بینگ) به سمت حفظ واژه‌های توقف، ریشه در پیشرفت‌های سخت‌افزاری و نرم‌افزاری زیر دارد:

توجیه اقتصادی و پیشرفت سخت‌افزاری (Cheap Storage): در دهه‌های گذشته، حافظه اصلی (**RAM**) و فضای ذخیره‌سازی بسیار گران‌قیمت بودند. کلمات توقف گاهی تا **30** الی **40** درصد حجم یک متن را تشکیل می‌دادند. امروزه با ارزان شدن فضای ذخیره‌سازی و توسعه الگوریتم‌های پیشرفته فشرده‌سازی نمایه، هزینه نگهداری این کلمات دیگر چالش و مانع مهندسی محسوب نمی‌شود.

اهمیت ساختار نحوی در مدل‌های زبانی (NLP & Transformers): موتورهای جستجوی مدرن دیگر صرفاً بر اساس تطابق کلمه کلیدی (**Keyword Matching**) کار نمی‌کنند. آن‌ها از مدل‌های زبانی عمیق (مانند **BERT**) بهره می‌برند. این مدل‌ها بر اساس مکانیزم توجه کار می‌کنند و برای درک منظور کاربر، به بررسی کل ساختار جمله و روابط کلمات با یکدیگر نیاز دارند. حذف کلمات توقف، این ساختار نحوی را متلاشی کرده و درک ماشین از زمینه (**Context**) را کور می‌کند.

پشتیبانی از جستجوهای عبارتی (Phrase Queries): زمانی که کاربر به دنبال یک عبارت دقیق است، ترتیب و حضور تمامی کلمات الزامی است. حذف کلمات توقف باعث می‌شود سیستم نتواند تفاوت میان عبارات دقیق را تشخیص دهد.

حفظ یکپارچگی نمایه موقعیتی (Positional Indexing): برای محاسبه فاصله کلمات از یکدیگر (**Proximity Search**)، سیستم باید موقعیت دقیق هر توکن را در سند ذخیره کند. حذف کلمات توقف باعث جابجایی این موقعیت‌ها شده و محاسبه فواصل واقعی کلمات را برای الگوریتم‌های رتبه‌بندی پیچیده می‌سازد.

بخش 3) بررسی نمونه‌های کاربردی

برای درک بهتر فاجعه معنایی ناشی از حذف واژه‌های توقف در جستجوهای خاص، به این دو سناریو توجه کنید:

الف) مثال تغییر جهت و رابطه (Relational Context):

فرض کنید کاربر پرس‌وجوی «پرواز از تهران به مشهد» را جستجو می‌کند. اگر کلمات توقف حذف شوند، موتور جستجو کلمات «از» و «به» را حذف کرده و پرس‌وجو به توکن‌های ['پرواز', 'تهران', 'مشهد'] تبدیل می‌شود. اکنون سیستم نمی‌داند مبدأ کجاست و مقصد کدام است! ممکن است بلیط‌های «پرواز از مشهد به تهران» را به عنوان اولین نتیجه نشان دهد که کاملاً اشتباه است. در سیستم مدرن، با حفظ «از» و «به»، جهت و رابطه منطقی کلمات حفظ شده و نتایج مطابق نیاز کاربر فیلتر می‌شوند.

ب) مثال نام‌های خاص و عبارات مشهور (Entities & Famous Quotes):

در نظر بگیرید کاربری عبارت معروف شکسپیر یعنی **"To be or not to be"** را جستجو کند. از آنجا که تمام کلمات این جمله در لیست کلاسیک Stop Words قرار دارند، پس از فیلتر شدن، یک رشته توکن کاملاً خالی باقی می‌ماند و سیستم هیچ نتیجه‌ای بر نمی‌گرداند. همچنین جستجوی فیلم سینمایی **"The Matrix"** در برابر مفهوم ریاضی **"Matrix"** نیازمند حفظ کلمه **"The"** است؛ چرا که نشان‌دهنده یک نام خاص است و حذف آن نتایج نامرتبط را به کاربر نشان خواهد داد.

بخش 4) نکات تکمیلی و جمع‌بندی

رویکرد بینابینی:

در طراحی سیستم‌های بازیابی اطلاعات مدرن برای کتابخانه‌ها و موتورهای جستجو، رویکرد بینابینی (**Soft Stop Words**) نیز رایج است. در این روش، واژه‌های توقف به طور کامل از نمایه پایگاه داده حذف نمی‌شوند (تا جستجوی عبارتی و معنایی مختل نشود)، اما در فرمول‌های وزن‌دهی (مانند **TF-IDF** یا **BM25**) وزن بسیار ناچیزی به آن‌ها اختصاص داده می‌شود تا در رتبه‌بندی نهایی اسناد، تأثیر مخربی بر کلمات کلیدی اصلی نداشته باشند.

پاسخنامه مسئله 2.7: نرمال‌سازی حروف (Normalization & Case Folding)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در فاز پیش‌پردازش سیستم‌های بازیابی اطلاعات، هدف از نرمال‌سازی (**Normalization**) ایجاد یک «کلاس هم‌ارزی» (**Equivalence Class**) میان شکل‌های ظاهری مختلف از یک واژه است. مسئله تفاوت میان **U.S.A** و **USA** یکی از چالش‌های کلاسیک در زمینه پردازش اختصارات (**Acronyms**) و یکسان‌سازی حروف (**Case Folding**) است که مستقیماً بر روی میزان فراخوانی (**Recall**) تأثیر می‌گذارد. هدف این است که فرم‌های گوناگون نگارشی، در نهایت به یک توکن واحد و استاندارد در نمایه معکوس تبدیل شوند.

بخش 2) تشریح گام‌به‌گام: فرآیند مدیریت اختصارات و حروف بزرگ

برای آنکه سیستم بتواند هر دو عبارت را کاملاً معادل یکدیگر ارزیابی کند، باید یک خط لوله پردازشی (**Processing Pipeline**) شامل 2 گام اساسی، هم بر روی متن سند (هنگام ساخت نمایه) و هم بر روی پرس‌وجوی کاربر (هنگام جستجو) اعمال گردد:

گام اول - فیلتر کردن علائم نگارشی (Punctuation Removal): در برخورد با کلمات اختصاری، سیستم باید قوانینی برای مدیریت نقطه‌ها داشته باشد. یک رویکرد استاندارد، استفاده از فیلترهای کاراکتری یا عبارات باقاعده (**Regular Expressions**) برای شناسایی حروفی است که با نقطه از هم جدا شده‌اند. در این مرحله، تمامی نقطه‌های میانی از عبارت **U.S.A** حذف شده و رشته به شکل پیوسته **USA** درمی‌آید.

گام دوم - یکسان‌سازی اندازه حروف (Case Folding): پس از حذف علائم اضافی، باید تفاوت میان حروف بزرگ و کوچک از بین برود. رایج‌ترین روش در موتورهای جستجو، تبدیل تمامی حروف متن به فرمت حروف کوچک (**Lowercasing**) است. در این گام، عبارت **USA** به توکن استاندارد **usa** تبدیل می‌شود.

مکانیسم تطابق (Matching Mechanism): با پیاده‌سازی این خط لوله، فرقی نمی‌کند متن خام داخل سند چگونه نوشته شده باشد، یا کاربر پرس‌وجوی خود را به صورت **U.S.A**، **USA**، یا حتی **Usa** وارد کند. تمامی این ورودی‌ها پس از عبور از الگوریتم نرمال‌سازی، به توکن نهایی **[usa]** نگاشت می‌شوند. بدین ترتیب، سیستم موتور جستجو تطابق دقیق (**Exact Match**) را پیدا کرده و سند مرتبط را بازیابی می‌کند.

جهت نمایش عملی این منطق، قطعه کد زیر نحوه عملکرد خط لوله نرمال‌سازی را نشان می‌دهد:

```
normalization_pipeline.py

import re

def normalize_acronyms_and_case(raw_token):
    # remove periods between characters using regex
    cleaned_token = re.sub(r'(?<=[a-zA-Z])\.(?=[a-zA-Z]|$)', '', raw_token)

    # apply lowercasing for case folding
    final_token = cleaned_token.lower()
    return final_token

token_1 = "U.S.A"
token_2 = "USA"
token_3 = "u.s.a"

print(normalize_acronyms_and_case(token_1))
# Output: usa
print(normalize_acronyms_and_case(token_2))
# Output: usa
print(normalize_acronyms_and_case(token_3))
# Output: usa
```

بخش 4) تحلیل کارایی و چالش‌های معنایی

با وجود اینکه روش فوق مشکل تطابق را حل می‌کند، اما در مهندسی سیستم‌های جستجو باید به محدودیت‌ها و چالش‌های معنایی آن نیز دقت کرد:

چالش ابهام معنایی (Semantic Ambiguity): تبدیل کلمات تمام‌بزرگ (**All-Caps**) به حروف کوچک همیشه بی‌خطر نیست. به عنوان مثال، تبدیل کلمه **US** (مخفف ایالات متحده) به **us** (ضمیر مفعولی «ما» در انگلیسی) باعث افت شدید دقت (**Precision**) می‌شود. در این حالت، جستجو برای کشور آمریکا ممکن است اسنادی را برگرداند که صرفاً حاوی کلمه پرکاربرد **us** هستند.

راهکار سیستم‌های مدرن (Truecasing):

در موتورهای جستجوی پیشرفته امروزی، به جای انجام یکسان‌سازی کورکورانه برای تمام کلمات، از تکنیک **Truecasing** یا فرهنگ لغات اختصاصی (**Dictionaries**) استفاده می‌شود. در این رویکرد، سیستم با استفاده از روش‌های آماری و شناسایی موجودیت‌های نام‌دار (**NER**)، کلماتی که ذاتاً ماهیت اختصاری یا نام خاص دارند را تشخیص داده و آن‌ها را به صورت یکپارچه و مجزا از کلمات عمومی ذخیره می‌کند تا معنای دقیق آن‌ها حفظ شود.

پاسخ‌نامه مسئله 2.8: ریشه‌یابی و بن‌واژه‌سازی (Stemming vs Lemmatization)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در سیستم‌های بازیابی اطلاعات، هدف از فاز تقلیل واژگان (**Word Reduction**)، رساندن اشکال مختلف یک کلمه به یک ریشه واحد است تا میزان فراخوانی (**Recall**) افزایش یابد. با این حال، زبان طبیعی پر از ابهامات ساختاری و افعال بی‌قاعده (**Irregular Verbs**) است. کلمه **saw** یک نمونه کلاسیک از این ابهامات است؛ چرا که می‌تواند شکل گذشته فعل دیدن (**See**)، یک اسم به معنای ابزار «اره»، و یا حتی یک فعل مستقل به معنای «اره کردن» باشد.

هدف از این مسئله، بررسی تفاوت رفتار مکانیکی روش «ریشه‌یابی» (**Stemming**) در مقابل رفتار هوشمند و معناگرای روش «بن‌واژه‌سازی» (**Lemmatization**) در مواجهه با این ابهام زبانی است.

بخش 2) بررسی عملکرد الگوریتم Porter Stemmer (ریشه‌یابی مکانیکی)

الگوریتم پورتر (طراحی شده در سال 1980) یک روش کاملاً مبتنی بر قواعد مکاشفه‌ای (**Heuristic Rule-Based**) است.

نحوه کارکرد: این الگوریتم هیچ درکی از معنای کلمه، نقش دستوری (**Part of Speech**) یا فرهنگ لغت ندارد. پورتر تنها به انتهای کلمه نگاه می‌کند و با استفاده از یک سری قواعد متوالی، پسوندها (مانند **s**، **ed**، **ing**) را قطع می‌کند (**Suffix Stripping**).

خروجی برای کلمه saw: از آنجا که کلمه **saw** در ظاهر خود هیچ‌یک از پسوندهای تعریف‌شده در قواعد پورتر را ندارد، این الگوریتم بدون هیچ تغییری، دقیقاً همان کلمه **saw** را به عنوان ریشه برمی‌گرداند.

نتیجه‌گیری در جستجو: اگر کاربر عبارت **see** را جستجو کند، سندی که در آن کلمه **saw** (به معنای دید) به کار رفته است، توسط استمر پورتر بازیابی نخواهد شد؛ زیرا ریشه **see** همان **see** و ریشه **saw** همان **saw** محاسبه می‌شود و تطابقی رخ نمی‌دهد.

بخش 3) بررسی عملکرد Lemmatization (بن‌واژه‌سازی معنایی)

روش لماتی‌زیشن (**Lemmatization**) یک رویکرد پیشرفته و مبتنی بر پردازش زبان طبیعی (**NLP**) است که به جای قطع کردن حروف، کلمه را در یک فرهنگ لغت (**Dictionary/Lexicon**) جستجو کرده و با تحلیل صرفی (**Morphological Analysis**)، آن را به فرم استاندارد فرهنگ‌لغتی آن (**Lemma**) تبدیل می‌کند.

این روش برای عملکرد صحیح، نیازمند دانستن **نقش اداتی (POS Tag)** کلمه در داخل جمله است. خروجی این الگوریتم برای کلمه **saw** بسته به بستر جمله (**Context**) متفاوت خواهد بود:

حالت اول (فعل بی‌قاعده): اگر در مرحله پیش‌پردازش، نقش کلمه به عنوان فعل (**Verb**) تشخیص داده شود (مانند جمله **I saw him**)، الگوریتم درک می‌کند که این شکل گذشته‌ی فعل است و خروجی **see** را برمی‌گرداند.

حالت دوم (اسم): اگر نقش کلمه به عنوان اسم (**Noun**) تشخیص داده شود (مانند جمله **He bought a new saw**)، الگوریتم آن را به عنوان یک شیء (اژه) شناسایی کرده و خروجی **saw** را برمی‌گرداند.

حالت سوم (فعل باقاعده): اگر نقش کلمه به عنوان فعلی مستقل تشخیص داده شود (مانند جمله **I will saw the wood**)، به معنای عمل اژه کردن است و مجدداً خروجی **saw** خواهد بود.

بخش 4) راهکار عملی و پیاده‌سازی

برای درک بهتر تفاوت این دو روش، قطعه کد زیر با استفاده از کتابخانه استاندارد **nlTK** در پایتون تدوین شده است. مطابق با اصول مهندسی نرم‌افزار، تمامی شناسه‌ها و مستندات کد به زبان انگلیسی نوشته شده‌اند.

```
stem_vs_lemma.py

import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer

# Initialize instances
stemmer_obj = PorterStemmer()
lemmatizer_obj = WordNetLemmatizer()

target_word = "saw"

# 1. Porter Stemmer execution
stem_result = stemmer_obj.stem(target_word)
print(f"Porter Stemmer Output: {stem_result}")

# 2. Lemmatization execution with specific POS tags
lemma_as_verb = lemmatizer_obj.lemmatize(target_word, pos='v') # 'v' stands for verb
lemma_as_noun = lemmatizer_obj.lemmatize(target_word, pos='n') # 'n' stands for noun
```

```
print(f"Lemmatizer Output (If Verb): {lemma_as_verb}")
print(f"Lemmatizer Output (If Noun): {lemma_as_noun}")
```

بخش 5) تحلیل کارایی و نکات تکمیلی (Trade-offs)

در معماری موتورهای جستجو، انتخاب میان **Stemming** و **Lemmatization** نیازمند ارزیابی هزینه‌هاست:

پیچیدگی و سرعت (Time Complexity): استمر پورتر بسیار سبک و سریع است و پردازش آن از مرتبه **O(1)** برای هر توکن انجام می‌شود. در مقابل، لماتی‌زیشن به دلیل نیاز به اجرای الگوریتم‌های **POS Tagging** و جستجو در گراف‌های فرهنگ لغت، سرشار محاسباتی (**Computational Overhead**) بسیار بالاتری دارد.

دقت و کیفیت (Precision & Quality):

لماتی‌زیشن مشکل افعال بی‌قاعده را به طور کامل حل کرده و دقت معنایی سیستم را به شدت افزایش می‌دهد. در سیستم‌های بازیابی اطلاعات مدرن وب (مانند موتور جستجوی گوگل)، با توجه به توان پردازشی بالا، استفاده از **Lemmatization** استاندارد بلامنازع است؛ اما در موتورهای جستجوی درون‌سازمانی کوچک که سرعت نمایه‌سازی اهمیت بیشتری دارد، الگوریتم **Porter** همچنان کاربرد وسیعی دارد.

پاسخ‌نامه مسئله 2.9: چالش‌های ریشه‌یابی بیش از حد (Over-stemming)

بخش 1) تحلیل صورت مسئله و مفاهیم پایه

در طراحی الگوریتم‌های ریشه‌یاب (**Stemmers**)، مهندسان همواره با یک نقطه تعادل (**Trade-off**) میان دو نوع خطا روبرو هستند: ریشه‌یابی کمتر از حد (**Under-stemming**) و ریشه‌یابی بیش از حد (**Over-stemming**).

پدیده **Over-stemming** زمانی رخ می‌دهد که الگوریتم در حذف پسوندها بسیار تهاجمی عمل کرده و بخش زیادی از انتهای کلمه را قطع می‌کند. هدف از این مسئله، بررسی عواقب مخرب این رفتار تهاجمی بر روی معنای واژگان و کیفیت نتایج بازیابی‌شده است.

بخش 2) پدیده ادغام معنای (Semantic Conflation)

برای درک مشکل ایجاد شده، باید ماهیت معنایی (**Semantic**) دو کلمه مطرح‌شده در صورت مسئله را کالبدشکافی کنیم:

کلمه Organization: از نظر لغوی به معنای «سازمان»، «تشکیلات» یا «نهاد» است (یک مفهوم ساختاری و اداری).

کلمه Organ: از نظر لغوی به معنای «اندام / عضو بدن» (در زیست‌شناسی) یا «ارگ» (یک ساز موسیقی) است.

زمانی که یک ریشه‌یاب ساده‌لوح (**Naive Stemmer**) پسوند **ization** را از کلمه اول به طور کامل حذف می‌کند، هر دو کلمه در نمایه معکوس (**Inverted Index**) به یک توکن واحد یعنی **[organ]** تبدیل و ادغام می‌شوند. در علوم بازیابی اطلاعات، به این خطای مهلك «تقاطع کاذب» (**False Conflation**) یا ادغام واژگان نامرتب‌ت گفته می‌شود.

بخش 3) تبعات مخرب در سیستم موتور جستجو (Precision vs. Recall)

هنگامی که این دو مفهوم مجزا به یک ریشه مشترک تقلیل می‌یابند، موتور جستجو در زمان پاسخ‌گویی به کاربر دچار کوری معنایی می‌شود که تبعات زیر را به همراه دارد:

سقوط آزاد معیار دقت (Precision Drop): مهم‌ترین و مخرب‌ترین نتیجه‌ی ریشه‌یابی بیش از حد، کاهش شدید دقت نتایج (افزایش **False Positives**) است.

مثال عملی: فرض کنید یک دانشجوی پزشکی عبارت **Human organ donation** (اهدای عضو انسان) را جستجو می‌کند. از آنجا که سیستم کلمه **organization** را نیز به صورت **organ** ذخیره کرده است، موتور جستجو ده‌ها سند نامرتب دربارۀ **Human rights organization** (سازمان حقوق بشر) را برای این کاربر بازایی می‌کند. کاربر با انبوهی از نتایج کاملاً بی‌ربط مواجه شده و اعتماد خود را به سیستم از دست می‌دهد.

رشد کاذب فراخوانی (Artificial Recall Increase): اگرچه از نظر ریاضیاتی ممکن است تعداد اسناد بازایی‌شده (**Recall**) افزایش یابد، اما این افزایش ارزشی ندارد، زیرا اسناد اضافه‌شده هیچ‌گونه ارتباط موضوعی با نیاز اطلاعاتی کاربر (**User Intent**) ندارند.

بخش 4) راهکار عملی و اثبات نرم‌افزاری

برای مشاهده این باگ در الگوریتم‌های کلاسیک (مانند **Porter Stemmer**)، قطعه کد زیر رفتار سیستم را شبیه‌سازی می‌کند:

```
over_stemming_demo.py

from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer

stemmer_obj = PorterStemmer()
lemmatizer_obj = WordNetLemmatizer()

word_1 = "organization"
word_2 = "organ"

# Demonstrating Over-stemming in Porter Stemmer
print(f"Porter Output 1: {stemmer_obj.stem(word_1)}")
print(f"Porter Output 2: {stemmer_obj.stem(word_2)}")

# Demonstrating the solution using Lemmatization
print(f"Lemmatizer Output 1: {lemmatizer_obj.lemmatize(word_1, pos='n')}")
print(f"Lemmatizer Output 2: {lemmatizer_obj.lemmatize(word_2, pos='n')}")
```

بخش 5) نکات تکمیلی و جمع‌بندی

برای جلوگیری از خطای **Over-stemming** در معماری سیستم‌های بازایی اطلاعات مدرن، مهندسان از راهکارهای زیر استفاده می‌کنند:

- استفاده از الگوریتم‌های ملایم‌تر:** جایگزینی الگوریتم‌های تهاجمی با نسخه‌های بهینه‌تر مانند **Snowball Stemmer** که قواعد محتاطانه‌تری برای قطع پسوندها دارد.
- لیست استثنائات (Exception Lists):** تعریف یک دیکشنری از کلمات خاص در موتور جستجو که سیستم اجازه ریشه‌یابی آن‌ها را ندارد (**Protected Words**).
- گذر به سمت Lemmatization:** همان‌طور که در مسئله قبل بررسی شد، استفاده از بن‌واژه‌سازی مبتنی بر فرهنگ لغت، با حفظ یکپارچگی معنایی کلمه، مشکل ادغام مفاهیم نامرتب را به طور کامل ریشه‌کن می‌کند.

پاسخ‌نامه مسئله 2.10: کلاس‌های هم‌ارزی در زبان‌های غیرانگلیسی (Equivalence Classes)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

مفهوم «کلاس هم‌ارزی» (**Equivalence Class**) در سیستم‌های بازیابی اطلاعات، به معنای گروه‌بندی مجموعه‌ای از توکن‌ها با ظواهر متفاوت است که از نظر معنایی دقیقاً به یک مفهوم اشاره دارند. در پردازش زبان طبیعی (**NLP**) زبان فارسی، یکی از بزرگترین چالش‌های پیش‌پردازش و نرمال‌سازی، مسئله «نیم‌فاصله» (**Zero Width Non Joiner**) یا (**ZWNJ**) است.

هدف از این مسئله، درک ساختار کاراکتر نیم‌فاصله در استاندارد یونیکد و طراحی یک رویکرد سیستماتیک برای همگام‌سازی و نگاشت فرم‌های مختلف نگارشی (مانند چسبیده، با فاصله کامل، و با نیم‌فاصله) به یک توکن استاندارد و واحد در نمایه معکوس (**Inverted Index**) است.

بخش 2) چالش نیم‌فاصله در پردازش زبان فارسی

برای ماشین، کاراکترهای متنی صرفاً دنباله‌ای از بایت‌ها هستند. تفاوت ظاهری و ساختاری واژگان در این سناریو به شرح زیر است:

حالت اول - نگارش استاندارد («می‌رود»): در این حالت، بین پیشوند استمراری «می» و ریشه فعل «رود»، از کاراکتر نامرئی نیم‌فاصله (با کد یونیکد **U+200C**) استفاده شده است. این کاراکتر از چسبیدن حروف جلوگیری می‌کند، اما به توکن‌گذار (**Tokenizer**) سیگنال می‌دهد که این مجموعه همچنان یک کلمه واحد است.

حالت دوم - نگارش چسبیده («میرود»): در این حالت، پیشوند مستقیماً به فعل چسبیده است. از نظر بصری خواناست، اما از نظر ماشین یک رشته کاراکتری کاملاً متفاوت از حالت اول محسوب می‌شود.

حالت سوم - نگارش با فاصله کامل («می رود»): کاربر از کلید **Space** (کد **U+0020**) استفاده کرده است. توکن‌گذارهای مبتنی بر فضای خالی، به اشتباه این فعل را به **2** توکن بی‌معنی [می] و [رود] می‌شکنند که منجر به از دست رفتن کامل معنای فعل می‌گردد.

مشکل بازیابی (Recall Drop): اگر این **3** حالت در یک کلاس هم‌ارزی قرار نگیرند، کاربری که کلمه «می‌رود» را جستجو می‌کند، اسنادی را که نویسنده در آن‌ها از فرم «میرود» استفاده کرده است، از دست خواهد داد و فراخوانی سیستم به شدت افت می‌کند.

بخش 3) راهکار عملی: ایجاد کلاس هم‌ارزی (Normalization Rules)

برای حل این چالش در موتورهای جستجوی فارسی، سیستم باید در فاز نرمال‌سازی (قبل از توکن‌گذاری نهایی)، یک سری قواعد مبتنی بر الگو (**Pattern-matching**) را اجرا کند.

استانداردترین روش، «نگاشت به فرمت استاندارد نیم‌فاصله‌دار» (**Canonical Form Mapping**) است. در این رویکرد، سیستم پیشنندهای فعلی (مثل «می» و «نمی») و پسوندهای جمع (مثل «ها») را شناسایی کرده و هرگونه اتصال مستقیم یا فاصله کامل (**Space**) پس از آن‌ها را با یک کاراکتر استاندارد **ZWNJ** جایگزین می‌کند. به این ترتیب، فارغ از نحوه تایپ کاربر یا نویسنده سند، تمامی فرم‌ها به شکل استاندارد «می‌رود» ذخیره و جستجو می‌شوند.

بخش 4) پیاده‌سازی و کدنویسی بهینه

قطعه کد زیر، یک نرمال‌ساز سریع مبتنی بر عبارات باقاعده (**Regex**) را نشان می‌دهد که این اشکال مختلف را در یک کلاس هم‌ارزی قرار می‌دهد. کدهای این بخش با رعایت اصول کدنویسی تمیز (**Clean Code**) و بهینه‌سازی پردازشی تدوین شده‌اند.

```
zwnj_normalizer.py

import re
```

```
def normalize_persian_verb_prefix(raw_word):
    # Regex pattern to match 'mi' or 'nemi' at the beginning of a word,
    # followed optionally by a space or ZWNJ (\u200C)
    pattern = r'^([می|نی])[\s\u200C]?(.*)$'

    # Check for a match and reconstruct the word with a standard ZWNJ
    match = re.match(pattern, raw_word)
    if match:
        # match.group(1) is the prefix, match.group(2) is the stem
        canonical_form = match.group(1) + '\u200C' + match.group(2)
        return canonical_form

    # Return original word if pattern does not match
    return raw_word

# Test cases representing different equivalence forms
input_1 = "میرود"
input_2 = "می‌رود"
input_3 = "می رود"

print(normalize_persian_verb_prefix(input_1))
# Output: می‌رود

print(normalize_persian_verb_prefix(input_2))
# Output: می‌رود

print(normalize_persian_verb_prefix(input_3))
# Output: می‌رود
```

بخش 5) تحلیل کارایی و نکات تکمیلی (Complexity Analysis)

پیچیدگی زمانی (Time Complexity): پردازش عبارات باقاعده در قطعه کد فوق برای تطابق در ابتدای رشته (**Anchored Match**)، دارای پیچیدگی زمانی **O(K)** است (که در آن **K** طول کلمه است). از آنجا که طول کلمات در زبان طبیعی بسیار کوچک و دارای یک حد بالای ثابت (**Constant Upper Bound**) است، می‌توان پیچیدگی زمانی نرمال‌سازی هر توکن را **O(1)** و برای یک سند با **N** توکن، **O(N)** در نظر گرفت که این الگوریتم را برای استفاده در موتورهای جستجو کاملاً بهینه و کارآمد می‌سازد.

پیچیدگی مکانی (Space Complexity): حافظه مصرفی این تابع صرفاً متکی به ایجاد یک رشته خروجی جدید است که پیچیدگی مکانی **O(K)** برای هر کلمه دارد و کاملاً بهینه است.

چالش ابهام‌زدایی (Disambiguation):

در توسعه سیستم‌های تجاری، باید مراقب کلماتی بود که با «می» شروع می‌شوند اما «می» در آن‌ها پیشوند نیست (مانند کلمات «میدان»، «میکروب» یا «میتوان»). برای جلوگیری از تبدیل اشتباه این کلمات به «می‌دان»، در کنار **Regex** از یک فرهنگ لغت ریشه‌شناختی (**Lexicon**) برای اعتبارسنجی ریشه فعل پس از پیشوند استفاده می‌شود.

پاسخ‌نامه مسئله 2.11: تأثیر نرمال‌سازی بر اندازه واژگان (Impact on Vocabulary Size)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در معماری موتورهای جستجو، فاز تقلیل واژگان (**Word Reduction**) نقشی حیاتی در تعیین ساختار نمایه معکوس (**Inverted Index**) دارد. تصمیم‌گیری میان استفاده از ریشه‌یابی مکانیکی (**Stemming**) و بن‌واژه‌سازی معنایی (**Lemmatization**) یک توازن مهندسی (**Trade-off**) پیچیده میان فضای ذخیره‌سازی، دقت نتایج و زمان اجرای پرس‌وجو است. هدف از این مسئله، بررسی تحلیلی تأثیر این دو روش بر ابعاد ساختار داده نمایه و کارایی الگوریتم‌های بازیابی است.

بخش 2) بررسی تأثیر بر اندازه کل واژگان (Dictionary Size)

اندازه واژگان نمایه (تعداد توکن‌های یکتای ذخیره‌شده) ارتباط مستقیمی با میزان تهاجمی بودن الگوریتم پیش‌پردازش دارد.

رویکرد Stemming (کاهش شدید واژگان): الگوریتم‌هایی مانند **Porter** بسیار تهاجمی عمل می‌کنند و با قطع بی‌رویه پسوندها، کلمات متعددی را به یک ریشه تقلیل می‌دهند. به عنوان مثال، کلمات **universe** و **university** هر دو ممکن است به ریشه نامفهوم **univers** نگاشت شوند. این رفتار باعث ادغام گسترده واژگان مختلف (تقاطع کاذب) شده و اندازه کل واژگان لغت‌نامه نمایه را به شدت **کاهش** می‌دهد.

رویکرد Lemmatization (کاهش ملایم و کنترل‌شده): این روش با استفاده از تحلیل صرفی و فرهنگ لغت، تنها کلماتی را ادغام می‌کند که از نظر دستوری هم‌خانواده باشند (مانند **is**، **are** و **am** به فرم پایه **be**). کلمات مستقلی مانند **universe** و **university** در این روش به عنوان دو موجودیت (**Lemma**) کاملاً مجزا حفظ می‌شوند. بنابراین، اندازه واژگان در صورت استفاده از **Lemmatization**، به مراتب **بزرگتر** از زمان استفاده از **Stemming** خواهد بود.

بخش 3) بررسی تأثیر بر سرعت جستجو (Search Speed)

برای تحلیل سرعت پاسخگویی سیستم، باید فاز اجرای پرس‌وجو (**Query Execution**) را به بررسی لغت‌نامه و عملیات روی لیست‌های پستینگ (**Posting Lists**) تفکیک کرد.

تغییرات طول لیست‌های پستینگ: در روش **Stemming**، به دلیل ادغام چندین کلمه مجزا در یک ریشه واحد، تعداد اسناد مرتبط با آن توکن به شدت افزایش می‌یابد؛ در نتیجه لیست‌های پستینگ بسیار **طولانی‌تر** و متراکم‌تر می‌شوند. در مقابل، **Lemmatization** توزیع واژگان را دقیق‌تر و پراکنده‌تر نگه می‌دارد و لیست‌های پستینگ **کوتاه‌تر** و متمرکزتری تولید می‌کند.

زمان اشتراک‌گیری (Intersection Time): الگوریتم استاندارد برای یافتن اسناد مرتبط در پرس‌وجوهای چندکلمه‌ای (مانند **AND**)، پیمایش همزمان دو یا چند لیست پستینگ است. اگر طول دو لیست **P₁** و **P₂** باشد، پیچیدگی زمانی اشتراک‌گیری در بدترین حالت از مرتبه **O(P₁ + P₂)** خواهد بود. از آنجا که **Lemmatization** لیست‌های پستینگ کوتاه‌تری ایجاد می‌کند، زمان اشتراک‌گیری و فیلتر کردن اسناد به شکل چشمگیری **سریع‌تر** از روش **Stemming** خواهد بود.

بخش 4) تحلیل پیچیدگی زمانی و نکات مهندسی

در طراحی و ارزیابی کارایی کلی سیستم‌های بازیابی اطلاعات، باید میان «زمان نمایه‌سازی» (**Indexing Time**) و «زمان پاسخگویی» (**Latency**) تمایز قائل شد:

هزینه نمایه‌سازی آفلاین: اعمال **Stemming** بر روی یک کلمه معمولاً با پیچیدگی زمانی **O(1)** و صرفاً با اجرای چند دستور مکاشفه‌ای (**If-Else**) انجام می‌شود. اما **Lemmatization** نیازمند برچسب‌گذاری نقش کلمات (**POS Tagging**) و پیمایش ساختار داده فرهنگ لغت است که سربار محاسباتی سنگین‌تری ایجاد کرده و فاز نمایه‌سازی را کندتر می‌کند.

بهینه‌سازی عملیاتی (Online Optimization):

با وجود کندتر بودن فاز نمایه‌سازی در **Lemmatization**، این هزینه محاسباتی تنها یک‌بار و در پس‌زمینه سیستم پرداخت می‌شود. در محیط عملیاتی که سیستم باید در کسری از ثانیه به هزاران پرس‌وجوی همزمان پاسخ دهد، تولید لیست‌های پستینگ کوتاه‌تر و عدم ادغام کلمات نامرتب توسط **Lemmatization**، پیچیدگی اجرای پرس‌وجو را بهینه کرده و زمان پردازش آنلاین را به حداقل ممکن می‌رساند.

پاسخ‌نامه مسئله 2.12: توکن‌گذاری در عبارات ترکیبی (Compound Tokenization)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در پردازش زبان طبیعی و معماری سیستم‌های بازیابی اطلاعات، برخورد با کلمات ترکیبی (**Compound Words**) یکی از چالش‌برانگیزترین مراحل فاز توکن‌گذاری (**Tokenization**) است. این مسئله به بررسی دو ساختار کاملاً متفاوت می‌پردازد: کلمات متصل و بدون فاصله (مانند زبان آلمانی) و کلمات متصل با علائم نگارشی نظیر خط‌تیره (مانند اسامی خاص و صفات ترکیبی در زبان انگلیسی).

هدف از این مسئله، تعیین استراتژی بهینه‌ای است که ضمن حفظ هویت معنایی کلمات ترکیبی (دقت یا **Precision**)، مانع از ریزش نتایج در مواجهه با رفتارهای گوناگون کاربران هنگام جستجو (فراخوانی یا **Recall**) شود.

بخش 2) چالش زبان‌های چسباننده (مانند آلمانی)

در زبان‌هایی مانند آلمانی، هلندی یا فنلاندی، کلمات جدید و پیچیده از طریق چسباندن مستقیم کلمات پایه به یکدیگر ساخته می‌شوند. به عنوان مثال، کلمه **Lebensversicherungsgesellschaft** به معنای «شرکت بیمه عمر» است.

مشکل نمایه‌سازی یکپارچه: اگر موتور جستجو این کلمه طولانی را دقیقاً به عنوان یک توکن واحد در نمایه معکوس ذخیره کند، کاربری که صرفاً کلمه **Versicherung** (بیمه) را جستجو می‌کند، این سند را پیدا نخواهد کرد. این امر به معنای افت شدید معیار فراخوانی (**Recall Drop**) و افزایش کلمات خارج از دایره واژگان (**Out-of-Vocabulary**) است.

استراتژی بهینه (Decompounding): در برخورد با این زبان‌ها، استفاده از ابزارهای «ترکیب‌شکن» (**Decompiler**) در خط لوله پردازشی الزامی است. سیستم با استفاده از یک فرهنگ لغت جامع و تحلیل‌های آماری (مبتنی بر **N-gram** ها)، کلمه طولانی را به توکن‌های سازنده و معنادار آن (یعنی **Leben**، **Versicherung** و **Gesellschaft**) می‌شکند. در نهایت، هم کلمه اصلی و هم اجزای شکسته شده در نمایه ذخیره می‌شوند تا هرگونه پرس‌وجوی مرتبط را پوشش دهند.

بخش 3) چالش کلمات خط‌تیره‌دار (Hyphenated Words) در انگلیسی

در عباراتی مانند **Hewlett-Packard** (نام خاص یک شرکت) یا **state-of-the-art** (یک اصطلاح/صفت)، چالش اصلی رفتار غیرقابل‌پیش‌بینی کاربران هنگام جستجو است. یک کاربر ممکن است نام شرکت را با خط‌تیره، دیگری با فاصله (**Space**) و کاربر سوم به صورت کاملاً چسبیده تایپ کند.

مشکل توکن‌گذاری ساده‌لوحانه: اگر موتور جستجو خط‌تیره را به عنوان یک جداکننده (**Delimiter**) در نظر بگیرد و آن را حذف کند، عبارت **Hewlett-Packard** به 2 توکن مجزای **[Hewlett]** و **[Packard]** تجزیه می‌شود. این کار هویت یکپارچه و خاص این برند را نابود می‌کند. از سوی دیگر، اگر خط‌تیره به عنوان بخشی از کلمه حفظ شود **[Hewlett-Packard]**، کاربری که **Hewlett Packard** (با فاصله) را جستجو کرده، هیچ نتیجه‌ای دریافت نخواهد کرد (تطابق دقیق با شکست مواجه می‌شود).

بخش 4) راهکار بهینه مهندسی: استراتژی نمایه‌سازی چندگانه (Multi-Token Indexing)

برای حل قطعی این چالش در موتورهای جستجوی استاندارد و مدرن (مانند **Elasticsearch** یا **Solr**)، مهندسان از استراتژی «تولید توکن‌های هم‌پوشان» (**Overlapping Tokens**) بهره می‌برند.

در این استراتژی، به جای انتخاب اجباری میان شکستن یا نگه داشتن توکن، سیستم با استفاده از فیلترهای کلمات مترادف (**Synonym/WordDelimiter Filters**) ترکیبی از حالت‌های ممکن را در **یک موقعیت یکسان (Same Position)** از نمایه ذخیره می‌کند:

1. **حفظ فرم اصلی (Preservation):** کلمه به صورت یکپارچه با خط‌تیره ذخیره می‌شود (مثال: **hewlett-packard**).

2. **تجزیه به اجزای سازنده (Splitting):** کلمه از محل خطتیره شکسته شده و هر جزء به عنوان یک توکن مستقل ذخیره می‌گردد (مثال: hewlett و packard).

3. **حذف نگارشی و ادغام (Concatenation):** کلمات بدون خطتیره به یکدیگر چسبانده می‌شوند تا خطاهای تایپی رایج پوشش داده شوند (مثال: hewlettpackard).

بخش 5) جمع‌بندی تحلیل کارایی

تأثیر بر سرعت و حجم: اجرای استراتژی نمایه‌سازی چندگانه باعث می‌شود برای یک کلمه خطتیره‌دار، چندین توکن در پایگاه داده ذخیره شود. این امر حجم نمایه (**Index Size**) را اندکی افزایش می‌دهد؛ اما از آنجا که این عملیات در فاز آفلاین (**Indexing Time**) رخ می‌دهد، تأثیر منفی بر سرعت جستجوی کاربر ندارد.

تأثیر بر کیفیت نتایج: این روش تضمین می‌کند که فارغ از نحوه نگارش کاربر در نوار جستجو (با خطتیره، با فاصله، یا متصل)، سیستم بتواند سند مرتبط را پیدا کرده و با اعمال وزن‌دهی مناسب به توکن‌های دقیق‌تر، بالاترین سطح از دقت و فراخوانی را به صورت همزمان ارائه دهد.

پاسخ‌نامه مسئله 2.13: مفهوم نشانگر پرشی (Skip Pointers)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در ساختار «نمایه معکوس» (**Inverted Index**)، لیست‌های پستینگ (**Posting Lists**) حاوی شناسه‌های سند (**DocIDs**) هستند که به صورت مرتب‌شده ذخیره شده‌اند. هنگام جستجوی عبارت‌های ترکیبی (مثلاً **A AND B**)، سیستم باید 2 لیست پستینگ مرتب‌شده را پیمایش کرده و اشتراک (**Intersection**) آن‌ها را بیابد. هدف از این مسئله، بررسی یک تکنیک بهینه‌سازی کلاسیک به نام «نشانگر پرشی» (**Skip Pointer**) است که برای افزایش سرعت این پیمایش‌ها و کاهش عملیات مقایسه بدون نیاز به بررسی تک‌تک **DocID**ها ابداع شده است.

بخش 2) هدف اصلی از اضافه کردن Skip Pointerها

هدف بنیادین از پیاده‌سازی **Skip Pointer**ها، کاهش زمان پیمایش لیست‌های طولانی (**Skip-over Optimization**) است. بدون استفاده از این نشانگرها، الگوریتم اشتراک‌گیری مجبور است برای هر دو لیست، **DocID**ها را یکی‌یکی از ابتدا تا انتها مقایسه کند. در لیست‌های بسیار طولانی، این کار هزینه‌ی محاسباتی **O(N+M)** دارد (که **N** و **M** طول دو لیست هستند). **Skip Pointer**ها با افزودن میان‌برها به ساختار لیست، به سیستم اجازه می‌دهند تا بخش‌های بزرگی از لیست که قطعاً در اشتراک‌گیری نهایی نقشی ندارند را «پرش» کرده و نادیده بگیرد.

بخش 3) تأثیر بر عملگرهای منطقی

نشانگرهای پرشی بیشترین تأثیر را بر روی **عملگر منطقی AND** (اشتراک‌گیری) دارند.

استدلال در عملگر AND: در عملیات **AND**، ما به دنبال **DocID**هایی هستیم که در هر دو لیست موجود باشند. اگر در حین مقایسه **DocID** فعلی در لیست اول با لیست دوم، متوجه شویم که مقدار لیست اول بسیار بزرگتر از مقدار لیست دوم است، می‌توانیم با استفاده از **Skip Pointer**ها، لیست دوم را به سرعت به جلو جهش دهیم تا به مقداری نزدیک به مقدار لیست

اول برسیم. این کار باعث می‌شود از مقایسه‌های بیهوده جلوگیری شود و اشتراک‌گیری با جهش‌های بزرگ به سرعت به نتیجه نهایی برسد.

تفاوت در عملگر OR (اجتماع): در این عملگر، معمولاً ما مجبوریم تمام اعضای هر دو لیست را برای بازسازی لیست نهایی مرور کنیم، بنابراین پرش‌های بزرگ کمتر کاربرد دارند (اگرچه همچنان می‌توانند برای ترکیب سریع‌تر استفاده شوند، اما اثر آن‌ها در مقایسه با **AND** ناچیز است).

تفاوت در عملگر NOT (تفریق): ماهیت عملیات به گونه‌ای است که لیست حذف‌کننده (**Negative List**) باید دقیقاً پردازش شود و پرش‌ها در این زمینه پیچیدگی‌های خاص خود را دارند.

بخش 4) نکات تکمیلی مهندسی

محاسبه بهینه فاصله پرش‌ها: یک پرسش مهم در طراحی این سیستم‌ها این است که هر چند **DocID** یک‌بار باید **Skip Pointer** قرار داد؟ یک قاعده کلی مهندسی (**Heuristic**) وجود دارد که می‌گوید اگر طول لیست برابر با **L** باشد، تعداد مناسب **Skip Pointer** ها تقریباً $\sqrt{L}$ است. این مقدار، تعادلی میان «حافظه مصرفی برای ذخیره نشانگرها» و «میزان سرعت‌بخشی به پیمایش» برقرار می‌کند.

محدودیت‌های سخت‌افزاری: در سیستم‌های مدرن، به دلیل وجود حافظه نهان (**Cache Memory**) در پردازنده‌ها، گاهی استفاده از **Skip Pointer** ها به دلیل پرش‌های حافظه‌ای (**Memory Jumps**) ممکن است باعث کاهش نرخ **Cache Hit** شود. بنابراین، مهندسان امروزی اغلب **Skip Pointer** ها را با تکنیک‌های فشرده‌سازی لیست‌ها (مانند فشرده‌سازی مبتنی بر **Varint**) ترکیب می‌کنند تا لیست‌های پستینگ مستقیماً در حافظه اصلی پردازش شوند.

پاسخ‌نامه مسئله 2.14: ساختار بهینه پرش (Optimal Skip Length)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در طراحی سیستم‌های بازیابی اطلاعات، انتخاب فاصله بهینه برای نشانگرهای پرشی (**Skip Pointers**)، یک مسئله کلاسیک بهینه‌سازی است. اگر فاصله پرش‌ها خیلی بزرگ باشد، سیستم زمان زیادی را صرف بررسی لیست‌های پستینگ می‌کند (تأثیر پرش‌ها ناچیز می‌شود) و اگر خیلی کوچک باشد، سیستم زمان زیادی را صرف خواندن نشانگرهای پرشی می‌کند (سربار بیش از حد). هدف این مسئله، استخراج ریاضی «قاعده سرانگشتی» (**Rule of Thumb**) برای تعیین تعداد و فاصله نشانگرها به منظور کمینه‌سازی هزینه محاسباتی است.

بخش 2) تحلیل ریاضی و استخراج مقدار بهینه

فرض کنید طول لیست پستینگ برابر با **L** باشد. هدف ما این است که هزینه‌ی جستجو (شامل بررسی نشانگرها و پیمایش لیست) را به حداقل برسانیم. فرض کنیم فاصله هر پرش برابر با **S** باشد (یعنی هر **S** گام یک نشانگر داریم).

تعداد پرش‌ها: تعداد نشانگرهای پرش برابر خواهد بود با L/S .

هزینه پردازش: هزینه کلی برابر است با مجموع هزینه‌ی بررسی نشانگرهای پرشی و هزینه‌ی پیمایش گام‌های واقعی در لیست. به صورت تقریبی، تابع هزینه **f(S)** به شکل زیر است:

$$f(S) = \frac{L}{S} + S$$

برای یافتن مقدار بهینه S ، از تابع نسبت به S مشتق گرفته و آن را برابر با صفر قرار می‌دهیم:

$$f'(S) = -\frac{L}{S^2} + 1 = 0 \implies S^2 = L \implies S = \sqrt{L}$$

نتیجه‌گیری ریاضی: برای کمینه‌سازی هزینه پیمایش، فاصله بهینه بین نشانگرهای پرشی (**Skip Length**) باید برابر با جذر طول لیست پستینگ یعنی $\sqrt{L}$ باشد. به تبع آن، تعداد بهینه نشانگرهای پرشی نیز تقریباً برابر با $\sqrt{L}$ خواهد بود.

بخش 3) چرا این قاعده سرانگشتی کارآمد است؟ (تحلیل شهودی)

این تساوی $S = \sqrt{L}$ در واقع نقطه تعادل (**Balance Point**) میان دو عملیات هزینه‌بر است:

1. **هزینه بررسی نشانگرها (Skips):** اگر تعداد پرش‌ها خیلی زیاد باشد (یعنی S کوچک باشد)، پردازنده باید مدام با مقادیر پرش کار کند و شرط‌های **if** زیادی را بررسی کند که این کار باعث ایجاد سربار پردازشی (**Overhead**) می‌شود.

2. **هزینه پیمایش مستقیم (Linear Scan):** اگر تعداد پرش‌ها خیلی کم باشد (یعنی S بزرگ باشد)، سیستم مجبور است در فواصل طولانی، لیست را به صورت خطی پیمایش کند که باعث هدر رفتن مزیت اصلی **Skip Pointer** (یعنی نادیده گرفتن بخش‌های غیرمرتبط) می‌شود.

با انتخاب $S = \sqrt{L}$ ، سیستم دقیقاً به اندازه‌ای که برای «تصمیم‌گیری جهت پرش» هزینه می‌کند، برای «پیمایش لیست» نیز هزینه می‌کند. این توازن، کمترین زمان کل برای اجرای عملیات اشتراک‌گیری (**Intersection**) را تضمین می‌کند.

بخش 4) نکات کاربردی در مهندسی نرم‌افزار

تغییر طول لیست (Dynamic Indices): در سیستم‌های واقعی که لیست‌های پستینگ دائماً در حال تغییر هستند (افزودن اسناد جدید)، محاسبه دقیق $\sqrt{L}$ برای هر لیست عملاً ناممکن و هزینه‌بر است. معمولاً مهندسان از یک فاصله ثابت (مثلاً 16، 32 یا 128) استفاده می‌کنند که با معماری خطوط حافظه (**Cache Lines**) سخت‌افزار مدرن سازگاری دارد.

تأثیر سخت‌افزار: امروزه به دلیل وجود سلسله‌مراتب حافظه (**Memory Hierarchy**)، گاهی اوقات تعداد پرش کمتر (فواصل طولانی‌تر) از نظر نرخ **Cache Hit** عملکرد بهتری نسبت به محاسبات ریاضی ایده‌آل دارند، زیرا تعداد پرش‌های پرشی (**Jump**) را به حداقل می‌رسانند.

فشرده‌سازی (Compression): نشانگرهای پرشی باید همراه با داده‌های فشرده‌سازی شده (مثل **Delta-encoding**) استفاده شوند. اگر لیست به صورت فشرده ذخیره شود، پرش کردن بدون باز کردن (**Decompress**) کل لیست غیرممکن است. بنابراین، **Skip Pointer**‌ها معمولاً به «بلوک‌های فشرده» اشاره می‌کنند، نه به تک تک **DocID**‌ها.

پاسخ‌نامه مسئله 2.15: الگوریتم تقاطع با پرش (Intersection Algorithm with Skips)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

هدف از این الگوریتم، بهینه‌سازی عملیات اشتراک‌گیری (**AND**) بین 2 لیست پستینگِ مرتب‌شده است. در حالت عادی (بدون پرش)، اگر مقدار یک اشاره‌گر (**Pointer**) کوچک‌تر از دیگری باشد، ما صرفاً آن را یک گام به جلو می‌بریم. اما با وجود **Skip Pointer**ها، اگر شرایط مهیا باشد (یعنی مقصد پرش، از **DocID** مقابل کمتر یا مساوی باشد)، می‌توانیم به‌جای یک گام، یک «پرش بزرگ» انجام دهیم. این کار تعداد مقایسه‌های تکراری را به حداقل می‌رساند.

بخش 2) شبه‌کد الگوریتم اشتراک‌گیری (Intersection with Skips)

در شبه‌کد زیر، فرض بر این است که **P₁** و **P₂** اشاره‌گرهایی هستند که به لیست‌های پستینگ **L₁** و **L₂** اشاره می‌کنند. هر گره (**Node**) در لیست، شامل یک **docID** و یک **skipPointer** به گره‌های جلوتر است.

intersect_with_skips.algo

```
ALGORITHM Intersect_With_Skips(L1, L2):
    Result = []
    P1 = L1.head
    P2 = L2.head

    WHILE P1 != NULL AND P2 != NULL:
        IF docID(P1) == docID(P2):
            ADD docID(P1) TO Result
            P1 = P1.next
            P2 = P2.next

        ELSE IF docID(P1) < docID(P2):
            // بررسی امکان پرش برای لیست اول
            IF hasSkip(P1) AND docID(skipPointer(P1)) <= docID(P2):
                P1 = skipPointer(P1) // انجام پرش
            ELSE:
                P1 = P1.next // حرکت خطی

        ELSE: // docID(P2) < docID(P1)
            // بررسی امکان پرش برای لیست دوم
            IF hasSkip(P2) AND docID(skipPointer(P2)) <= docID(P1):
                P2 = skipPointer(P2) // انجام پرش
            ELSE:
                P2 = P2.next // حرکت خطی

    RETURN Result
```

بخش 3) تحلیل و تشریح گام‌به‌گام منطق شرطی

برای درک عمیق رفتار این الگوریتم، توجه به شروطِ درون **ELSE IF** بسیار حیاتی است:

1. شرط اول (**hasSkip(P₁)**): سیستم ابتدا بررسی می‌کند که آیا در موقعیت فعلی، اساساً نشانگر پرشی تعریف شده است یا خیر.
2. شرط حیاتی (**docID(skipPointer(P₁)) <= docID(P₂)**): این مهم‌ترین بخش منطق پرش است. ما فقط زمانی اجازه داریم بپریم که مطمئن باشیم با پریدن، از مقدار هدف (**P₂**) عبور نمی‌کنیم. اگر **docID** مقصدِ پرش، بزرگ‌تر از **P₂** باشد، ممکن است سندی که با **P₂** تطابق دارد در میانِ مسیرِ پرش وجود داشته باشد و ما با پریدن، آن را از دست بدهیم (خطای **False Negative**).
3. حرکت خطی (**P₁ = P₁.next**): اگر پرش ممکن نباشد یا شرط آن برقرار نشود، الگوریتم به رفتار کلاسیک بازگشته و لیست را به صورت گام‌به‌گام پیمایش می‌کند تا شانس تطابق حفظ شود.

بخش 4) ملاحظات مهندسی در پیاده‌سازی

سربار (**Overhead**): اگرچه این شبه‌کد ساده است، اما در پیاده‌سازی‌های واقعی، عملیات **hasSkip** و دسترسی به **skipPointer** باید بسیار سریع باشند (معمولاً با ذخیره‌سازی در حافظه نزدیک به گره فعلی).

تأثیر بر پیچیدگی: اگر لیست‌ها دارای N و M عضو باشند، در بدترین حالت بدون **Skip Pointer**، پیچیدگی $O(N+M)$ است. با وجود **Skip Pointer**‌ها، این پیچیدگی در سناریوهای دارای هم‌پوشانی کم، به شدت کاهش می‌یابد و به سمت $O(\sqrt{N} + \sqrt{M})$ میل می‌کند که تفاوت چشمگیری در پرفورمنس موتورهای جستجوی بزرگ ایجاد می‌نماید.

پاسخ‌نامه مسئله 2.16: هزینه‌ی ذخیره‌سازی در مقابل سرعت (Storage vs Speed Trade-off)

بخش 1) تحلیل صورت مسئله و مفاهیم پایه

در طراحی الگوریتم‌ها و معماری سیستم‌های بازیابی اطلاعات، همواره با یکی از بنیادی‌ترین مفاهیم علوم کامپیوتر یعنی «توازن میان فضا و زمان» (**Space-Time Trade-off**) روبرو هستیم. نشانگرهای پرشی (**Skip Pointers**) دقیقاً تجلی همین مفهوم هستند: ما بخشی از فضای ارزشمند حافظه را برای ذخیره داده‌های فراداده‌ای (**Metadata**) فدا می‌کنیم تا سرعت اجرای پرس‌وجو (زمان اشتراک‌گیری) را کاهش دهیم. هدف از این مسئله، ارزیابی دقیق این هزینه تحمیلی و بررسی منطق مهندسی در تصمیم‌گیری برای استفاده یا عدم استفاده از این تکنیک است.

بخش 2) بررسی و محاسبه هزینه ذخیره‌سازی (Storage Overhead)

اضافه کردن نشانگرهای پرشی به لیست‌های پستینگ، ساختار داده را از یک آرایه یا لیست پیوندی ساده، به ساختاری پیچیده‌تر با فیلدهای اضافی تبدیل می‌کند.

تحلیل پیچیدگی مکانی: اگر طول یک لیست پستینگ را L در نظر بگیریم و از قاعده سرانگشتی فاصله بهینه (یعنی قرار دادن یک نشانگر در هر $\sqrt{L}$ گام) استفاده کنیم، سیستم باید تعداد $\sqrt{L}$ نشانگر پرشی برای این لیست ذخیره کند. بنابراین، پیچیدگی مکانی سربار حافظه از مرتبه $O(\sqrt{L})$ خواهد بود.

محاسبه بایت‌های تحمیلی: هر نشانگر پرشی صرفاً یک فلش انتزاعی نیست. در سطح ماشین، هر **Skip Pointer** حداقل به 2 قطعه اطلاعات نیاز دارد: (1) شناسه سند مقصد (**Target DocID**) برای ارزیابی شرط پرش. (2) آدرس فیزیکی یا آفست (**Byte Offset**) برای پرش واقعی اشاره‌گر فایل به آن نقطه. اگر هر کدام از این مقادیر 4 بایت (یک عدد صحیح 32 بیتی) فضا اشغال کنند، هر پرش 8 بایت سربار دارد. در نمایه‌های عظیم که میلیاردها سند و هزاران لیست پستینگ طولانی دارند، این $O(\sqrt{L})$ ‌ها با یکدیگر جمع شده و می‌توانند گیگابایت‌ها فضای اضافی در دیسک یا **RAM (Main Memory)** اشغال کنند.

بخش 3) آیا استفاده از نشانگرهای پرشی همیشه توصیه می‌شود؟

پاسخ به این سوال در مهندسی نرم‌افزار **خیر** است. با وجود اینکه **Skip Pointer**‌ها در تئوری سرعت تقاطع (**Intersection**) را در پرس‌وجوهای **AND** افزایش می‌دهند، اما در سناریوهای زیر استفاده از آن‌ها نه تنها مفید نیست، بلکه مخرب نیز خواهد بود:

پرس‌وجوهای مبتنی بر عملگر OR (اجتماع): در عملگر **OR**، سیستم موظف است تمامی شناسه‌های موجود در هر دو لیست پستینگ را بخواند و ادغام کند. از آنجا که هیچ بخشی از لیست نمی‌تواند نادیده گرفته شود، **Skip Pointer**‌ها کاملاً بلااستفاده می‌مانند و صرفاً فضای حافظه را هدر داده‌اند.

لیست‌های پستینگ کوتاه: اگر یک کلمه کلیدی نادر باشد، لیست پستینگ آن کوتاه است. در لیست‌های کوتاه، هزینه اجرای دستورات شرطی (**If-Else**) برای بررسی امکان پرش، از زمان پیمایش خطی (**Linear Scan**) کل لیست بیشتر می‌شود.

الگوریتم‌ها معمولاً برای لیست‌هایی که طول آن‌ها از یک آستانه مشخص (**Threshold**) کمتر است، **Skip Pointer** تولید نمی‌کنند.

تداخل با معماری حافظه نهان (Cache Memory): پردازنده‌های مدرن (**CPUs**) برای خواندن داده‌های متوالی در حافظه بسیار بهینه‌سازی شده‌اند و داده‌ها را به صورت بلوک‌هایی در **Cache Line** بارگذاری می‌کنند. پیمایش خطی یک آرایه در حافظه اصلی به دلیل مکانیزم پیش‌واکشی (**Prefetching**)، فوق‌العاده سریع است. در مقابل، انجام پرش‌های تصادفی (**Pointer Chasing**) باعث خطای از دست رفتن حافظه نهان (**Cache Miss**) شده و پردازنده را مجبور می‌کند چرخه‌های کلاک زیادی را منتظر واکشی داده از **RAM** بماند.

بخش 4) چالش فشرده‌سازی و رویکرد سیستم‌های مدرن (نکات تکمیلی)

مهم‌ترین دلیلی که موتورهای جستجوی مدرن (نظیر **Lucene** که هسته **Elasticsearch** است) از شکل کلاسیک **Skip Pointer** ها فاصله گرفته‌اند، مبحث **فشرده‌سازی نمایه (Index Compression)** است.

برای کاهش حجم نمایه، شناسه‌های اسناد معمولاً به روش فواصل (**Delta-Encoding**) و الگوریتم‌هایی نظیر **Variable Byte** فشرده می‌شوند. ماهیت این فشرده‌سازی متوالی است؛ یعنی برای فهمیدن مقدار یک **DocID**، باید مقادیر قبلی را از حالت فشرده خارج (**Decompress**) کنید. بنابراین، شما **نمی‌توانید** مستقیماً به وسط یک لیست فشرده‌شده پرش کنید!

راهکار مدرن (Block-based Indexing): به جای تخصیص نشانگر پرش به تکتک گره‌ها، سیستم‌های امروزی لیست‌های پستینگ را به بلوک‌های ثابتی (مثلاً بلوک‌های **128** تایی) تقسیم کرده و هر بلوک را به صورت مجزا فشرده می‌کنند. در این معماری، متادیتاها و اطلاعات پرش تنها در ابتدای هر بلوک (**Block Header**) ذخیره می‌شوند. این روش تعادل بسیار ایده‌آلی میان فشرده‌سازی سنگین داده‌ها، حفظ ساختار **Cache-friendly** و امکان پرش‌های بزرگ در زمان اجرای پرس‌وجو ایجاد می‌کند.

پاسخ‌نامه مسئله 2.17: تأثیر فرکانس کلمات (Word Frequency Impact)

بخش 1) تحلیل صورت مسئله و مفاهیم پایه

در مباحث بازیابی اطلاعات، طول یک لیست پستینگ (**Posting List**) ارتباط مستقیمی با میزان تکرار یا فرکانس آن کلمه در مجموعه اسناد (**Corpus**) دارد. کلمات رایج (مانند «سیستم»، «اطلاعات» یا «الگوریتم») لیست‌های بسیار طولانی دارند، در حالی که کلمات کمیاب یا تخصصی (مانند «کوکتل‌شیکر» یا یک نام خاص) لیست‌های بسیار کوتاهی ایجاد می‌کنند.

هدف از این مسئله، بررسی تحلیل هزینه-فایده (**Cost-Benefit Analysis**) در استفاده از نشانگرهای پرشی (**Skip Pointers**) بر اساس طول لیست و درک دلیل اثربخشی بالای این تکنیک برای کلمات پرکاربرد است.

بخش 2) رفتار سیستم در مواجهه با کلمات کمیاب (Low-frequency terms)

زمانی که یک کلمه کمیاب است، لیست پستینگ آن شامل تعداد انگشت‌شماری شناسه سند (**DocID**) خواهد بود. در این حالت، استفاده از نشانگر پرشی نه تنها سودمند نیست، بلکه باعث افت کارایی می‌شود:

پیمایش خطی سریع: برای یک لیست کوتاه (مثلاً با **20** عضو)، پیمایش خطی (**Linear Scan**) در پردازنده‌های مدرن در کسری از نانوثانیه و مستقیماً از روی حافظه نهان (**Cache**) انجام می‌شود.

سربرار شرطی (Conditional Overhead): اگر برای این لیست کوتاه نشانگر پرشی تعبیه کنیم، پردازنده باید برای هر گام، دستورات شرطی (آیا نشانگر وجود دارد؟ آیا مقصد پرش از هدف کوچکتر است؟) را ارزیابی کند. این ارزیابی‌ها زمان‌برتر از خود مقایسه مستقیم در یک لیست کوتاه است. در نتیجه، برای کلمات کمیاب، سربرار مدیریت پرش‌ها بر مزایای آن غلبه می‌کند.

بخش 3) رفتار سیستم در مواجهه با کلمات رایج (High-frequency terms)

ارزش واقعی نشانگرهای پرشی زمانی مشخص می‌شود که پای لیست‌های پستینگ صدها هزار یا میلیونی به میان بیاید. بیشترین سودمندی این تکنیک در **تقاطع (Intersection)** یک کلمه رایج با یک کلمه کمیاب خود را نشان می‌دهد:

جهش‌های عظیم (Massive Leaps): فرض کنید دانشجویی پرس‌وجوی «الگوریتم AND کوکتل‌شیکر» را جستجو کرده است. کلمه «الگوریتم» یک کلمه رایج با 1,000,000 سند (لیست طولانی) و کلمه «کوکتل‌شیکر» یک کلمه کمیاب با تنها 5 سند (لیست کوتاه) است.

بدون نشانگر پرشی: سیستم مجبور است برای یافتن اشتراک با اولین سند کلمه کمیاب، لیست میلیونی را به صورت خطی و گام‌به‌گام به جلو ببرد که به معنای هزاران مقایسه بیهوده است.

با نشانگر پرشی: اشاره‌گر لیست طولانی می‌تواند با استفاده از **Skip Pointer**ها، بخش‌های عظیمی از لیست را که فاقد سند مورد نظر هستند به یکباره جا بگذارد. سیستم با چند پرش بزرگ خود را به محدوده عددی لیست کوتاه می‌رساند و محاسبات از مرتبه $O(N)$ به شکل چشمگیری به سمت $O(\sqrt{N})$ کاهش می‌یابد.

بخش 4) نکات تکمیلی و جمع‌بندی مهندسی برای طراحی موتور جستجو

با توجه به این رفتار ریاضی و منطقی، در پیاده‌سازی عملی و مهندسی موتورهای جستجو (طراحی ساختار **Inverted Index**)، یک قانون قطعی وجود دارد: نشانگرهای پرشی برای تمامی کلمات به صورت یکسان تولید نمی‌شوند.

تعریف آستانه (Thresholding): مهندسان برای ساخت نمایه، یک مقدار آستانه طول (**Minimum List Length**) تعریف می‌کنند. سیستم در زمان نمایه‌سازی (**Indexing**)، تنها برای کلماتی نشانگر پرشی و ساختارهای جهش‌دار تولید می‌کند که طول لیست پستینگ آن‌ها از این مقدار آستانه عبور کرده باشد.

بهینه‌سازی پویا: این رویکرد دوگانه باعث می‌شود تا سیستم به طور هوشمندانه‌ای، از هدررفت حافظه (**Storage Overhead**) برای کلمات کمیاب جلوگیری کرده و همزمان، سرعت اجرای پرس‌وجو (**Query Latency**) را برای کلمات رایج به حداقل برساند. چنین طراحی دقیقی، مرز میان یک سیستم آموزشی ساده و یک موتور جستجوی صنعتی با مقیاس‌پذیری بالا (**Scalable**) را مشخص می‌کند.

پاسخ‌نامه مسئله 2.18: نمایه‌های دوکلمه‌ای (Biword Indexes)

بخش 1) تحلیل صورت مسئله و مفاهیم پایه

در معماری کلاسیک سیستم‌های بازیابی اطلاعات، نمایه معکوس (**Inverted Index**) بر اساس تک‌کلمات یا تک‌گرام‌ها (**Unigrams**) ساخته می‌شود. با این حال، یکی از نیازمندی‌های پرکاربرد کاربران، «جستجوی عبارتی» (**Phrase Query**) است؛ یعنی زمانی که کاربر می‌خواهد کلمات دقیقاً با همان ترتیب مشخص‌شده در کنار یکدیگر قرار داشته باشند (مثلاً جستجوی "دانشگاه تهران").

نمایه دوکلمه‌ای (Biword Index) یک راهکار ساختاری برای تسریع پاسخ‌گویی به این نوع پرس‌وجوهاست. در این مدل، به جای آنکه هر تک‌کلمه به عنوان یک توکن مجزا در نظر گرفته شود، هر «**2 کلمه متوالی و مجاور**» در متن سند، با یکدیگر ترکیب شده و به عنوان یک مدخل (**Entry**) واحد در لغت‌نامه ذخیره می‌گردند.

بخش 2) تشریح گام‌به‌گام: استخراج بی‌وردها برای عبارت داده‌شده

برای تولید نمایه‌های دوکلمه‌ای، سیستم استخراج‌گر یک پنجره کشویی (**Sliding Window**) به طول **2** کلمه را روی متن حرکت می‌دهد و در هر گام، **1** واحد به جلو می‌رود.

برای عبارت **Information Retrieval Systems**، فرآیند توکن‌گذاری به شرح زیر خواهد بود:

گام اول (پنجره روی کلمات اول و دوم): استخراج توکن ترکیبی [**Information Retrieval**]

گام دوم (پنجره روی کلمات دوم و سوم): استخراج توکن ترکیبی [**Retrieval Systems**]

بنابراین، این عبارت **3** کلمه‌ای، دقیقاً **2** بی‌ورد (**Biword**) در نمایه تولید می‌کند. در زمان جستجو، اگر کاربر عبارت "**Information Retrieval**" را جستجو کند، سیستم بدون نیاز به محاسبه فاصله کلمات، مستقیماً این توکن دوکلمه‌ای را پیدا کرده و سند را در سریع‌ترین زمان ممکن بازیابی می‌کند. اگر کاربر کل عبارت **3** کلمه‌ای را جستجو کند، موتور جستجو آن را به **Information Retrieval AND Retrieval Systems** شکسته و لیست پستینگ این **2** بی‌ورد را با هم اشتراک‌گیری می‌کند.

بخش 3) تحلیل کارایی و محدودیت‌های مهندسی (Trade-offs)

در طراحی سیستم‌های اطلاعاتی، استفاده از **Biword Index** با توازن از مزایا و معایب همراه است که یک مهندس نرم‌افزار باید به آن دقت کند:

مزیت (سرعت بی‌نظیر): جستجوی عبارات دوکلمه‌ای را از یک عملیات پیچیده و زمان‌بر به یک جستجوی ساده و زمان‌ثابت تقلیل می‌دهد.

چالش اصلی (انفجار لغت‌نامه - Vocabulary Explosion): بزرگترین مشکل این روش، رشد نجومی ابعاد لغت‌نامه (**Dictionary Size**) است. از نظر آماری، تعداد ترکیبات دوکلمه‌ای ممکن در یک زبان طبیعی بسیار بیشتر از تعداد تک‌کلمات است. ساخت و نگهداری یک نمایه دوکلمه‌ای برای تمام اسناد، فضای ذخیره‌سازی (**Storage**) عظیمی را اشغال کرده و منجر به افت سرعت در فاز نمایه‌سازی می‌گردد.

پدیده تقاطع کاذب (False Positives): اگر کاربر عبارت "**Nylon University**" را جستجو کند و سیستم آن را به **Nylon AND University** بشکند، ممکن است سندی بازیابی شود که شامل جمله "...using Nylon. University of..." باشد. در اینجا **2** کلمه به دلیل وجود نقطه به هم مرتبط نیستند، اما سیستم مبتنی بر بی‌ورد ممکن است آن‌ها را اشتباه ارزیابی کند.

بخش 4) نکات تکمیلی مهندسی (رویکرد مدرن)

جایگزین در موتورهای جستجوی مدرن:

به دلیل چالش‌های مربوط به حافظه، موتورهای جستجوی مدرن امروزی هرگز متن کامل اسناد را به صورت **Biword** نمایه‌سازی نمی‌کنند. در عوض، از رویکردهای ترکیبی استفاده می‌شود: سیستم‌های امروزی تنها برای **موجودیت‌های نام‌دار (Named Entities)**، اسامی خاص و عبارات بسیار پرتکرار (مثل "**United States**" یا "**Computer Science**") یک مدخل

پاسخ‌نامه مسئله 2.19: محدودیت‌های ایندکس دوکلمه‌ای (Biword Index Limitations)

بخش 1) تحلیل صورت مسئله و مفاهیم پایه

در مباحث پیشین بررسی کردیم که نمایه دوکلمه‌ای (Biword Index) برای تسریع جستجوی عبارات دوکلمه‌ای بسیار کارآمد است. اما زمانی که کاربر به دنبال یک عبارت دقیق و طولانی (Phrase Query) شامل 3، 4 یا 5 کلمه متوالی باشد، معماری بی‌ورد دچار یک نقطه ضعف بنیادین به نام «کوری موقعیتی» (Positional Blindness) می‌شود. هدف از این مسئله، درک دلیل بروز خطای «مثبت کاذب» (False Positive) یعنی بازیابی سندی که از نظر ماشین مرتبط ارزیابی شده اما در واقعیت فاقد عبارت مورد نظر کاربر است در مواجهه با پرس‌وجوهای طولانی است.

بخش 2) تشریح گام‌به‌گام: مکانیسم پردازش عبارات طولانی

هنگامی که سیستم موتور جستجو (که صرفاً مبتنی بر بی‌ورد است) یک پرس‌وجوی 5 کلمه‌ای مانند $W_1 W_2 W_3 W_4 W_5$ را دریافت می‌کند، به دلیل عدم وجود نمایه‌های سه‌کلمه‌ای یا بیشتر، مجبور است این عبارت یکپارچه را به قطعات دوکلمه‌ای بشکند و با استفاده از عملگر منطقی AND آن‌ها را جستجو کند.

بنابراین، پرس‌وجوی کاربر به شکل زیر ترجمه و اجرا می‌شود:

$(W_1 W_2) \text{ AND } (W_2 W_3) \text{ AND } (W_3 W_4) \text{ AND } (W_4 W_5)$

سیستم در لغت‌نامه خود به دنبال اسنادی می‌گردد که تمامی این 4 بی‌ورد در آن‌ها وجود داشته باشند.

بخش 3) ریشه مشکل: گسستگی ساختاری (Structural Disconnection)

ریشه خطای مثبت کاذب در کلمه AND نهفته است. عملگر AND تنها «حضور» این قطعات را در یک سند تضمین می‌کند، اما هیچ‌گونه تضمینی در مورد ترتیب، توالی و مجاورت این قطعات نسبت به یکدیگر ارائه نمی‌دهد.

به عبارت علمی‌تر، ماشین بررسی می‌کند که آیا حلقه اول و دوم زنجیر وجود دارند یا خیر، اما نمی‌تواند بررسی کند که آیا این حلقه‌ها واقعاً به هم متصل هستند تا یک زنجیر 5 کلمه‌ای را تشکیل دهند؟ این قطعات می‌توانند در پاراگراف‌ها یا حتی صفحات مختلفی از یک سند پراکنده باشند.

بخش 4) مثال عملی و اثبات بروز خطای کاذب

برای درک دقیق این فاجعه در جستجو، یک پرس‌وجوی 5 کلمه‌ای کلاسیک انگلیسی را بررسی می‌کنیم. فرض کنید کاربر عبارت زیر را جستجو کرده است:

"The quick brown fox jumps"

سیستم این عبارت را به 4 بی‌ورد زیر می‌شکند و آن‌ها را با عملگر AND جستجو می‌کند:

1. The quick
2. quick brown
3. brown fox
4. fox jumps

اکنون فرض کنید در پایگاه داده ما، سندی با متن زیر وجود دارد:

"The quick brown dog barked at the tree. A small brown fox watched from a distance. Suddenly, the fox jumps over the fence."

تحلیل ماشین (موتور جستجو):

سیستم متن سند را بررسی می‌کند:

بررسی اول: آیا بی‌ورد **The quick** در سند هست؟ بله.

بررسی دوم: آیا بی‌ورد **quick brown** در سند هست؟ بله.

بررسی سوم: آیا بی‌ورد **brown fox** در سند هست؟ بله.

بررسی چهارم: آیا بی‌ورد **fox jumps** در سند هست؟ بله.

از آنجا که تمام شروط **AND** برقرار است، ماشین این سند را به عنوان یک تطابق کامل (**Exact Match**) ارزیابی کرده و با بالاترین امتیاز به کاربر نشان می‌دهد.

تحلیل انسان (کاربر):

آیا عبارت یکپارچه **5** کلمه‌ای **"The quick brown fox jumps"** در متن وجود دارد؟ **خیر!** کلمات به صورت گسسته در **3** جمله مجزا پخش شده‌اند. این دقیقاً تعریف پدیده مثبت کاذب (**False Positive**) در سیستم‌های بازیابی اطلاعات است.

بخش 5) راهکار جایگزین در معماری مدرن

به دلیل همین محدودیت فلج‌کننده، مهندسان نرم‌افزار در موتورهای جستجوی صنعتی مدرن، ایده اتکای انحصاری به **Biword Index** را سال‌هاست که کنار گذاشته‌اند. راهکار استاندارد امروزی برای جستجوی عبارات با هر طولی، استفاده از **نمایه موقعیتی (Positional Index)** است.

در نمایه موقعیتی، موتور جستجو علاوه بر ذخیره شناسه سند (**DocID**)، **موقعیت عددی** هر کلمه در متن را نیز ذخیره می‌کند. در زمان جستجوی عبارت **5** کلمه‌ای فوق، سیستم به جای خرد کردن آن به دوکلمه‌ای‌ها، به سادگی بررسی می‌کند که آیا کلمه دوم دقیقاً در موقعیت (موقعیت کلمه اول + 1)، کلمه سوم در (موقعیت کلمه اول + 2) و به همین ترتیب تا انتها قرار گرفته‌اند یا خیر. این روش خطای تقاطع کاذب را در عبارات طولانی به طور کامل از بین می‌برد.

پاسخ‌نامه مسئله 2.20: ساختار نمایه‌های موقعیتی (Positional Indexes)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در جستجوهای پیشرفته (مانند جستجوی عبارتی یا جستجوی مجاورت - **Proximity Search**)، سیستم تنها به دانستن «وجود» یک کلمه در سند نیاز ندارد، بلکه باید بداند آن کلمه دقیقاً در «کجای» سند قرار گرفته است. هدف از این مسئله، بررسی ساختار داده‌ای پیشرفته‌ای به نام نمایه موقعیتی (**Positional Index**) است که با ذخیره آدرس دقیق هر توکن در

متن، امکان محاسبه فواصل کلمات را برای موتور جستجو فراهم می‌کند و مشکل «کوری موقعیتی» در نمایه‌های ساده را برطرف می‌سازد.

بخش 2) رسم ساختار منطقی لیست پستینگ موقعیتی

در نمایه معکوس استاندارد، لیست پستینگ تنها حاوی شناسه‌های سند است. اما در نمایه موقعیتی، این ساختار بسط یافته و 1 بُعد جدید (آرایه موقعیت‌ها) به آن افزوده می‌شود. ساختار منطقی این نمایه برای 2 کلمه فرضی به شکل زیر رسم می‌شود:

positional_index_structure.txt

Vocabulary (لغتنامه)	Posting Lists (لیست‌های پستینگ موقعیتی)
Information	---> [DocID: 42, TF: 3, Pos: (10, 45, 88)] ---> [DocID: 99, TF: 1, Pos: (5)]
Retrieval	---> [DocID: 15, TF: 1, Pos: (9)] ---> [DocID: 42, TF: 2, Pos: (11, 50)]

بخش 3) اطلاعات الزامی در هر رکورد برای محاسبه فاصله

برای آنکه الگوریتم موتور جستجو بتواند فاصله میان 2 کلمه را با موفقیت محاسبه کند، هر «رکورد» (گره) در این لیست پستینگ باید دقیقاً شامل 3 جزء اطلاعاتی کلیدی باشد:

جزء اول - شناسه سند (Document ID / DocID): اولین شرط برای محاسبه فاصله این است که هر 2 کلمه باید در یک سند مشترک حضور داشته باشند. این شناسه برای یافتن تقاطع اسناد (**Intersection**) و اطمینان از هم‌مکانی کلمات استفاده می‌شود.

جزء دوم - فرکانس کلمه (Term Frequency / TF): این فیلد نشان‌دهنده تعداد دفعات تکرار کلمه در آن سند خاص است. حضور این عدد برای مدیریت حافظه در سطح سیستم‌عامل و تعیین مرز خواندن آرایه موقعیت‌ها در زمان پردازش (**Parsing**) ضروری است.

جزء سوم - آرایه موقعیت‌ها (Positions Array): مهم‌ترین بخش ساختار که شماره ترتیب (**Index**) قرارگیری کلمه در متن را ذخیره می‌کند. اگر کلمه‌ای به تعداد دفعات مشخصی در یک سند تکرار شده باشد، این آرایه شامل همان تعداد عدد صحیح و مرتب‌شده (به صورت صعودی) خواهد بود که جایگاه دقیق هر تکرار را نشان می‌دهند.

بخش 4) تشریح گام‌به‌گام: منطق محاسبه «فاصله بین کلمات»

برای درک نحوه استفاده از این اطلاعات، فرض کنید کاربر عبارت "**Information Retrieval**" را جستجو کرده است. موتور جستجو باید بررسی کند که آیا این 2 کلمه دقیقاً و بدون هیچ فاصله‌ای پشت سر هم آمده‌اند یا خیر.

گام اول (یافتن سند مشترک): سیستم لیست پستینگ هر 2 کلمه را پیمایش کرده و سندی را می‌یابد که هر 2 در آن حضور دارند (مطابق رسم بالا، هر 2 در سند 42 وجود دارند).

گام دوم (واکشی آرایه موقعیت‌ها): آرایه موقعیت‌های هر 2 کلمه در سند 42 استخراج می‌شود:

- موقعیت‌های **Information** در سند 42: (10, 45, 88)
- موقعیت‌های **Retrieval** در سند 42: (11, 50)

گام سوم (محاسبه تفاضل و ارزیابی): سیستم با استفاده از 2 اشاره‌گر، مقادیر این 2 آرایه را مقایسه می‌کند. با کسر اولین موقعیت کلمه اول از کلمه دوم (11 منهای 10)، تفاضل عدد 1 به دست می‌آید. از آنجا که فاصله دقیقاً برابر با 1 واحد است،

سیستم اثبات می‌کند که این 2 کلمه در موقعیت‌های 10 و 11، دقیقاً متوالی و چسبیده به هم قرار گرفته‌اند و سند به عنوان 1 پاسخ کاملاً مرتبط (Exact Match) بازیابی می‌شود.

بخش 5) تحلیل کارایی و چالش‌های مهندسی

اضافه کردن آرایه موقعیت‌ها، اگرچه دقت معنایی و قابلیت‌های جستجو را به شدت ارتقا می‌دهد، اما حجم نمایه معکوس را به شکل چشمگیری متورم می‌کند (معمولاً حجم نمایه به 30 الی 50 درصد حجم کل متون خام می‌رسد).

به همین دلیل، در طراحی موتورهای جستجوی صنعتی مدرن، مقادیر داخل آرایه موقعیت‌ها به صورت خام ذخیره نمی‌شوند؛ بلکه به شکل «فواصل متوالی» (Delta-Encoding) محاسبه شده (مثلاً به جای 10 و 45، اعداد 10 و 35 ذخیره می‌شوند) و سپس با استفاده از الگوریتم‌های فشرده‌سازی در سطح بیت (مانند Variable Byte یا Elias Gamma) متراکم می‌گردند تا سرعت خواندن از روی دیسک (Disk I/O) دچار افت نگردد.

پاسخ‌نامه مسئله 2.21: مقایسه فضای مصرفی (Storage Comparison)

بخش 1) تحلیل صورت مسئله و مفاهیم پایه

در طراحی معماری موتورهای جستجو، برآورد حافظه و فضای ذخیره‌سازی (Capacity Planning) یکی از ارکان اصلی است. نمایه موقعیتی (Positional Index) ابزار قدرتمندی برای پشتیبانی از جستجوی عبارتی (Phrase Queries) و جستجوی مجاورت (Proximity Search) است، اما این قدرت با هزینه سنگین افزایش حجم نمایه همراه است. حجم یک نمایه موقعیتی معمولاً بین 30 تا 50 درصد حجم کل متون خام (Corpus) است، در حالی که این رقم برای یک نمایه استاندارد بسیار کمتر است. هدف از این مسئله، کالبدشکافی ساختار داده و درک دلایل این تورم حافظه است.

بخش 2) دلیل اصلی اختلاف حجم: سطح دانه‌بندی (Granularity Level)

تفاوت بنیادین این دو نمایه در سطح ارجاع آن‌هاست:

نمایه استاندارد (Non-positional): در سطح «سند» (Document-level) کار می‌کند. اگر یک کلمه 100 بار در یک سند تکرار شده باشد، سیستم تنها 1 رکورد شامل شناسه سند (DocID) و فرکانس آن ($TF = 100$) ذخیره می‌کند.

نمایه موقعیتی (Positional): در سطح «توکن» (Token-level) کار می‌کند. اگر کلمه‌ای 100 بار در یک سند تکرار شود، سیستم علاوه بر شناسه سند و فرکانس، باید 100 عدد صحیح متوالی را که نشان‌دهنده جایگاه دقیق هر تکرار است، در یک آرایه ذخیره کند.

بنابراین، حجم نمایه استاندارد به تعداد اسناد و تنوع واژگان وابسته است، اما حجم نمایه موقعیتی به **تعداد کل کلمات موجود در کل مجموعه اسناد** گره خورده است.

بخش 3) تشریح گام‌به‌گام: عوامل مؤثر بر افزایش حجم (Factors of Size Bloat)

3 عامل مهندسی و ساختاری زیر بیشترین تأثیر را در تورم حجم نمایه موقعیتی دارند:

عامل اول - آرایه موقعیت‌ها برای کلمات با فرکانس بالا (High TF Arrays): هرچه فرکانس تکرار یک کلمه (TF) در یک سند بیشتر باشد، آرایه موقعیت‌های آن طولانی‌تر خواهد بود. ذخیره یک عدد صحیح 32 بیتی (4 بایت) برای هر موقعیت، در اسناد

طولانی به سرعت حجم عظیمی از مگابایت‌ها را اشغال می‌کند. به عنوان مثال، کلمه‌ای مانند "اطلاعات" در یک کتاب 500 صفحه‌ای ممکن است هزاران بار تکرار شود و به ازای هر تکرار، یک شناسه موقعیت فیزیکی به پایگاه داده اضافه می‌گردد.

عامل دوم - الزام به حفظ واژه‌های توقف (Stop Words Retention): همان‌طور که در مسائل پیشین بررسی شد، نمایه موقعیتی معمولاً برای پاسخگویی به جستجوهای عبارتی دقیق (مثل ضرب‌المثل‌ها یا نام‌های خاص) ساخته می‌شود. لازمه این کار، عدم حذف واژه‌های توقف (مانند «از»، «به»، «در»، "the"، "is") است، زیرا حذف آن‌ها فواصل موقعیتی کلمات را به هم می‌ریزد. از آنجا که واژه‌های توقف بیش از 30 درصد حجم هر متن طبیعی را تشکیل می‌دهند، ذخیره آرایه موقعیت برای تک‌تک تکرارهای این کلمات پرکاربرد، باعث انفجار حجم نمایه (Index Bloat) می‌شود.

عامل سوم - بزرگی اعداد موقعیت در اسناد طولانی (Large Position Integers): در یک سند بسیار طولانی، اعداد نشان‌دهنده موقعیت به شدت بزرگ می‌شوند. مثلاً آخرین کلمه در یک کتاب ممکن است در موقعیت 350,000 قرار داشته باشد. ذخیره این اعداد بزرگ، حتی با وجود الگوریتم‌های فشرده‌سازی، نسبت به شناسه‌های کوچک و متراکم، به فضای بیشتری از بیت‌ها نیاز دارد.

بخش 4) تحلیل کارایی و راهکار سیستم‌های مدرن

با وجود این حجم عظیم، استفاده از نمایه موقعیتی در موتورهای جستجوی مدرن امری گریزناپذیر است. مهندسان نرم‌افزار برای مهار این هزینه‌ی ذخیره‌سازی از 2 تکنیک اصلی استفاده می‌کنند:

1. کدگذاری تفاضلی درون‌آرایه‌ای (Delta-Encoding): به جای ذخیره اعداد مطلق (مثلاً موقعیت‌های 10، 45، 88)، فاصله هر موقعیت از موقعیت قبلی ذخیره می‌شود (یعنی 10، 35، 43). این کار باعث می‌شود اعداد کوچک‌تر شده و با بایت‌های کمتری قابل ذخیره باشند.

2. الگوریتم‌های فشرده‌سازی مقادیر صحیح (Integer Compression): استفاده از روش‌هایی مانند Variable Byte Encoding یا Simple9 برای فشرده‌سازی لیست‌های پستینگ، که می‌توانند حجم نمایه موقعیتی را تا حد قابل قبولی برای پردازش در حافظه اصلی (RAM) کاهش دهند.

پاسخ‌نامه مسئله 2.22: پرس‌وجوهای مجاورت (Proximity Queries)

بخش 1) تحلیل صورت مسئله و مفاهیم پایه

در سیستم‌های بازیابی اطلاعات پیشرفته، کاربران همواره به دنبال تطابق دقیق کلمات (**Exact Phrase**) نیستند. گاهی نیاز است 2 مفهوم در 1 زمینه متنی نزدیک به یکدیگر به کار رفته باشند، اما لزوماً بلافاصله پشت سر هم نیایند. عملگر مجاورت یا نزدیکی (**Proximity Operator**) دقیقاً برای پوشش این نیاز طراحی شده است.

در پرس‌وجویی با فرمت **Gates /3 Microsoft** (که عملگر **/k** در آن نشان‌دهنده پارامتر فاصله است)، سیستم به دنبال اسنادی می‌گردد که هر 2 کلمه **Gates** و **Microsoft** در آن‌ها وجود داشته باشند، با این شرط حیاتی که فاصله بین این 2 کلمه، حداکثر 3 کلمه (توکن) باشد. به عبارت دیگر، در متن سند، میان این 2 کلمه نهایتاً می‌تواند 0، 1، 2 یا 3 کلمه واسط قرار گرفته باشد. معمولاً در عملگرهای استاندارد **/k**، ترتیب قرارگیری کلمات اهمیتی ندارد (یعنی هم **Gates [words] Microsoft** و هم **Microsoft [words] Gates** مورد قبول است).

بخش 2) نقش نمایه موقعیتی (Positional Index) در پاسخ‌گویی

همان‌طور که در مسائل پیشین اشاره شد، **1** نمایه معکوس استاندارد (**Non-positional**) تنها قادر است به وسیله عملگر **AND** تأیید کند که هر **2** کلمه در **1** سند وجود دارند؛ اما کاملاً نسبت به «مکان» آن‌ها نابینا (**Blind**) است.

برای پاسخ به پرس‌وجوهای مجاورت، استفاده از **نمایه موقعیتی (Positional Index)** الزامی و غیرقابل جایگزین است. نقش این ساختار داده به شرح زیر است:

نگهداری آرایه موقعیت‌ها: این نمایه برای هر کلمه در هر سند، **1** «آرایه از موقعیت‌ها» (**Positions Array**) را نگهداری می‌کند که نشان‌دهنده شماره جایگاه آن توکن در متن است.

محاسبه فاصله فیزیکی: موتور جستجو با در اختیار داشتن این موقعیت‌های عددی خام، می‌تواند با استفاده از عملیات تفریق ساده ریاضی، فاصله فیزیکی میان هر **2** تکرار از این کلمات را در متن محاسبه کرده و شرط مجاورت را ارزیابی کند.

بخش 3) تشریح گام‌به‌گام: الگوریتم پردازش پرس‌وجوی مجاورت

هنگامی که سیستم پرس‌وجوی **Gates /3 Microsoft** را دریافت می‌کند، الگوریتم زیر را اجرا می‌نماید:

گام اول (اشتراک‌گیری در سطح سند): ابتدا سیستم لیست پستینگ هر **2** کلمه را می‌خواند و اسنادی را پیدا می‌کند که شناسه آن‌ها (**DocID**) در هر **2** لیست مشترک باشد. اسنادی که تنها حاوی **1** کلمه از کلمات هستند به سرعت فیلتر و حذف می‌شوند.

گام دوم (استخراج آرایه‌های موقعیت): برای هر سند مشترک یافت‌شده، سیستم آرایه موقعیت‌های کلمه **Gates** و آرایه موقعیت‌های کلمه **Microsoft** را به حافظه اصلی می‌آورد.

گام سوم (پیمایش دوگانه و ارزیابی شرط ریاضی): سیستم با استفاده از **1** الگوریتم مبتنی بر **2** اشاره‌گر (**Two-pointer Algorithm**)، موقعیت‌ها را مقایسه می‌کند. از نظر ریاضی، اگر موقعیت کلمه اول **P₁** و موقعیت کلمه دوم **P₂** باشد، شرط تطابق مجاورت به صورت زیر ارزیابی می‌شود:

$$|P_1 - P_2| \leq k + 1$$

(نکته: دلیل استفاده از **k + 1** این است که اگر **2** کلمه کاملاً چسبیده به یکدیگر باشند، تعداد کلمات واسط میان آن‌ها **k = 0** است، اما تفاضل جایگاه آن‌ها برابر با **1** خواهد بود).

گام چهارم (تأیید و بازیابی): اگر در جریان مقایسه آرایه‌ها، حداقل **1** زوج موقعیت یافت شود که شرط فوق را برآورده کند، آن سند به عنوان **1** «تطابق موفق» نشانه‌گذاری شده و به کاربر بازگردانده می‌شود.

بخش 4) تحلیل کارایی و چالش‌های مهندسی

پردازش پرس‌وجوهای مجاورت از نظر محاسباتی بسیار سنگین‌تر از پرس‌وجوهای بولی ساده (**Boolean AND**) است.

پیچیدگی زمانی: اگر کلمه اول در سند مشترک **N** بار و کلمه دوم **M** بار تکرار شده باشد، سیستم در بدترین حالت باید **N × M** موقعیت را با یکدیگر مقایسه کند. هرچند با توجه به مرتب‌بودن آرایه موقعیت‌ها، این زمان با الگوریتم‌های بهینه به مرتبه زمانی خطی متناسب با تعداد کل موقعیت‌ها کاهش می‌یابد.

کاربرد تجاری: در موتورهای جستجوی مدرن صنعتی، از فواصل مجاورت به عنوان سیگنالی قدرتمند برای **رتبه‌بندی (Ranking)** استفاده می‌شود. به این معنا که حتی اگر کاربر عملگر **k** را وارد نکرده باشد، الگوریتم‌هایی نظیر **BM25F** یا مدل‌های یادگیری

ماشین، اسنادی را که کلمات کلیدی پرس‌وجو در آن‌ها به یکدیگر نزدیک‌ترند (با **Proximity Score** بالاتر)، در رتبه‌های بسیار بالاتری از صفحه نتایج (**SERP**) قرار می‌دهند.

پاسخ‌نامه مسئله 2.23: ترکیب روش‌ها (Combination Schemes)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در طراحی معماری موتورهای جستجو، ما همواره با 2 ساختار داده‌ای رقیب برای پردازش جستجوهای عبارتی (**Phrase Queries**) مواجه هستیم: نمایه دوکلمه‌ای (**Biword Index**) که سرعت اجرای بی‌نظیری دارد اما حافظه عظیمی می‌طلبد، و نمایه موقعیتی (**Positional Index**) که حافظه را بهینه‌تر مصرف می‌کند اما به دلیل نیاز به محاسبه فواصل در زمان اجرا، سرعت پاسخگویی پایین‌تری دارد.

هدف از این مسئله، بررسی رویکرد «نمایه‌های ترکیبی» یا «چندسطحی» (**Tiered/Hybrid Indexes**) است؛ 1 معماری هوشمندانه که مزایای هر 2 روش را استخراج کرده و به 1 توازن ایده‌آل میان «فضا» (**Space**) و «زمان» (**Time**) دست می‌یابد.

بخش 2) ایده نمایه‌های چندسطحی (Tiered / Hybrid Indexes)

ایده اصلی در معماری نمایه‌های چندسطحی این است که رفتار کاربران در موتورهای جستجو از 1 توزیع یکنواخت پیروی نمی‌کند؛ بلکه تابع قانون زیف (**Zipf's Law**) است. یعنی درصد کمی از عبارات (مانند "**Computer Science**" یا "**New York**") درصد بسیار بالایی از کل جستجوها را تشکیل می‌دهند.

بر این اساس، موتور جستجو به جای استفاده از 1 ساختار یکپارچه، نمایه خود را به 2 سطح (**Tier**) تقسیم می‌کند:

سطح اول (فست‌ترک یا Biword): 1 نمایه دوکلمه‌ای کوچک و بهینه‌سازی‌شده که تنها عبارات بسیار پرتکرار، نام‌های خاص و موجودیت‌های شناخته‌شده در آن ذخیره می‌شوند.

سطح دوم (نمایه جامع یا Positional): 1 نمایه موقعیتی کامل که تمامی تک‌کلمات (**Unigrams**) متن همراه با آرایه موقعیت‌هایشان در آن ذخیره می‌گردند.

هنگامی که کاربری 1 عبارت را جستجو می‌کند، سیستم ابتدا سطح اول را بررسی می‌کند. اگر عبارت از نوع پرتکرار باشد، پاسخ را در کسری از ثانیه از نمایه **Biword** بازمی‌گرداند. اما اگر عبارت در سطح اول یافت نشد (مثلاً 1 جمله خاص و طولانی)، سیستم به سراغ سطح دوم رفته و با استفاده از محاسبات نمایه موقعیتی، اسناد مرتبط را استخراج می‌کند.

بخش 3) استدلال مهندسی: چرا برخی عبارات Biword و برخی Positional ذخیره می‌شوند؟

دلیل این تقسیم‌بندی، مدیریت بهینه منابع سخت‌افزاری (پردازنده و حافظه) است:

الف) استدلال استفاده از Biword برای عبارات متداول:

کاهش سربار پردازشی (CPU Optimization): اگر عبارتی مانند "**United States**" روزانه میلیون‌ها بار جستجو شود و ما فقط نمایه موقعیتی داشته باشیم، پردازنده موتور جستجو باید میلیون‌ها بار آرایه موقعیت‌های کلمه **United** و **States** را واکشی کرده و تفاضل آن‌ها را محاسبه کند. این کار هدررفتِ شدیدِ توان پردازشی است. با ذخیره این عبارت به عنوان 1 بی‌ورد، زمان پاسخگویی برای این پرس‌وجوی سنگین از **O(N)** به **O(1)** تقلیل می‌یابد.

جلوگیری از انفجار لغتنامه (Memory Optimization): همان‌طور که در مسائل پیشین ثابت شد، اگر بخواهیم تمام ترکیبات دوکلمه‌ای موجود در کل اسناد را در نمایه **Biword** ذخیره کنیم، با پدیده انفجار لغتنامه (**Vocabulary Explosion**) روبرو می‌شویم و حجم نمایه به قدری متورم می‌شود که در حافظه اصلی (**RAM**) سرورها جا نمی‌گیرد. با اتکا به نمایه موقعیتی برای عبارات غیرمتداول، سیستم با اشغال فضای بسیار کمتری، توانایی پاسخگویی به هر ترکیب غیرقابل‌پیش‌بینی را حفظ می‌کند.

بخش 4) استراتژی انتخاب عبارات (Selection Criteria)

1 پرسش اساسی در این معماری این است: سیستم چگونه می‌فهمد کدام عبارات باید در سطح اول (**Biword**) ذخیره شوند؟ در موتورهای جستجوی صنعتی، این انتخاب به صورت پویا (**Dynamic**) و بر اساس معیارهای زیر انجام می‌شود:

1. تحلیل لاگ جستجوها (Query Log Analysis): سیستم تاریخچه جستجوی کاربران را استخراج کرده و **N-gram**هایی را که بالاترین فرکانس جستجو را دارند (**High-frequency Queries**) در نمایه **Biword** قرار می‌دهد.

2. شناسایی موجودیت‌های نام‌دار (NER): با استفاده از مدل‌های زبانی پردازش زبان طبیعی (**NLP**)، اسامی اشخاص معروف، مکان‌های جغرافیایی، برندهای تجاری و اصطلاحات علمی رایج پیش از نمایه‌سازی شناسایی شده و مستقیماً به عنوان بی‌ورد ثبت می‌شوند.

بخش 5) تحلیل کارایی و جمع‌بندی

معماری نمایه‌های چندسطحی، تجلی یکی از مهم‌ترین اصول مهندسی نرم‌افزار است: «**بهینه‌سازی برای مسیرهای رایج**» (**Optimize the Common Case**).

با ترکیب روش‌ها، موتور جستجو موفق می‌شود بدون پرداخت هزینه سنگین حافظه برای کل مجموعه (که نقطه ضعف **Biword** است) و بدون تحمیل بار پردازشی بالا برای جستجوهای پرتکرار (که نقطه ضعف **Positional** است)، سیستمی مقیاس‌پذیر (**Scalable**) با بالاترین سرعت و دقت ممکن طراحی کند. این الگو، معماری پایه در موتورهای جستجوی قدرتمندی نظیر **Elasticsearch** و **Solr** است.

پاسخ‌نامه مسئله 2.24: الگوریتم تقاطع موقعیتی (Positional Intersection Algorithm)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در سیستم‌های بازیابی اطلاعات، پشتیبانی از جستجوی عبارتی (**Phrase Query**) نیازمند درک دقیق ماشین از ترتیب و توالی کلمات است. تفاوت معنایی میان عبارت "**Search Engine**" (موتور جستجو) و "**Engine Search**" (جستجوی موتور) تنها در جابجایی کلمات نهفته است. الگوریتم تقاطع موقعیتی، با استفاده از آرایه موقعیت‌ها در نمایه معکوس (**Positional Index**)، این تفاوت توالی را از طریق محاسبات ساده تفاضلی (**Mathematical Subtraction**) و شرط تقدم شناسایی می‌کند.

بخش 2) منطق ریاضی و شرط تقدم موقعیتی

هنگامی که الگوریتم تقاطع، **2** آرایه موقعیت را برای **1** سند مشترک در حافظه بارگذاری می‌کند، صرفاً به دنبال وجود کلمات نیست، بلکه **1** «شرط فاصله جهت‌دار» (**Directional Distance Condition**) را ارزیابی می‌کند.

فرض کنید **P₁** نمایانگر آرایه موقعیت‌های کلمه **Search** و **P₂** نمایانگر آرایه موقعیت‌های کلمه **Engine** در یک سند مشخص باشد.

تشخیص عبارت "Search Engine": الگوریتم به دنبال زوج موقعیت‌هایی به فرم **(p₁ ∈ P₁ , p₂ ∈ P₂)** می‌گردد که شرط زیر در آن‌ها صادق باشد:

$$p_2 - p_1 = 1$$

این معادله از نظر ریاضی ثابت می‌کند که کلمه **Engine** دقیقاً **1** جایگاه بعد از کلمه **Search** قرار گرفته است.

تشخیص عبارت "Engine Search": الگوریتم شرط تفاضلی را تغییر داده و به دنبال زوج‌هایی می‌گردد که معادله زیر را ارضا کنند:

$$p_1 - p_2 = 1$$

در این حالت، کلمه **Search** باید دقیقاً **1** جایگاه پس از کلمه **Engine** ظاهر شود.

بخش 3) بررسی یک سناریوی عملی

برای درک ملموس‌تر، سندی با متن انگلیسی زیر را در نظر بگیرید:

"To build a **Search Engine**, one must avoid a blind **Engine Search** algorithm."

پس از فاز توکن‌گذاری و اختصاص شماره موقعیت به تکتک کلمات، جایگاه کلمات کلیدی در این سند به شکل زیر ذخیره می‌شود:

آرایه موقعیت‌های کلمه Search: برابر است با **P₁ = [4, 11]**

آرایه موقعیت‌های کلمه Engine: برابر است با **P₂ = [5, 10]**

رفتار ماشین هنگام جستجوی "Search Engine":

موقعیت‌گیری: اشاره‌گر اول روی عدد **4** (از **P₁**) و اشاره‌گر دوم روی عدد **5** (از **P₂**) قرار می‌گیرد.

ارزیابی شرط: سیستم تفاضل را محاسبه می‌کند: **5 - 4 = 1**. از آنجا که شرط برقرار است، سیستم این زوج را به عنوان کشف عبارت **"Search Engine"** ثبت می‌کند.

رفتار ماشین هنگام جستجوی "Engine Search":

ارزیابی معکوس: الگوریتم اکنون بررسی می‌کند که آیا مقداری در **P₁** وجود دارد که دقیقاً **1** واحد از مقداری در **P₂** بزرگتر باشد؟

موقعیت‌گیری: اشاره‌گرها روی اعداد **11** (از **P₁**) و **10** (از **P₂**) قرار می‌گیرند.

ارزیابی شرط: سیستم تفاضل را محاسبه می‌کند: **11 - 10 = 1**. شرط برقرار است و ماشین این بخش از متن را به درستی به عنوان **"Engine Search"** تشخیص می‌دهد.

بخش 4) پیاده‌سازی هسته الگوریتم

بر اساس استانداردهای مهندسی و توافقات پیشین، تمامی متغیرها و مستندات داخل کد به زبان انگلیسی نوشته شده‌اند تا از تداخل کدگذاری جلوگیری شود. الگوریتم زیر نحوه پیمایش **2** آرایه موقعیت را با پیچیدگی زمانی خطی نشان می‌دهد:

```
positional_intersect.py

def positional_intersect(p1_list, p2_list, target_difference):
    match_results = []
    ptr1 = 0
    ptr2 = 0

    while ptr1 < len(p1_list) and ptr2 < len(p2_list):
        pos1 = p1_list[ptr1]
        pos2 = p2_list[ptr2]

        current_diff = pos2 - pos1

        if current_diff == target_difference:
            match_results.append((pos1, pos2))
            ptr1 += 1
            ptr2 += 1

        elif pos2 > pos1 + target_difference:
            ptr1 += 1

        else:
            ptr2 += 1

    return match_results

# Usage example for "Search Engine"
# expected target_difference is 1

# Usage example for "Engine Search"
# expected target_difference is -1
```

بخش 5) نکات تکمیلی و جمع‌بندی

الگوریتم تقاطع موقعیتی مبتنی بر **2** اشاره‌گر (**Two-pointer Algorithm**)، دارای پیچیدگی زمانی از مرتبه $O(|P_1| + |P_2|)$ است. این معماری پیشرفته به موتور جستجو اجازه می‌دهد تا بدون نیاز به تولید و ذخیره‌سازی توکن‌های ترکیبی جدید (مانند **Biword** ها)، صرفاً با تکیه بر عملیات ریاضی تفریق، هرگونه ترکیب متوالی و جهت‌دار از کلمات را در زمان اجرای پرس‌وجو (**Query Time**) شناسایی کند. این حساسیت بالا به «جهت» (**Directional Sensitivity**)، موتور جستجو را در برابر خطای مثبت کاذب (**False Positives**) در مواجهه با عباراتی که کلمات یکسان اما معانی متضاد دارند، کاملاً مصون می‌دارد.

پاسخ‌نامه مسئله 2.25: تقاطع پویا با درخت‌های متوازن

(Dynamic Intersection with Balanced Trees)

بخش 1) تحلیل صورت مسئله و چالش‌های معماری ایستا

در موتورهای جستجوی سنتی (**Static**)، لیست‌های پستینگ تنها **1** بار ساخته شده و در دیسک ذخیره می‌شوند. در این حالت، فواصل نشانگرهای پرشی (**Skip Pointers**) معمولاً با فرمول بهینه $\sqrt{L}$ تعیین می‌گردند. اما در **1** موتور جستجوی بلادرنگ (مانند توییت‌ر یا سیستم‌های خبری) که در هر ثانیه هزاران سند جدید (داکیومنت) نمایه‌سازی یا حذف می‌شوند، این فواصل ثابت به سرعت نامتوازن شده و کارایی خود را از دست می‌دهند.

ساخت مجدد و آفلاین این نشانگرها نیز به دلیل سربار محاسباتی بالا (**Re-indexing Overhead**) در سیستم‌های بلادرنگ توجیه‌پذیر نیست. هدف از این مسئله، طراحی **1** ساختار داده پویا (**Dynamic Data Structure**) است که بتواند با کمترین

بخش 2) معماری پیشنهادی: درخت AVL نخ‌کشی‌شده (Threaded AVL Tree)

برای حل این چالش، ما لیست پیوندی ساده را با 1 ساختار «درخت جستجوی دودویی متوازن نخ‌کشی‌شده» جایگزین می‌کنیم. در این معماری ترکیبی، هر لیست پستینگ در واقع 1 درخت AVL است که دارای 2 ویژگی کلیدی است:

1. **خاصیت درختی (Dynamic Skips):** گره‌های میانی (Internal Nodes) و ریشه درخت، نقش نشانگرهای پرشی کاملاً متوازن را ایفا می‌کنند. با هر درج یا حذف (Insertion/Deletion)، الگوریتم AVL با انجام چرخش‌های استاندارد (Rotations)، ارتفاع درخت را به صورت پویا متوازن نگه می‌دارد.

2. **خاصیت نخ‌کشی (In-order Threads):** برای حفظ قابلیت پیمایش خطی (که در لیست‌های پستینگ حیاتی است)، هر گره در درخت علاوه بر اشاره‌گرهای left و right، 1 اشاره‌گر به نام next_inorder دارد که مستقیماً به گره بعدی در پیمایش میان‌ترب (In-order Successor) اشاره می‌کند. این اشاره‌گرها، در پایین‌ترین سطح منطقی، همان لیست پیوندی مرتب‌شده را تشکیل می‌دهند.

بخش 3) الگوریتم تقاطع پویا (Dynamic Intersection Algorithm)

هنگام تقاطع 2 لیست پستینگ مبتنی بر AVL به طول‌های M و N (با فرض $M \leq N$)، سیستم موتور جستجو بر اساس اندازه 2 لیست، به صورت تطبیقی (Adaptive) تصمیم‌گیری می‌کند.

استراتژی اول - رویکرد متقارن (زمانی که $M \approx N$): اگر طول 2 لیست به هم نزدیک باشد، پرش‌های بزرگ کمکی به سرعت الگوریتم نمی‌کنند. در این حالت، سیستم ریشه درخت را نادیده گرفته، به کوچکترین گره (سمت چپ‌ترین برگ) در هر 2 درخت می‌رود و با استفاده از اشاره‌گرهای next_inorder، 1 پیمایش خطی و اشتراک‌گیری کلاسیک (Linear Merge) انجام می‌دهد.

استراتژی دوم - جستجوی دودویی متقارن (زمانی که $M \ll N$): اگر طول 1 لیست بسیار کوچکتر از دیگری باشد (مثلاً تقاطع 1 کلمه نادر با 1 کلمه بسیار رایج)، سیستم لیست کوچکتر (M) را به صورت خطی با استفاده از next_inorder پیمایش می‌کند. سپس برای هر شناسه سند (DocID) در لیست M، 1 جستجوی دودویی (Binary Search) را از ریشه درخت N آغاز می‌کند. از آنجا که درخت N یک AVL متوازن است، هر جستجو از طریق گره‌های پرشی با بالاترین سرعت انجام می‌پذیرد.

بخش 4) تحلیل دقیق پیچیدگی زمانی (Big O Analysis)

عملکرد این معماری در فاز اشتراک‌گیری، 1 توازن ریاضیاتی و الگوریتمی بسیار کارآمد را به نمایش می‌گذارد. سیستم هزینه هر 2 استراتژی را در زمان اجرا (Runtime) محاسبه کرده و مسیر بهینه را انتخاب می‌کند:

پیچیدگی فاز بروزرسانی (Insertion/Deletion): درج یا حذف هر شناسه سند، به دلیل خاصیت خودمتوازن‌کنندگی درخت AVL، در زمان اکیداً لگاریتمی یعنی $O(\log L)$ انجام می‌شود (که L طول آن لیست پستینگ است).

پیچیدگی استراتژی اول (Linear Merge): پیمایش خطی هر 2 درخت با استفاده از رشته‌های نخ‌کشی‌شده (Threads)، دقیقاً معادل تقاطع 2 آرایه مرتب‌شده است. بنابراین پیچیدگی زمانی برابر است با:

$$O(M + N)$$

پیچیدگی استراتژی دوم (Logarithmic Skip Search): سیستم برای تمامی **M** عضو موجود در لیست کوچکتر، **1** جستجوی دودویی در درخت **N** انجام می‌دهد. از آنجا که ارتفاع درخت **AVL** دقیقاً متعادل و متناسب با **log N** است، پیچیدگی زمانی برابر است با:

$$O(M \log N)$$

مرز تصمیم‌گیری (Decision Boundary): هسته موتور جستجو از رابطه $\min(O(M + N), O(M \log N))$ پیروی می‌کند. به لحاظ ریاضی، سیستم تا زمانی از استراتژی پرشی (درخت **AVL**) استفاده می‌کند که معادله زیر برقرار باشد:

$$M \log N < M + N$$

این تحلیل ثابت می‌کند که معماری **Threaded AVL Tree** در سیستم‌های بلادرنگ، نه تنها چالش به‌هم‌ریختگی نشانگرهای پرشی را حل می‌کند، بلکه بدون نیاز به بازسازی آفلاین نمایه (**Re-indexing**)، همیشه بهینه‌ترین پیچیدگی زمانی ممکن را برای پرس‌وجوها تضمین می‌نماید.

بخش 5) پیاده‌سازی عملیاتی

قطعه کد زیر، هسته تصمیم‌گیرنده و منطق تقاطع را در این معماری نوین نشان می‌دهد. طبق استانداردهای مهندسی نرم‌افزار، تمامی نام‌گذاری‌ها و ساختارها به صورت بهینه‌سازی‌شده تدوین شده‌اند.

dynamic_avl_intersection.py

```
import math

class ThreadedAVLNode:
    def __init__(self, doc_id):
        self.doc_id = doc_id
        self.left = None
        self.right = None
        self.height = 1
        self.next_inorder = None # Base linked list thread

def get_minimum_node(root):
    current = root

    while current and current.left:
        current = current.left

    return current

def search_in_avl(root, target_id):
    current = root

    while current:
        if target_id == current.doc_id:
            return True
        elif target_id < current.doc_id:
            current = current.left
        else:
            current = current.right

    return False

def adaptive_dynamic_intersect(root_m, size_m, root_n, size_n):
    # Ensure M is always the smaller list
    if size_m > size_n:
        root_m, root_n = root_n, root_m
        size_m, size_n = size_n, size_m

    intersection_result = []

    # Cost evaluation decision boundary
    cost_linear = size_m + size_n
```

```
cost_logarithmic = size_m * math.log2(size_n) if size_n > 0 else 0

# Strategy 2: O(M log N) - Logarithmic Skip Search
if cost_logarithmic < cost_linear:
    current_m = get_minimum_node(root_m)

    while current_m is not None:
        if search_in_avl(root_n, current_m.doc_id):
            intersection_result.append(current_m.doc_id)

        current_m = current_m.next_inorder

# Strategy 1: O(M + N) - Linear Merge using threads
else:
    current_m = get_minimum_node(root_m)
    current_n = get_minimum_node(root_n)

    while current_m is not None and current_n is not None:
        if current_m.doc_id == current_n.doc_id:
            intersection_result.append(current_m.doc_id)
            current_m = current_m.next_inorder
            current_n = current_n.next_inorder

        elif current_m.doc_id < current_n.doc_id:
            current_m = current_m.next_inorder

        else:
            current_n = current_n.next_inorder

    return intersection_result
```

پاسخنامه مسئله 2.26: ماشین حالت توکن ساز و اعتبارسنجی (Tokenizer State Machine and Validation)

بخش 1) تحلیل صورت مسئله و هدف گذاری

در معماری موتورهای جستجو، فاز توکن سازی (**Tokenization**) و جداسازی اسناد (**Document Delineation**) اولین خط پردازش بر روی جریان عظیمی از لاگ ها و متون خام محسوب می شود. در این فاز، استخراج شناسه های معتبر (**Identifiers**) معمولاً از طریق یک ماشین حالت قطعی (**DFA**) انجام می پذیرد که بر اساس عبارات باقاعده (**Regular Expressions**) کامپایل شده است.

هدف از این مسئله، طراحی یک **DFA** بهینه برای استخراج شناسه های متنی است که ضمن رعایت دقیق قوانین نظریه اتوماتا (تعیین **q1** به عنوان حالت نهایی)، نشان دهد چگونه ساده سازی عبارات باقاعده پیش از تبدیل به ماشین حالت، می تواند سربار پردازشی را در مقیاس کلان کاهش دهد.

بخش 2) تحلیل نظریه اتوماتا و بهینه سازی

قاعده ریاضی زیر یکی از قوانین بنیادین در جبر کلین (**Kleene Algebra**) است:

$$(R_1^*)^* \equiv R_1^*$$

حالت غیربهینه: اگر یک کامپایلر عبارات باقاعده (**Regex Engine**) بخواهد بخش چپ این تساوی را مستقیماً به ماشین حالت تبدیل کند، ابتدا یک حلقه تکرار برای **R1*** می سازد و سپس یک مکانیزم تکرار خارجی ایجاد می کند. این کار منجر به تولید یک ماشین حالت غیرقطعی (**NFA**) پیچیده می شود که تبدیل آن به **DFA** باعث انفجار حالت ها (**State Explosion**) یا ایجاد حلقه های پردازشی زائد می گردد.

حالت بهینه‌شده: با اعمال این اتحاد ریاضی، تکرارهای تودرتو حذف شده و کل ساختار صرفاً به یک حلقه ساده (Self-loop) بر روی یک حالت واحد تقلیل می‌یابد.

در تعریف یک شناسه معتبر استاندارد، معمولاً کاراکتر اول باید یک حرف باشد و کاراکترهای بعدی می‌توانند ترکیبی از حروف و اعداد به هر تعداد باشند. اگر R_1 برابر با ترکیب حروف و اعداد باشد، عبارت باقاعده شناسه به فرم $Letter \cdot R_1^*$ خواهد بود. استفاده از قانون فوق تضمین می‌کند که هیچ حلقه تکرار مضاعفی در توصیف R_1 لحاظ نگردد.

بخش 3) طراحی ماشین حالت قطعی (DFA) با شرط پذیرش در q_1

برای پیاده‌سازی این منطق بهینه‌شده، یک ماشین حالت قطعی با 3 حالت طراحی می‌کنیم. در این طراحی، انتقال‌ها به گونه‌ای مهندسی شده‌اند که اعتبارسنجی توکن لزوماً در **حالت نهایی q_1** اتفاق بیفتد.

توصیف حالت‌ها:

حالت q_0 (حالت اولیه): ماشین در این حالت منتظر دریافت اولین کاراکتر است.

حالت q_1 (حالت نهایی و پذیرش): ماشینی که به این حالت برسد، یک شناسه معتبر را تشخیص داده است و پردازش تکرارها دقیقاً روی همین حالت انجام می‌شود.

حالت q_{err} (حالت خطا): در صورت دریافت هرگونه کاراکتر غیرمجاز، ماشین به این حالت سقوط کرده و در آن حبس می‌شود.

توصیف انتقال‌ها (Transitions):

انتقال‌های خروجی از q_0 : با دریافت یک حرف به q_1 منتقل می‌شود. با دریافت عدد یا نماد غیرمجاز به q_{err} منتقل می‌شود.

انتقال‌های خروجی از q_1 : با دریافت حرف یا عدد (R_1) در قالب یک حلقه روی خودش (q_1) باقی می‌ماند. با دریافت نماد غیرمجاز به q_{err} منتقل می‌شود.

انتقال‌های خروجی از q_{err} : با دریافت هر کاراکتری در قالب یک حلقه روی خودش (q_{err}) باقی می‌ماند.

بخش 4) تأثیر بهینه‌سازی بر سربار پردازنده

فاز تفکیک اسناد نیازمند پردازش جریانی از میلیون‌ها سند و میلیاردها کاراکتر است. اعمال بهینه‌سازی ریاضی روی DFA مزایای سخت‌افزاری زیر را در سطح پردازنده (CPU) به همراه دارد:

1. کاهش چرخه ساعت (Clock Cycles): در مدل غیربهینه، پردازنده برای مدیریت ستاره‌های تودرتو مجبور به انجام پرش‌های حافظه‌ای میان حالت‌های واسط است. با تقلیل اتوماتا به یک حلقه ساده در q_1 ، خواندن هر کاراکتر از شناسه، تنها به یک دستور شرطی ساده نیاز دارد که زمان پردازش هر کاراکتر را به **O(1)** مطلق کاهش می‌دهد.

2. بهبود نرخ اصابت حافظه نهان (Cache Hit Rate): جدول انتقال حالت برای یک DFA بهینه‌شده با 3 حالت، به قدری کوچک است که به طور کامل در سریع‌ترین سطح حافظه نهان پردازنده (**L1 Cache**) قرار می‌گیرد.

3. جلوگیری از عقب‌گرد (Backtracking): عدم وجود انتقال‌های مبهم، نیاز موتور پردازش متن به بازگشت و بررسی مجدد مسیرها را در هنگام مواجهه با رشته‌های طولانی از بین برده و مصرف پردازنده را خطی نگه می‌دارد.

بخش 5) پیاده‌سازی عملیاتی اتوماتا

قطعه کد زیر پیاده‌سازی ساختار این **DFA** بهینه‌شده را با رعایت دقیق نام‌گذاری‌ها، مستندات یکپارچه و بدون استفاده از کاراکترهای اضافی نشان می‌دهد. در این ساختار، پذیرش صرفاً منوط به توقف در وضعیت **q1** است.

```
def dfa_tokenizer_validator(input_string):
    # Initialize states and variables
    state_q0 = 'q0'
    state_q1 = 'q1'
    state_err = 'q_err'

    current_state = state_q0

    if not input_string:
        return False

    for char in input_string:
        # Processing from Initial State (q0)
        if current_state == state_q0:
            if char.isalpha():
                current_state = state_q1
            else:
                current_state = state_err

        # Processing from Final/Accepting State (q1)
        # Implementing optimized R1* loop on q1
        elif current_state == state_q1:
            if char.isalnum():
                current_state = state_q1
            else:
                current_state = state_err

        # Trap State handling
        elif current_state == state_err:
            break

    # Final validation strict check for q1
    if current_state == state_q1:
        return True

    return False

# Test cases implementation
test_token_1 = "ValidToken123"
test_token_2 = "123Invalid"
test_token_3 = "Invalid@Token"

print(f"Token: {test_token_1} -> Accepted: {dfa_tokenizer_validator(test_token_1)}")
print(f"Token: {test_token_2} -> Accepted: {dfa_tokenizer_validator(test_token_2)}")
print(f"Token: {test_token_3} -> Accepted: {dfa_tokenizer_validator(test_token_3)}")
```

پاسخ‌نامه مسئله 2.27: اتوگریدر و تشخیص تقلب در کدهای منبع

(Autograder and Plagiarism Detection)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در محیط‌های آموزشی و پروژه‌های برنامه‌نویسی، رایج‌ترین نوع تقلب (کپی‌برداری نوع دوم یا **Type II Clones**)، کپی کردن منطق و ساختار کد، اما تغییر دادن اسامی متغیرها، توابع و مقادیر ثابت است تا سامانه و استاد را فریب دهند. در این سناریو، ما سورس‌کدها را به عنوان متون خام (**Documents**) در نظر می‌گیریم.

هدف از این مسئله، طراحی یک خط لوله پردازشی (**Pipeline**) مبتنی بر بازیابی اطلاعات است که از «نمایه موقعیتی» (**Positional Index**) برای کشف شباهت‌های ساختاری استفاده کند. کلید موفقیت این سیستم، فاز نرمال‌سازی (**Normalization**) است که باید تفاوت‌های ظاهری را پیش از ورود کد به موتور جستجو خنثی نماید.

بخش 2) استراتژی نرمال‌سازی: تجرید لغوی (Lexical Abstraction)

برای جلوگیری از دور زدن سیستم با تغییر نام متغیرها، استراتژی نرمال‌سازی باید کد را از یک «متن وابسته به کاربر» به یک «اسکلت منطقی انتزاعی» تبدیل کند. این کار در **3 گام** اساسی انجام می‌شود:

گام اول - حذف نویزها (Noise Removal): در ابتدا تمامی کامنت‌ها (توضیحات کد)، فاصله‌های خالی (**Whitespaces**) و تورفتگی‌ها (**Indentations**) به طور کامل حذف می‌شوند. این موارد هیچ تأثیری در منطق اجرایی ندارند و معمولاً توسط دانشجویان برای متفاوت جلوه دادن کد دستکاری می‌شوند.

گام دوم - حفظ کلمات کلیدی و عملگرها (Keyword Preservation): کلمات کلیدی زبان برنامه‌نویسی (مانند **while**، **if**، **for**، **return**) و عملگرهای منطقی و ریاضی (مانند **==**، **+**، **{}**) دقیقاً به همان شکل اصلی حفظ می‌شوند؛ زیرا این کلمات دی‌ان‌ای (**DNA**) ساختاری کد را تشکیل می‌دهند.

گام سوم - نقاب‌گذاری متغیرها و مقادیر (Variable & Literal Masking): تمامی شناسه‌هایی که توسط دانشجو تعریف شده‌اند (نام متغیرها، نام کلاس‌ها و توابع) با یک توکن استاندارد و واحد مانند **[VAR]** جایگزین می‌شوند. همچنین تمامی مقادیر ثابت (رشته‌ها و اعداد) با توکن **[VAL]** جایگزین می‌گردند.

مثال عملی نرمال‌سازی:

کد اصلی دانشجو:

```
if (user_age == 20) { print(user_age); }
```

تبدیل می‌شود به توکن‌های:

```
if, (, [VAR], ==, [VAL], ), {, [VAR], (, [VAR], ), , ; }
```

با این استراتژی، اگر دانشجوی دیگری کد را به **if (x == 50) { show(x); }** تغییر دهد، خروجی نرمال‌ساز دقیقاً همان توالی انتزاعی بالا خواهد بود و سیستم آن‌ها را به عنوان یک نسخه یکسان ارزیابی می‌کند.

بخش 3) طراحی پایپ‌لاین بازیابی اطلاعات مبتنی بر نمایه موقعیتی

پس از نرمال‌سازی کدهای **100** دانشجو، خط لوله سیستم تشخیص تقلب با استفاده از معماری موتور جستجو به شکل زیر عمل می‌کند:

مرحله اول (Indexing): توکن‌های انتزاعی تولید شده، وارد یک نمایه موقعیتی (**Positional Index**) می‌شوند. لغت‌نامه (**Vocabulary**) این نمایه بسیار کوچک است (فقط شامل کلمات کلیدی زبان، **[VAR]**، **[VAL]** و عملگرهاست)، اما لیست‌های پستینگ آن حاوی شماره دانشجویی (به عنوان **DocID**) و آرایه‌ای از موقعیت‌های دقیق (**Positions**) هر توکن در کد آن‌ها است.

مرحله دوم (Query Generation / Shingling): سیستم به صورت خودکار، کد هر دانشجو را به پنجره‌های لغزان (**Sliding Windows**) با طول مشخص (مثلاً **15** توکن متوالی) تقسیم می‌کند. هر پنجره به عنوان یک پرس‌وجوی عبارتی (**Phrase Query**) به موتور جستجو ارسال می‌شود.

مرحله سوم (Positional Intersection): موتور جستجو برای هر پرس‌وجوی **15** توکنی، با استفاده از الگوریتم تقاطع موقعیتی (با بررسی تفاضل $P_i - P_{i-1} = 1$)، پایگاه داده را جستجو می‌کند.

مرحله چهارم (Scoring): اگر سیستم تشخیص دهد که بین کد دانشجوی **A** و دانشجوی **B**، مثلاً **50** توکن انتزاعی دقیقاً با همان توالی و مجاورت موقعیتی تکرار شده است، یک «امتیاز شباهت» (**Similarity Score**) بالا برای این جفت‌کد ثبت می‌کند

که نشان‌دهنده کپی‌برداری ساختاری است.

بخش 4) پیاده‌سازی عملیاتی نرمال‌ساز

قطعه کد زیر، منطق فاز نرمال‌سازی را به صورت مهندسی‌شده نشان می‌دهد:

```
ast_normalizer.py

import re
import keyword

def normalize_source_code(raw_code):
    # Removing single-line comments to ignore superficial text differences
    code_cleaned = re.sub(r'#.*/', '', raw_code)

    # Tokenizing based on words, numbers, and structural operators
    token_list = re.findall(r'[a-zA-Z_]\w*|\d+|[\^\w\s]', code_cleaned)

    normalized_ast_tokens = []

    for tok in token_list:
        if tok in keyword.kwlist:
            # Preserve language keywords defining logic structure
            normalized_ast_tokens.append(tok)

        elif re.match(r'^[a-zA-Z_]\w*$', tok):
            # Mask user-defined variables and function identifiers
            normalized_ast_tokens.append('VAR')

        elif re.match(r'^\d+$', tok):
            # Mask hardcoded literal values and numeric constants
            normalized_ast_tokens.append('VAL')

        else:
            # Preserve operators, brackets, and punctuation
            normalized_ast_tokens.append(tok)

    return normalized_ast_tokens

# Test execution
student_code = "for counter in range(10): total_sum += 1"
print(normalize_source_code(student_code))
# Output: ['for', 'VAR', 'in', 'VAR', '(', 'VAL', ')', ':', 'VAR', '+', '=', 'VAL']
```

بخش 5) تحلیل کارایی مهندسی (Performance Analysis)

استفاده از معماری موتور جستجو **(Positional Index)** به جای مقایسه خطبه‌خط **(String Matching)** 2 مزیت عظیم برای اتوگریدرها دارد:

1. **سرعت پردازش (Scalability):** مقایسه دوجه‌دوی **(Pairwise)** جامعه 100 نفره پروژه‌ها نیازمند 4950 عملیات مقایسه سنگین است:

$$\frac{100 \times 99}{2} = 4950$$

اما با استفاده از نمایه معکوس موقعیتی، سیستم هر قطعه از کد را تنها با پیچیدگی زمانی متناسب با طول لیست پستینگ در یک پایگاه داده متمرکز جستجو می‌کند که سرعت کشف تقلب را به صورت نمایی افزایش می‌دهد.

2. **مقاومت در برابر جابجایی (Resilience to Reordering):** اگر دانشجویی جای 2 تابع کاملاً مستقل را در فایل جابجا کند، الگوریتم‌های سنتی مقایسه متن (مانند **Diff**) دچار خطای محاسباتی می‌شوند؛ اما از آنجا که نمایه موقعیتی بر اساس تقاطع بلوک‌های کوچک‌تر **(Shingles)** کار می‌کند، توالی‌های مشابه را فارغ از اینکه در کجای فایل (ابتدا یا انتهای کد) قرار گرفته باشند، با بالاترین سطح دقت شناسایی و گزارش می‌نماید.

پاسخ‌نامه مسئله 2.28: پیاده‌سازی تقاطع پوزیشنی بهینه (Optimized Positional Intersection)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

برای پاسخ‌گویی به پرس‌وجوهای عبارتی دقیق (Exact Phrase Queries) مانند "algorithm design"، یافتن اشتراک اسناد (DocIDs) به تنهایی کفایت نمی‌کند. سیستم بازیابی اطلاعات باید تضمین کند که در اسناد مشترک، کلمه "design" دقیقاً 1 جایگاه بعد از کلمه "algorithm" قرار گرفته است.

هدف از این مسئله، پیاده‌سازی 1 الگوریتم تقاطع دوگانه (Dual-Intersection) است که با استفاده از تکنیک 2 اشاره‌گر (Two-Pointer Technique)، ابتدا اسناد مشترک را یافته و سپس درون آن اسناد، فواصل موقعیتی کلمات را بررسی کند. این کد باید بهینه‌سازی شده و آماده برای استقرار در هسته موتور جستجو باشد.

بخش 2) تشریح گام‌به‌گام: منطق الگوریتم

ساختار الگوریتم از 2 حلقه تو در تو تشکیل می‌شود که هر 2 به صورت خطی عمل می‌کنند:

حلقه خارجی (پیمایش در سطح سند): الگوریتم دارای 2 اشاره‌گر است که روی لیست‌های پستینگ 2 کلمه حرکت می‌کنند. اگر شناسه سند در لیست اول با لیست دوم برابر باشد، الگوریتم برای بررسی دقیق‌تر وارد حلقه داخلی می‌شود. در غیر این صورت، اشاره‌گری که به شناسه سند کوچکتر اشاره می‌کند، 1 گام به جلو می‌رود تا شانس رسیدن به سند مشترک در گام‌های بعدی حفظ شود.

حلقه داخلی (پیمایش در سطح موقعیت): پس از یافتن سند مشترک، سیستم آرایه موقعیت‌های هر 2 کلمه را در آن سند استخراج می‌کند. 2 اشاره‌گر جدید، روی آرایه موقعیت‌ها حرکت کرده و شرط موفقیت را ارزیابی می‌کنند: $Pos_2 - Pos_1 = 1$. اگر این شرط ارضا شود، شناسه سند به لیست نهایی اضافه می‌گردد و سیستم از حلقه داخلی خارج می‌شود (زیرا یافتن حتی 1 مورد تطابق برای بازیابی سند کافی است). اگر $Pos_2 > Pos_1 + 1$ باشد، اشاره‌گر موقعیت کلمه اول را جلو می‌بریم، در غیر این صورت، اشاره‌گر کلمه دوم را جلو می‌بریم.

بخش 3) پیاده‌سازی عملیاتی (کدنویسی)

بر اساس اصول مهندسی نرم‌افزار برای کدنویسی تمیز (Clean Code) و طبق استانداردهای توسعه، کدهای زیر کاملاً به زبان انگلیسی پیاده‌سازی شده‌اند تا از هرگونه تداخل در پردازش جلوگیری شود:

```
optimized_positional_intersect.py

def optimized_positional_intersect(postings_word1, postings_word2):
    result_doc_ids = []

    ptr1 = 0
    ptr2 = 0

    len1 = len(postings_word1)
    len2 = len(postings_word2)

    while ptr1 < len1 and ptr2 < len2:
        doc_id1 = postings_word1[ptr1]['doc_id']
        doc_id2 = postings_word2[ptr2]['doc_id']

        if doc_id1 == doc_id2:
            pos_list1 = postings_word1[ptr1]['positions']
            pos_list2 = postings_word2[ptr2]['positions']

            pos_ptr1 = 0
            pos_ptr2 = 0

            match_found = False

            while pos_ptr1 < len(pos_list1) and pos_ptr2 < len(pos_list2):
```

```
pos1 = pos_list1[pos_ptr1]
pos2 = pos_list2[pos_ptr2]

if pos2 - pos1 == 1:
    match_found = True
    break

elif pos2 > pos1 + 1:
    pos_ptr1 += 1

else:
    pos_ptr2 += 1

if match_found:
    result_doc_ids.append(doc_id1)

ptr1 += 1
ptr2 += 1

elif doc_id1 < doc_id2:
    ptr1 += 1

else:
    ptr2 += 1

return result_doc_ids
```

بخش 4) تحلیل کارایی و پیچیدگی زمانی (Complexity Analysis)

این پیاده‌سازی از نظر تخصیص منابع محاسباتی به شدت بهینه است:

پیچیدگی زمانی (Time Complexity): پیمایش حلقه خارجی متناسب با طول لیست پستینگ اسناد است. اگر تعداد اسناد کلمه اول **N** و کلمه دوم **M** باشد، پیچیدگی پایه **O(N + M)** است. حلقه داخلی تنها زمانی اجرا می‌شود که سند مشترکی یافت شود. در آن صورت، پیمایش آرایه موقعیت‌ها به طول **P₁** و **P₂** نیازمند زمان **O(P₁ + P₂)** است. از آنجا که به محض یافتن اولین تطابق، الگوریتم حلقه داخلی را متوقف می‌کند (فرمان **break**)، نیازی به پردازش بیهوده تمام موقعیت‌ها نیست. بنابراین در بدترین حالت، پیچیدگی کل به شکل خطی باقی می‌ماند.

پیچیدگی مکانی (Space Complexity): الگوریتم بدون ایجاد کپی از داده‌های ورودی، تنها از **4** اشاره‌گر ساده به عنوان متغیر حالت استفاده می‌کند و حافظه مصرفی اضافی آن تنها محدود به آرایه خروجی است. بنابراین پیچیدگی مکانی، به جز خروجی، معادل **O(1)** می‌باشد.

پاسخ‌نامه مسئله 2.29: ساختمان داده لیست پرشی سفارشی

(Custom Skip List Data Structure)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در موتورهای جستجو، کلمات پرکاربرد (**High-frequency terms**) به دلیل داشتن لیست‌های پستینگ (**Posting Lists**) بسیار طولانی، گلوگاه اصلی در فاز اشتراک‌گیری (**Intersection**) محسوب می‌شوند. پیمایش خطی این لیست‌ها نیازمند زمان **O(N)** است که برای **1** سیستم بلادرنگ قابل قبول نیست.

هدف از این مسئله، طراحی و پیاده‌سازی شیء‌گرا (**Object-Oriented**) **1** ساختمان داده اختصاصی است که نشانگرهای پرشی (**Skip Pointers**) را بر اساس قاعده ریاضی **[vP]** (که در آن **P** طول لیست است) به صورت خودکار تولید کند. این ساختار به ماشین اجازه می‌دهد در زمان جستجو، از روی بخش‌های نامرتب جهش کرده و سرعت پردازش را به شدت افزایش دهد.

بخش 2) تشریح گام به گام: معماری کلاس و منطق پرش

برای پیاده سازی این سیستم، نیازمند **2** کلاس مجزا هستیم:

کلاس Node (گره): هر گره در این لیست پیوندی شامل **3** عنصر است: شناسه سند (**doc_id**)، اشاره گر به گره بعدی (**next_node**) و اشاره گر پرشی (**skip_node**).

کلاس SkipPostingList (مدیریت لیست):

فاز سازنده (Constructor): آرایه مرتب شده را دریافت کرده و ابتدا **1** لیست پیوندی استاندارد می سازد. سپس با محاسبه فاصله پرش معادل با ریشه دوم طول آرایه (بدون در نظر گرفتن اعشار)، گره ها را در فواصل مشخص شده با استفاده از **skip_node** به یکدیگر متصل می کند.

فاز تقاطع (Intersection): الگوریتم با استفاده از **2** اشاره گر، روی **2** شیء از این کلاس حرکت می کند. منطق اصلی این است که پیش از هر حرکت خطی، سیستم بررسی می کند که آیا در گره فعلی مسیر پرشی وجود دارد؟ و اگر وجود دارد، آیا مقصد آن پرش، از شناسه سند لیست مقابل کوچک تر یا مساوی است؟ در صورت تأیید، جهش انجام می پذیرد.

بخش 3) پیاده سازی عملیاتی

مطابق با اصول مهندسی نرم افزار، تمامی نام گذاری ها، متغیرها و مستندات داخل کد به زبان انگلیسی پیاده سازی شده اند:

```
skip_list.py

import math

class Node:
    def __init__(self, doc_id):
        self.doc_id = doc_id
        self.next_node = None
        self.skip_node = None

    def has_skip(self):
        return self.skip_node is not None

class SkipPostingList:
    def __init__(self, doc_ids):
        self.head = None
        self.length = len(doc_ids)

        if self.length == 0:
            return

        self.head = Node(doc_ids[0])
        current_node = self.head

        # Array to hold node references for O(1) skip assignment
        node_references = [self.head]

        # Constructing the base linked list
        for i in range(1, self.length):
            new_node = Node(doc_ids[i])
            current_node.next_node = new_node
            current_node = new_node
            node_references.append(current_node)

        # Calculating optimal skip gap: floor(sqrt(P))
        skip_gap = int(math.sqrt(self.length))

        # Assigning skip pointers dynamically
        if skip_gap > 1:
            for i in range(0, self.length - skip_gap, skip_gap):
                node_references[i].skip_node = node_references[i + skip_gap]

    def intersect_with_skips(self, other_list):
        intersection_result = []
        ptr1 = self.head
        ptr2 = other_list.head

        while ptr1 is not None and ptr2 is not None:
            if ptr1.doc_id == ptr2.doc_id:
                intersection_result.append(ptr1.doc_id)
                ptr1 = ptr1.next_node
                ptr2 = ptr2.next_node
```

```
elif ptr1.doc_id < ptr2.doc_id:
    # Check condition for jumping in list 1
    if ptr1.has_skip() and ptr1.skip_node.doc_id <= ptr2.doc_id:
        ptr1 = ptr1.skip_node
    else:
        ptr1 = ptr1.next_node

else:
    # Check condition for jumping in list 2
    if ptr2.has_skip() and ptr2.skip_node.doc_id <= ptr1.doc_id:
        ptr2 = ptr2.skip_node
    else:
        ptr2 = ptr2.next_node

return intersection_result

# Example usage for verification:
# list1 = SkipPostingList([2, 4, 8, 16, 32, 64, 128, 256, 512])
# list2 = SkipPostingList([1, 2, 3, 8, 11, 16, 25, 32, 50, 100, 128])
# print(list1.intersect_with_skips(list2))
```

بخش 4) تحلیل پیچیدگی و کارایی (Complexity Analysis)

زمان ساخت (Initialization Time): پیمایش اولیه برای ساخت لیست و پیمایش ثانویه برای اتصال نشانگرهای پرشی، هر 2 در زمان متناسب با طول لیست انجام می‌شوند. بنابراین پیچیدگی زمانی سازنده از مرتبه خطی $O(P)$ است و به عنوان 1 پیش‌پردازش آفلاین (**Offline Processing**)، بسیار بهینه ارزیابی می‌گردد.

زمان تقاطع (Intersection Time): اگر 2 لیست دارای طول‌های N و M باشند، در بدترین حالت (بدون استفاده از پرش‌ها) زمان تقاطع از مرتبه $O(N + M)$ خواهد بود. اما در سناریوهای معمول موتور جستجو (به ویژه تقاطع 1 کلمه پرتکرار با 1 کلمه کم‌تکرار)، استفاده از نشانگرهای پرشی تعداد مقایسه‌ها را به شدت کاهش داده و پیچیدگی را به صورت تقریبی به مجاورت مرتبه $O(\sqrt{N} + \sqrt{M})$ می‌رساند که باعث آزادسازی چشمگیر چرخه‌های پردازنده (**CPU Cycles**) در فاز پاسخ‌گویی آنلاین می‌شود.

پاسخ‌نامه مسئله 2.30: پایپ‌لاین ترکیبی نمایه‌ساز دوکلمه‌ای پویا

(Dynamic Biword Index Pipeline)

بخش 1) تحلیل صورت مسئله و هدف‌گذاری

در موتورهای جستجوی سازمانی (**Enterprise Search Engines**)، سرعت پاسخ‌گویی به پرس‌وجوهای متداول (مانند نام دپارتمان‌ها، اصطلاحات تخصصی و محصولات) از اهمیت بالایی برخوردار است. همان‌طور که می‌دانیم، نمایه‌های دوکلمه‌ای (**Biword Indexes**) سرعت جستجوی عبارات دقیق را به شدت افزایش می‌دهند.

چالش اصلی در این معماری، «هزینه نمایه‌سازی» (**Indexing Cost**) است. پیمایش چندین باره متون برای استخراج ابتدا تک‌کلمات و سپس توکن‌های ترکیبی، بار پردازشی سنگینی به سرورها تحمیل می‌کند. هدف از این مسئله، طراحی 1 خط لوله پردازش داده (**Data Pipeline**) است که بتواند با الگوریتم «پنجره لغزان فیلتردار» (**Filtered Sliding Window**)، تمام این عملیات را دقیقاً در 1 بار پیمایش (**Single Pass**) انجام دهد.

بخش 2) تشریح گام‌به‌گام: منطق الگوریتم یک‌مرحله‌ای (Single-Pass Algorithm)

برای آنکه بتوانیم لغت‌نامه‌های (**Vocabularies**) تک‌کلمه‌ای و دوکلمه‌ای را به صورت همزمان بسازیم و کلمات توقف را نیز نادیده بگیریم، از 1 متغیر حالت (**State Variable**) برای نگهداری «آخرین کلمه معتبر پردازش‌شده» استفاده می‌کنیم. روند کار به این شکل است:

گام اول (فیلترینگ): هر توکن از جریان ورودی خوانده می‌شود. سیستم بررسی می‌کند که آیا این توکن در مجموعه کلمات توقف (**Stop Words Set**) قرار دارد یا خیر. استفاده از ساختار داده **Set** در اینجا الزامی است تا هزینه جستجو از مرتبه **O(1)** باشد.

گام دوم (بروزرسانی نمایه تک‌کلمه‌ای): اگر توکن معتبر بود (جزو **Stop words** نبود)، مستقیماً به لغت‌نامه **Unigram** اضافه شده و فرکانس آن **1** واحد افزایش می‌یابد.

گام سوم (تولید و بروزرسانی نمایه دوکلمه‌ای): سیستم بررسی می‌کند که آیا کلمه معتبر قبلی (**Previous Valid Token**) در حافظه موقت وجود دارد؟ اگر پاسخ مثبت بود، کلمه قبلی و کلمه فعلی با **1** فاصله (**Space**) به یکدیگر متصل شده و به عنوان **Biword 1** در لغت‌نامه دوم ثبت می‌شوند.

گام چهارم (انتقال حالت): در نهایت، کلمه فعلی به عنوان کلمه معتبر قبلی برای چرخه بعدی (**Iteration**) ذخیره می‌گردد. کلمات توقف به دلیل فیلتر شدن در گام اول، هرگز به این متغیر حالت راه پیدا نمی‌کنند؛ در نتیجه **Biword**ها فقط از کلمات کلیدی و معنادار تشکیل می‌شوند.

بخش 3) پیاده‌سازی عملیاتی خط لوله

کد زیر دقیقاً با رعایت اصول مهندسی و بهینه‌سازی حافظه تدوین شده است. مطابق با استانداردهای توافق‌شده، تمامی بخش‌های کد، متغیرها و توضیحات صرفاً با کاراکترهای انگلیسی نوشته شده‌اند.

```

def process_hybrid_indexing_pipeline(token_stream, stop_words_set):
    unigram_vocabulary = {}
    biword_vocabulary = {}

    # State variable to hold the last valid token in the single pass
    prev_valid_token = None

    for raw_token in token_stream:
        current_token = raw_token.lower()

        # Step 1: Filter standard stop words efficiently O(1) lookup
        if current_token in stop_words_set:
            continue

        # Step 2: Build and update Unigram Index
        if current_token not in unigram_vocabulary:
            unigram_vocabulary[current_token] = 0

        unigram_vocabulary[current_token] += 1

        # Step 3: Build and update Biword Index dynamically
        if prev_valid_token is not None:
            biword_entry = f"{prev_valid_token} {current_token}"

            if biword_entry not in biword_vocabulary:
                biword_vocabulary[biword_entry] = 0

            biword_vocabulary[biword_entry] += 1

        # Step 4: Update state for the next iteration
        prev_valid_token = current_token

    return unigram_vocabulary, biword_vocabulary

# Execution setup for autograder test cases
sample_stream = [
    "The",
    "operating",
    "system",
    "is",
    "very",
    "fast",
    "and",
    "secure"
]

standard_stop_words = {
    "the",
    "is",
    "very",

```

```
"and",
"a",
"of",
"in"
}

unigram_result, biword_result = process_hybrid_indexing_pipeline(
    sample_stream,
    standard_stop_words
)

print("Unigram Index:")
for term, freq in unigram_result.items():
    print(f"[{term}]: {freq}")

print("\nBiword Index:")
for biword, freq in biword_result.items():
    print(f"[{biword}]: {freq}")
```

بخش 4) تحلیل کارایی مهندسی (Complexity & Architectural Analysis)

این پیاده‌سازی به دلیل بهره‌گیری از تکنیک **Single-Pass**، دارای ویژگی‌های عملکردی زیر است:

پیچیدگی زمانی (Time Complexity): الگوریتم تنها 1 بار بر روی جریان توکن‌ها که دارای طول **N** است پیمایش می‌کند. تمامی عملیات‌های درونی (بررسی وجود در **Set** و درج در دیکشنری‌های مبتنی بر هش‌تبل) با پیچیدگی زمانی ثابت **O(1)** انجام می‌شوند. بنابراین پیچیدگی زمانی کل این خط لوله دقیقاً **O(N)** است که برای پردازش جریانی (**Stream Processing**) و نمایه‌سازی بلادرنگ کاملاً بهینه است.

تحلیل معنایی خروجی (Semantic Correctness): با بررسی خروجی اسکرپت بالا، کلماتی مانند **is, the, very** و **and** به طور کامل حذف می‌شوند. در نتیجه، عبارت خروجی در دیکشنری دوکلمه‌ای به شکل **[operating system]** و **[system fast]** تولید می‌شود. این رفتار دقیقاً از تولید **Biword**های بی‌ارزشی مانند **[is very]** یا **[system is]** جلوگیری کرده و باعث صرفه‌جویی عظیمی در فضای حافظه (**Space Complexity**) می‌گردد.

پاسخنامه مسئله 2.31: پردازش واژگان در مقالات امنیتی و سیگنال

(Vocabulary Processing in Security and Signal Articles)

بخش 1) تحلیل صورت مسئله و چالش‌های امنیتی در پردازش متن

در سیستم‌های بازیابی اطلاعاتی که با مقالات تخصصی امنیت سایبری و لاگ‌های بدافزار سروکار دارند، مهاجمان برای فرار از سیستم‌های تشخیص امضا (**Signature-based Detection**) و الگوریتم‌های تطابق متن، از تکنیک **Homoglyph Attacks** (یا **IDN Homograph**) استفاده می‌کنند. در این تکنیک، کاراکترهای استاندارد لاتین با کاراکترهایی از الفبای دیگر (مانند سیریلیک یا یونانی) که از نظر بصری (**Visually**) کاملاً مشابه اما از نظر کد یونیکد (**Unicode Point**) کاملاً متفاوت هستند، جایگزین می‌شوند.

به عنوان مثال، کلمه **malware** با حروف استاندارد لاتین و کلمه **malware** (که در آن **a** و **e** سیریلیک هستند) برای کاربر انسانی یکسان دیده می‌شوند، اما توکن‌سازهای استاندارد آن‌ها را **2** کلمه کاملاً مجزا (**Out-of-Vocabulary**) فرض می‌کنند. هدف ما طراحی فاز نرمال‌سازی مقاومی (**Robust Normalization**) است که این توهم بصری را برای ماشین خنثی کند.

بخش 2) استراتژی مهندسی: اسکلت‌بندی یونیکد (Unicode Skeleton Folding)

استفاده از نرمال‌سازهای استاندارد مانند **NFC** یا **NFKC** به تنهایی این مشکل را حل نمی‌کند، زیرا این استانداردها برای حل مشکل کاراکترهای ترکیبی (مثل آواها) و فاصله‌ها طراحی شده‌اند، نه نگاشت الفباهای مختلف به یکدیگر.

برای مقاوم‌سازی سیستم در برابر **Homoglyph**، باید از استاندارد امنیتی کنسرسیوم یونیکد یعنی **UTS #39 (Unicode Security Mechanisms)** بهره برد. این استاندارد مفهومی به نام «**Confusables**» (کاراکترهای گیج‌کننده) را معرفی می‌کند.

استراتژی بهینه، اجرای تکنیک **Skeleton Mapping** است:

سیستم پیش از نمایه‌سازی (**Indexing**)، هر کاراکتر ورودی را به پایه و ساختار اصلی آن (اسکلت) نگاشت می‌کند. در این روش، چه حرف **a** لاتین، چه **a** سیریلیک و چه **а** یونانی وارد سیستم شوند، همگی به **1** توکن استاندارد معادل (همان **a** لاتین) در کلاس هم‌ارزی تقلیل می‌یابند.

بخش 3) معماری پایپ‌لاین پردازشی

برای اعمال این استراتژی، خط لوله (**Pipeline**) نرمال‌سازی کلمات در موتور جستجوی شما باید مراحل زیر را دقیقاً با این ترتیب طی کند:

1. Case Folding (کوچک‌سازی): تبدیل تمام توکن‌ها به حروف کوچک، زیرا تفاوت‌های نگارشی در حروف بزرگ و کوچک می‌تواند جداول **Confusables** را دور بزند.

2. Skeleton Mapping (نگاشت اسکلتی): عبور دادن کاراکترها از فرهنگ لغت تطبیق بصری (**Visual Matching Dictionary**) برای تبدیل حروف سیریلیک / یونانی / مبهم به معادل لاتین آن‌ها.

3. Diacritic Removal (حذف آواها و علائم): استفاده از استاندارد **NFKD** برای جدا کردن کاراکترهای ترکیبی و حذف آن‌ها (**ASCII Folding**)، تا کلماتی که با تغییرات آوایی مبهم شده‌اند نیز به ریشه اصلی بازگردند.

بخش 4) پیاده‌سازی عملیاتی

بر اساس استانداردهای توسعه و برای حفظ یکپارچگی، کدهای این بخش شامل نام متغیرها و مستندات، به صورت کاملاً انگلیسی نوشته شده‌اند:

```
security_normalizer.py

import unicodedata

# A simplified subset of the UTS #39 Confusables mapping dictionary
CONFUSABLES_MAP = {
    '\u0430': 'a', # Cyrillic small letter a
    '\u03B1': 'a', # Greek small letter alpha
    '\u0435': 'e', # Cyrillic small letter e
    '\u0501': 'd', # Cyrillic small letter komi de
    '\u043E': 'o', # Cyrillic small letter o
    '\u0441': 'c', # Cyrillic small letter es
    '\u0555': 'o', # Armenian small letter oh
}

def robust_security_normalizer(raw_token):
    if not raw_token:
        return ""

    normalized_chars = []
    token_lower = raw_token.lower()

    for char in token_lower:
        if char in CONFUSABLES_MAP:
            normalized_chars.append(CONFUSABLES_MAP[char])
        else:
            normalized_chars.append(char)

    skeleton_token = "".join(normalized_chars)
```

```
final_token = "".join(
    c for c in unicodedata.normalize('NFKD', skeleton_token)
    if not unicodedata.combining(c)
)

return final_token

# Execution logic for detecting obfuscated security terms
standard_term = "malware"
obfuscated_term = "malware" # Uses Cyrillic 'a' and 'e'

print(f"Standard processed   : {robust_security_normalizer(standard_term)}")
print(f"Obfuscated processed : {robust_security_normalizer(obfuscated_term)}")

is_match = robust_security_normalizer(standard_term) == robust_security_normalizer(obfuscated_term)
print(f"Equivalence match    : {is_match}")
```

بخش 5) تحلیل کارایی و چالش‌های مهندسی (Trade-offs)

پیچیدگی زمانی: جستجو در جدول هش (**Hash Table**) برای یافتن کاراکترهای گیج‌کننده دارای پیچیدگی زمانی ثابت **O(1)** است. بنابراین، نرمال‌سازی هر کلمه در زمان **O(K)** انجام می‌شود (که در آن **K** طول کلمه است). این امر بهینه‌سازی پردازشی فاز **Indexing** را تضمین می‌کند.

چالش مثبت کاذب (False Positives): اگرچه این پایپ‌لاین در مقالات امنیت سایبری برای کشف کلمات مخرب فوق‌العاده کارآمد است، اما استفاده افراطی از آن در متون عادی می‌تواند منجر به ادغام کلماتی شود که در زبان‌های دیگر معنای متفاوتی دارند (تقاطع کاذب). بنابراین، این فیلتر معمولاً بر روی فیلدهای خاصی از نمایه (مانند فیلد **code_snippets** یا **IOCs**) اعمال می‌گردد.

پاسخ‌نامه مسئله 2.32: بهینه‌سازی مرتب‌سازی در ساخت نمایه (Index Construction Sorting)

بخش 1) تحلیل مسئله و چالش مرتب‌سازی نمایه‌ها

در معماری موتورهای جستجو و فاز ساخت نمایه (**Index Construction**)، پس از مرحله توکن‌گذاری، با حجم عظیمی از جفت‌های (**Term, DocID**) روبرو هستیم که باید ابتدا بر اساس **Term** (لغت) و سپس بر اساس **DocID** (شناسه سند) مرتب شوند تا لیست‌های پستینگ یکپارچه شکل بگیرند.

از آنجا که اسناد معمولاً به صورت ترتیبی پردازش می‌شوند، شناسه‌های اسناد برای هر لغت غالباً از پیش مرتب هستند و تنها به دلیل پردازش‌های موازی (**Concurrency**) یا خطاهای جزئی در بافرهای حافظه، تعداد کمی جابجایی (**Inversion**) محلی در لیست نهایی رخ می‌دهد. در نظریه الگوریتم‌ها، به چنین مجموعه‌ای «**داده‌های تقریباً مرتب‌شده**» (**Nearly Sorted Data**) می‌گویند.

بخش 2) تحلیل الگوریتم Cocktail Shaker Sort

الگوریتم **Cocktail Shaker Sort** (که با نام‌های **Bidirectional Bubble Sort** یا مرتب‌سازی لرزان نیز شناخته می‌شود)، **1** نسخه بهینه‌سازی‌شده از مرتب‌سازی حبابی است. در حالی که **Bubble Sort** در هر پاس تنها از چپ به راست حرکت می‌کند، این الگوریتم به صورت رفت و برگشتی (پاندولی) عمل می‌کند.

برای درک دلیل برتری این الگوریتم، باید **2** مفهوم کلاسیک در آرایه‌ها را بشناسیم:

خرگوش‌ها (Rabbits): عناصر بزرگی که در ابتدای آرایه قرار دارند. این عناصر در الگوریتم حبابی استاندارد، به سرعت و در همان پاس‌های اول به انتهای آرایه (مکان صحیح خود) منتقل می‌شوند.

لاکپشت‌ها (Turtles): عناصر کوچکی که در انتهای آرایه قرار دارند. در الگوریتم حبابی استاندارد، **1** «لاکپشت» در هر پاس تنها **1** خانه به سمت چپ حرکت می‌کند؛ بنابراین اگر لاکپشتی در انتهای **1** آرایه طولانی باشد، باعث می‌شود الگوریتم به بدترین حالت خود یعنی $O(N^2)$ میل کند.

الگوریتم **Cocktail Shaker Sort** با اضافه کردن **1** پیمایش معکوس (از راست به چپ)، مشکل لاکپشت‌ها را به طور کامل حل می‌کند و آن‌ها را در **1** حرکت سریع به ابتدای آرایه پرتاب می‌نماید.

بخش 3) اثبات ریاضی و پیچیدگی زمانی (Big O Analysis)

برای اثبات برتری این الگوریتم در لیست‌های تقریباً مرتب، باید تعداد وارونگی‌ها (**Inversions**) را k در نظر بگیریم. (وارونگی به معنای تعداد جفت‌هایی است که در مکان اشتباه نسبت به هم قرار دارند).

پیچیدگی بهترین حالت (Best Case): اگر داده‌ها کاملاً مرتب باشند، الگوریتم با **1** پیمایش و بررسی پرچم (**Flag**)، در مرتبه زمانی $O(N)$ متوقف می‌شود.

مقایسه با الگوریتم‌های استاندارد: الگوریتم‌های قدرتمندی نظیر **Merge Sort** یا **Quick Sort** (نسخه استاندارد)، بدون توجه به میزان مرتب بودن داده‌ها، همواره با پیچیدگی زمانی $O(N \log N)$ اجرا می‌شوند و سربار حافظه‌ای یا بازگشتی (**Recursive Overhead**) بالایی دارند.

برتری در سناریوی مفروض: در لیست‌های پوزیشنی که تقریباً مرتب هستند، مقدار k بسیار کوچک است ($k \ll N$). الگوریتم **Cocktail Shaker** با بهره‌گیری از پیمایش‌های دوطرفه و حل مشکل لاکپشت‌ها، تمام وارونگی‌های محلی را در چند پاس کوتاه برطرف کرده و به دلیل مکانیزم خروج زودهنگام (**Early Exit**)، پیچیدگی زمانی آن به مجاورت مجانب زیر تقلیل می‌یابد:

$$O(N + k) \approx O(N)$$

بنابراین در سناریوی ساخت نمایه با داده‌های تقریباً مرتب، این الگوریتم به شکل چشمگیری سریع‌تر از $O(N \log N)$ عمل می‌کند و بدون نیاز به حافظه اضافی (**In-place**)، پرفورمنس را به حداکثر می‌رساند.

بخش 4) پیاده‌سازی عملیاتی

کد زیر پیاده‌سازی این الگوریتم در پایتون برای مرتب‌سازی جفت‌های (**Term, DocID**) است. بر اساس استانداردهای مهندسی نرم‌افزار، تمامی نام‌گذاری‌ها، کامنت‌ها و هشتک‌ها در این بلاک منحصراً به زبان انگلیسی تدوین شده‌اند تا از هرگونه تداخل در کامپایلر جلوگیری شود. در پایتون، مقایسه **Tuple**‌ها به صورت خودکار ابتدا روی عنصر اول (**Term**) و در صورت برابری روی عنصر دوم (**DocID**) انجام می‌شود.

```
cocktail_shaker_sort.py

def cocktail_shaker_sort(posting_pairs):
    # posting_pairs is a list of tuples: (term, doc_id)
    # Python naturally compares tuples lexicographically:
    # First by term, then by doc_id

    n = len(posting_pairs)
    swapped = True
    start_idx = 0
    end_idx = n - 1

    while swapped:
        # Reset swapped flag for the forward pass
        swapped = False

        # Forward pass: Left to Right (Moves 'Rabbits' to the end)
        for i in range(start_idx, end_idx):
```

```
        if posting_pairs[i] > posting_pairs[i + 1]:
            # Swap elements if they are in the wrong order
            posting_pairs[i], posting_pairs[i + 1] = posting_pairs[i + 1], posting_pairs[i]
            swapped = True

# Early exit: If no swaps occurred, the array is perfectly sorted
if not swapped:
    break

# Reset swapped flag for the backward pass
swapped = False

# The last element is already in place, decrease end index
end_idx -= 1

# Backward pass: Right to Left (Resolves 'Turtles' quickly)
for i in range(end_idx - 1, start_idx - 1, -1):
    if posting_pairs[i] > posting_pairs[i + 1]:
        posting_pairs[i], posting_pairs[i + 1] = posting_pairs[i + 1], posting_pairs[i]
        swapped = True

# The first element is already in place, increase start index
start_idx += 1

return posting_pairs

# Test case array simulating nearly sorted (Term, DocID) pairs
sample_postings = [
    ("algorithm", 10),
    ("algorithm", 15),
    ("algorithm", 12), # Local inversion (Turtle)
    ("algorithm", 20),
    ("design", 5),
    ("design", 8),
    ("design", 7)      # Local inversion
]

sorted_postings = cocktail_shaker_sort(sample_postings)
# Expected logic: will group terms alphabetically, and doc_ids ascendingly
```

پاسخ‌نامه مسئله 2.33: زمان‌بندی پردازش پرس‌وجوهای عبارتی (Phrase Query Processing Scheduling)

بخش 1) تحلیل معماری و تفکیک مسئله

در 1 موتور جستجوی چندنخی (**Multi-threaded**) که تحت بار ترافیکی بالایی قرار دارد، بهینه‌سازی زمان پاسخگویی (**Response Time**) نیازمند حل مسئله در 2 لایه پردازشی متمایز است:

- لایه خرد (Micro-level):** برای 1 پرس‌وجوی عبارتی مشخص (مثلاً $W_1W_2W_3W_4$)، تقاطع‌ها باید با چه ترتیبی انجام شوند تا کمترین پردازش صورت گیرد؟
- لایه کلان (Macro-level):** صف پرس‌وجوهای کاربران باید با چه استراتژی‌ای به رزوه‌های پردازنده (**Threads**) تخصیص یابد؟

بخش 2) بهینه‌سازی لایه خرد: ترتیب تقاطع‌ها (Intersection Ordering)

برای 1 پرس‌وجوی عبارتی طولانی، کلمات باید لزوماً به صورت مجاور (**Adjacent**) تقاطع داده شوند. این مسئله از نظر ریاضی دقیقاً معادل **مسئله ضرب زنجیره‌ای ماتریس‌ها (Matrix Chain Multiplication)** است.

تخمین هزینه (Cost Function): هزینه تقاطع 2 لیست پوزیشنی متوالی با طول‌های L_i و L_j در بدترین حالت برابر با $O(L_i + L_j)$ است. خروجی این تقاطع، 1 لیست جدید است که طول آن حداکثر $\min(L_i, L_j)$ خواهد بود.

الگوریتم برنامه‌نویسی پویا (Dynamic Programming): سیستم به جای تقاطع خطی از چپ به راست، ابتدا 1 جدول **DP** تشکیل می‌دهد تا کم‌هزینه‌ترین ترتیب پرانتزگذاری را بیابد. به عنوان مثال، در عبارت

"Advanced Algorithm Design Concepts"، اگر کلمات "Algorithm" و "Design" نادرترین کلمات باشند (کوتاه‌ترین لیست‌ها)، سیستم ابتدا این 2 را تقاطع می‌دهد، سپس نتیجه را با "Advanced" و در نهایت با "Concepts" ترکیب می‌کند. این کار از پردازش بیهوده لیست‌های طولانی جلوگیری می‌کند.

بخش 3) بهینه‌سازی لایه کلان: زمان‌بندی پردازنده (CPU Scheduling)

برای به حداقل رساندن میانگین زمان پاسخگویی (Average Response Time) در تئوری سیستم‌عامل، الگوریتم Shortest Job First (SJF) یا نسخه غیرانحصاری آن، بهینه‌ترین اثبات ریاضی را دارد. در موتور جستجو، ما «زمان اجرای دقیق» را نمی‌دانیم، اما می‌توانیم آن را از روی طول لیست‌ها تخمین بزنیم.

معماری زمان‌بند (The Scheduler):

- هنگامی که 1 پرس‌وجو وارد سیستم می‌شود، پردازنده ابتدا طول لیست‌های پستینگ کلمات آن را از دیسک یا Cache می‌خواند.
 - با استفاده از الگوریتم DP (مرحله قبل)، هزینه کل پردازش این پرس‌وجو (Total CPU Burst Estimate) محاسبه می‌شود.
 - پرس‌وجو به همراه امتیاز هزینه، وارد 1 صف اولویت‌دار (Priority Queue / Min-Heap) می‌شود.
 - رزوه‌های پردازنده (Worker Threads) به محض آزاد شدن، پرس‌وجویی را از بالای Heap برمی‌دارند که کمترین هزینه محاسباتی را دارد.
- مدیریت گرسنگی (Starvation Prevention): برای جلوگیری از مسدود شدن پرس‌وجوهای سنگین (مانند عباراتی که تماماً از کلمات پرکاربرد تشکیل شده‌اند)، سیستم از مکانیزم Aging استفاده می‌کند. در این مکانیزم، هر ثانیه‌ای که 1 پرس‌وجو در صف منتظر می‌ماند، ضریبی از هزینه تخمینی آن کسر می‌شود تا در نهایت اولویت پردازش پیدا کند.

بخش 4) تحلیل پیچیدگی زمانی (Big O Analysis)

پیچیدگی زمانی این الگوریتم زمان‌بند باید به تفکیک مراحل استخراج شود:

فاز تخمین هزینه و یافتن ترتیب بهینه (DP Phase): الگوریتم برنامه‌نویسی پویا برای یافتن ترتیب بهینه تقاطع کلمات، دارای پیچیدگی زمانی $O(N^3)$ است که در آن N تعداد کلمات موجود در عبارت جستجو شده است. از آنجا که در دنیای واقعی، طول 1 پرس‌وجوی عبارتی معمولاً بسیار کوچک است (غالباً $N \leq 7$)، این هزینه محاسباتی در مقیاس پردازنده عملاً از مرتبه $O(1)$ محسوب شده و سرشار (Overhead) کاملاً ناچیزی دارد.

فاز مدیریت صف اولویت (Queue Management): درج (Insertion) 1 پرس‌وجوی جدید در صف Min-Heap نیازمند زمان $O(\log Q)$ است (که در آن Q تعداد پرس‌وجوهای فعلی منتظر در صف است). استخراج کم‌هزینه‌ترین پرس‌وجو (Extract-Min) توسط Worker Thread‌ها نیز در زمان $O(\log Q)$ انجام می‌پذیرد.

فاز بروزسانی Aging: اعمال فاکتور Aging بر روی پرس‌وجوهای منتظر، اگر با ساختارهای داده پیشرفته (مانند درخت‌های متوازن یا Fibonacci Heap) ترکیب شود، پیچیدگی استهلاکی بهینه‌ای خواهد داشت، اما در 1 آرایه ساده $O(Q)$ است.

نتیجه‌گیری ساختاری: پیچیدگی کلی زمان‌بند به ازای هر پرس‌وجو برابر با $O(N^3 + \log Q)$ خواهد بود. این معماری تضمین می‌کند که پرس‌وجوهای سبک به صورت بلادرنگ پاسخ داده شوند و پرس‌وجوهای سنگین باعث ایجاد قفل در منابع سرور (Thread Starvation) نگردند.

